Laboratorul 1
Introducere în pthreads



Utilitate APD

- multithreading:
  - mobile (Android)
  - backend - sunt multe întrebări de interviu legate de multithreading
- programare distribuită - servere, clustere (Kafka, Cassandra)

De ce multithreading?

- performanță
- scalabilitate
- eficiență
- reducerea costurilor

Thread vs. proces

Thread:

- fiecare thread are stiva sa proprie și propriul său instruction pointer

Proces:

- fiecare proces are propriul său spațiu virtual de memorie (un proces nu poate accesa spațiul de memorie al altui proces)
- în cadrul unui proces, pot rula mai multe thread-uri, care partajează același spațiu de memorie al procesului în cadrul căruia rulează

Multithreading la APD

La APD vom lucra cu multithreading în:

- C - pthreads - POSIX (3 laboratoare + o temă)
- Java (4 laboratoare + o temă)

La ASC veți mai lucra cu thread-uri în Python și în CUDA și la APP cu OpenMP în C.

pthreads

- reprezintă o libr… o bibliotecă în C/C++
- includerea acesteia:

```
#include <pthread.h>
```

- compilarea:

```
gcc -o program program.c -lpthread

./program
```

Crearea unui thread

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void
*(*start_routine) (void *), void *arg);
```

- thread - thread-ul pe care vrem să-l pornim
- attr - atributele unui thread (NULL - atribute default)
- start_routine - funcția pe care să o execute thread-ul
- arg - parametrul trimis la funcția executată de thread (dacă vrem să trimitem mai mulți parametri, îi împachetăm într-un struct)

Notă: un thread poate crea alte thread-uri, nu există ierarhii

Terminarea unui thread

```
void *f(void *arg) {

// do stuff

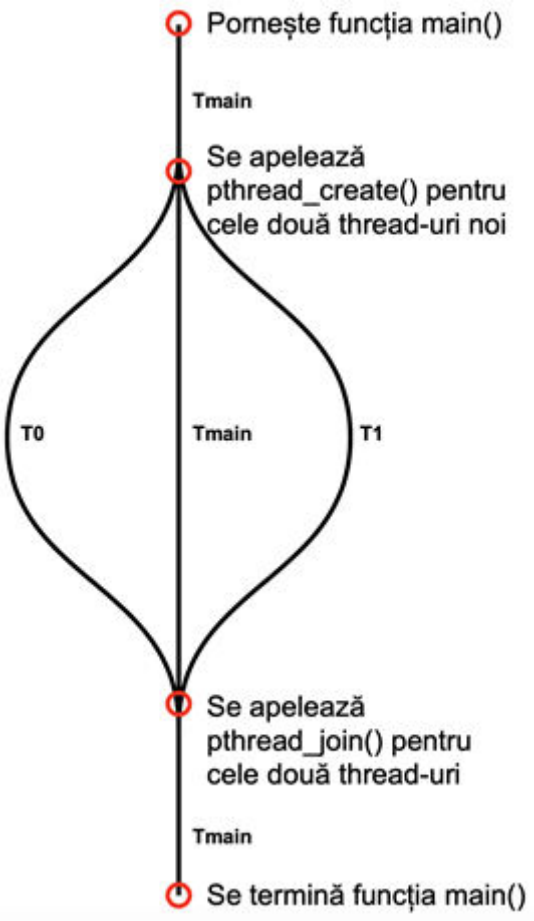
    pthread_exit(NULL); // termină un thread - mereu apelat la finalul unei funcții executate de thread, dacă nu întoarcem un rezultat în funcție

}
```

Terminarea unui thread

```
int pthread_join(pthread_t thread, void **retval);
```

- se **așteaptă** terminarea unui thread - pthread_join **nu termină** execuția unui thread
- thread - thread-ul pe care îl așteptăm să termine
- retval - valoarea de retur a funcției executate de thread (poate fi NULL)



Crearea, rularea și terminarea thread-urilor

Cum trimitem mai mulți parametri la funcția thread-ului?

- În caz că dorim să trimitem mai mulți parametri funcției executate de threads, facem un `struct`, în care incapsulăm datele, și îl trimitem ca parametru al funcției executate de threads.
- Exemplu: https://gist.github.com/florinrm/361d0ff0b150fa77ece86cbbba47894d

Împărțirea unui array pe thread-uri

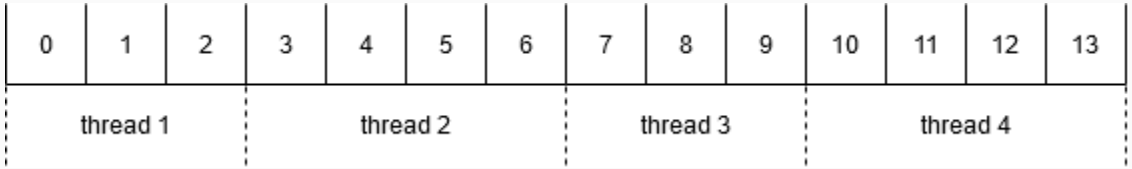
Când dorim să paralelizăm operații efectuate pe arrays, fiecărui thread îi va reveni o bucată din array, pe acea bucată executând funcția atribuită lui (thread-ului).

Formula de împărțire:

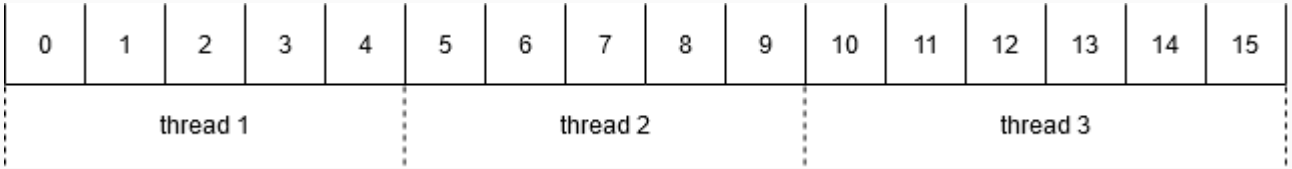
- `start_index = id * (double) n / p`
- `end_index = min(n, (id + 1) * (double) n / p))`, unde `id = id-ul thread-ului`, `n = dimensiunea array-ului`, `p = numărul de threads`

Împărțirea unui array pe thread-uri - exemplu

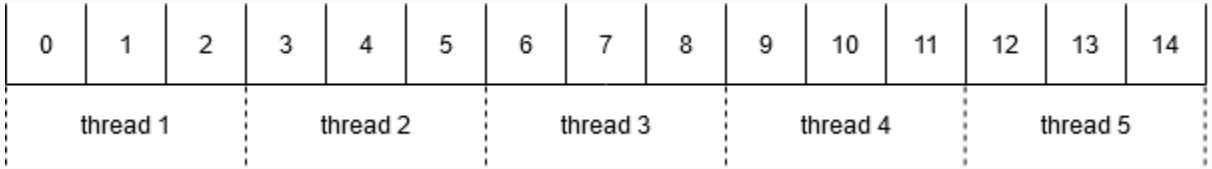
- n = 14, p = 4 => 3, 4, 3, 4 (th1: [0, 3), th2: [3, 7), th3: [7, 10), th4: [10, 14))



- n = 16, p = 3 => 5, 5, 6 (th1: [0, 5), th2: [5 -> 10), th3: [10 -> 16))



- n = 15, p = 5 => 3, 3, 3, 3, 3 ([0,3), [3, 6), [6, 9), [9, 12), [12 -> 15))



Întrebări (potențiale de interviu)

- Ce se va întâmpla dacă porniți un program cu mai multe threads decât cores?
- Care este diferența între un thread și un proces?
- Care este diferența între un thread și un procesor?
- Cine decide pe ce core va rula un anumit thread?
- Ce înseamna scalabilitate bună în programarea paralelă?
- Ce probleme pot apărea în cadrul unui program care funcționează pe mai multe threads?
- Dacă se creează două thread-uri folosind `pthread_create`, câte thread-uri vor exista în program (înainte de `pthread_join`)?

Feedback și trimiterea laboratorului

- Dați feedback legat de cum a fost la laborator aici.
- Deadline trimitere laborator: sâmbătă seara, pe moodle (este un assignment la fiecare laborator)

Resurse

https://man7.org/linux/man-pages/man3/pthread_create.3.html

https://man7.org/linux/man-pages/man3/pthread_exit.3.html

https://man7.org/linux/man-pages/man3/pthread_join.3.html

https://computing.llnl.gov/tutorials/pthreads/

Laboratorul 2
Elemente de sincronizare



Sincronizare - ce este și de ce

- în programarea paralelă, există situații când thread-urile accesează niște resurse comune (partajate) simultan (de regulă prin operații de read-write)
- situație exemplu: avem o variabilă globală `int a = 0` și două threads, care execută `a += 2`; este de așteptat ca rezultatul final să fie `a = 4`, dar nu mereu este așa

Incrementarea a += 2, cu doi threads

a = 0 (inițial)

T0	T1
`load(a, eax0) -> 0`	`load(a, eax1) -> 0`
`eax0 = eax0 + 2 -> 2`	`eax1 = eax1 + 2 -> 2`
`write(a, eax0) -> 2`	`write(a, eax1) -> 2`

Remember lab 1:
un thread are propria sa stivă, deci propriile registre

a = 4 sau a = 2

race condition

Aici nu avem operații atomice (vom discuta despre ele la laboratorul 5)

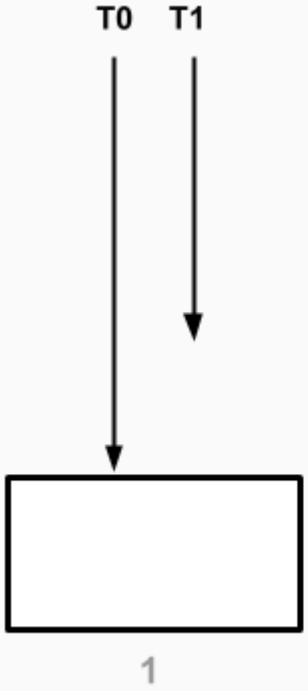
Primitive de sincronizare

- mutex
- barieră
- semafor (o să vorbim la laboratorul 4, există implementare de semafor în pthreads)
- variabile condiție (detalii la ASC și la SO)

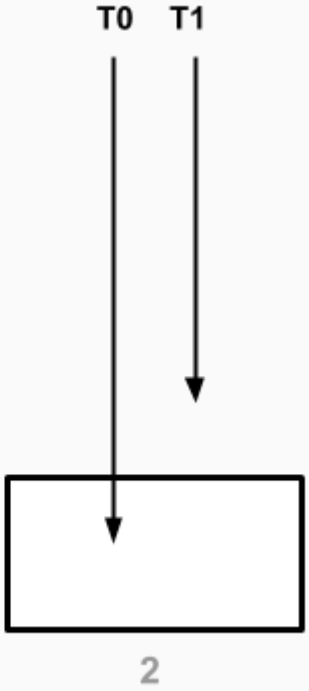
Lectură suplimentară: https://ocw.cs.pub.ro/courses/so/laboratoare/laborator-08

Mutex

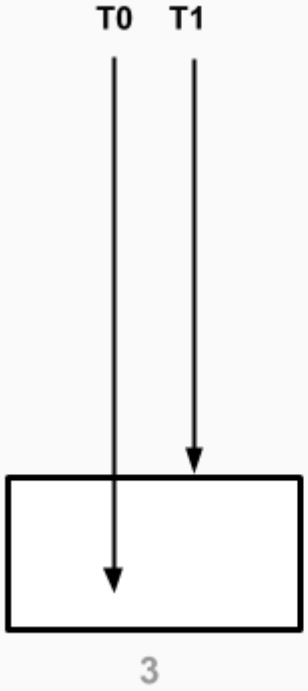
- un mutex (mutual exclusion) este folosit pentru a delimita și pentru a proteja o zonă critică, unde au loc, de regulă, operații de citire și de scriere
- un singur thread intră în zona critică (o rezervă pentru el - lock), unde se execută instrucțiuni, iar celelalte thread-uri așteaptă ca thread-ul curent să termine de executat instrucțiunile din zona critică
- după ce thread-ul curent termină de executat instrucțiuni în zona critică, aceasta o eliberează (unlock) și următorul thread urmează aceiași pași



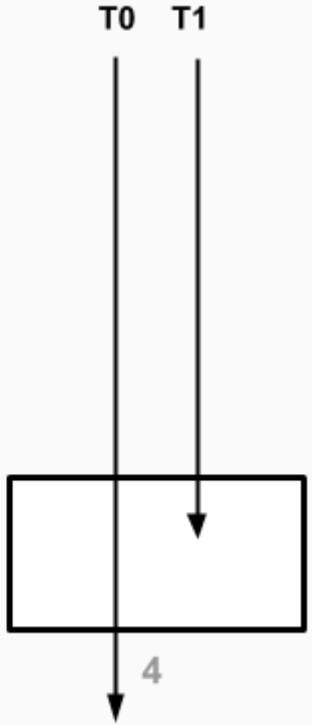
T0 face lock



T0 e în zona critică



T0 e în zona critică, T1 așteaptă



T0 iese din zona critică și face unlock, T1 intră în zona critică și face lock

Funcții

- crearea unui mutex: `int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);`
- lock pe mutex: `int pthread_mutex_lock(pthread_mutex_t *mutex);`
- unlock pe mutex: `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
- distrugerea unui mutex: `int pthread_mutex_destroy(pthread_mutex_t *mutex);`

Folosire mutex

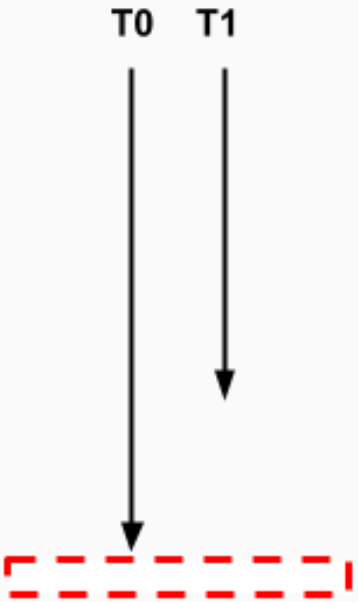
```
pthread_mutex_t mutex; // variabila globala
pthread_mutex_init(&mutex, NULL); // in main, inainte de pthread_create

void* f(void* arg) {
    pthread_mutex_lock(&mutex);
    [zona critica]
    pthread_mutex_unlock(&mutex);
}

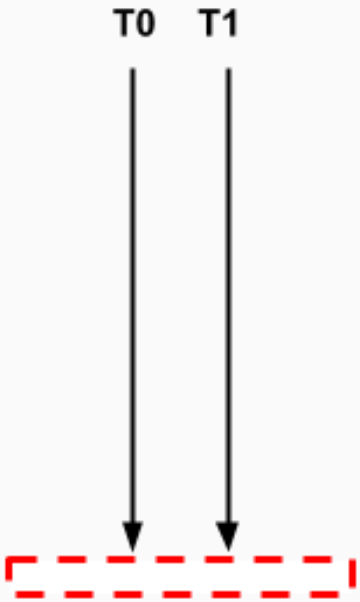
pthread_mutex_destroy(&mutex); // in main, dupa pthread_join
```

Barieră

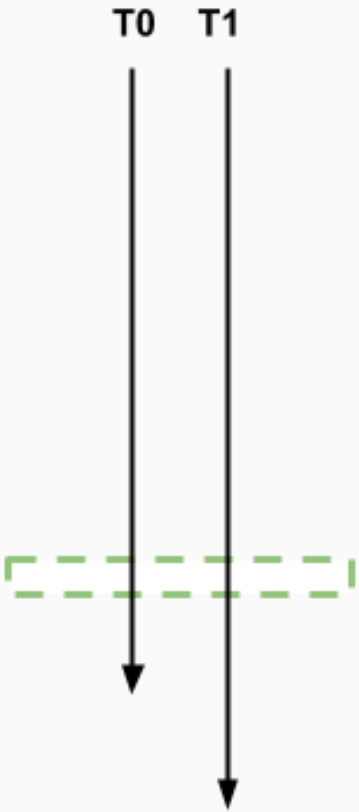
- bariera este folosită atunci când dorim să sincronizăm thread-urile încât să ajungă (să se sincronizeze) în același punct
- mai concret, ea asigură faptul că niciun thread, gestionat de barieră, nu trece mai departe de zona în care aceasta este amplasată decât atunci când toate thread-urile gestionate de barieră ajung în aceeași zonă



Bariera blocată, T0 e la barieră, se așteaptă T1



Bariera blocată, T1 ajunge la barieră



Bariera deblocată, thread-urile își continuă execuția

Funcții

- crearea unei bariere: `int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);`
- așteptarea thread-urilor la barieră: `int pthread_barrier_wait(pthread_barrier_t *barrier);`
- distrugerea unei bariere: `int pthread_barrier_destroy(pthread_barrier_t *barrier);`

Folosire barieră

```
pthread_barrier_t barrier; // variabila globala
pthread_barrier_init(&barrier, NULL, 2); // in main, inainte de pthread_create

void* f(void* arg) {
    [...]
    pthread_barrier_wait(&barrier);
    [...]
}

pthread_barrier_destroy(&barrier); // in main, dupa pthread_join
```

Înmulțirea matricelor - versiunea naivă

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        for (k = 0; k < N; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

Înmulțirea matricelor - Strassen

- de paralelizat folosind 7 threads, câte un thread pentru fiecare matrice M

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_2 = (A_{21} + A_{22})B_{11}$$
$$M_3 = A_{11}(B_{12} - B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{12})B_{22}$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{22} = M_1 - M_2 + M3 + M_6$$

Resurse utile

- Scripturi pentru testat cod paralelizat la laborator: https://gist.github.com/florinrm/c5edbff84b8a7f94423e2065b870235e

Întrebări (potențiale de interviu)

- Cum se poate delimita o zonă critică?
- Ce face o barieră?
- Care este complexitatea înmulțirii a două matrici în paralel?
- Ce poate cauza un comportament nedeterminist a unui program paralel?
- Dați un exemplu de caz de race condition și cum se ajunge la race condition.

Laboratorul 3
Algoritmi paraleli - sortări

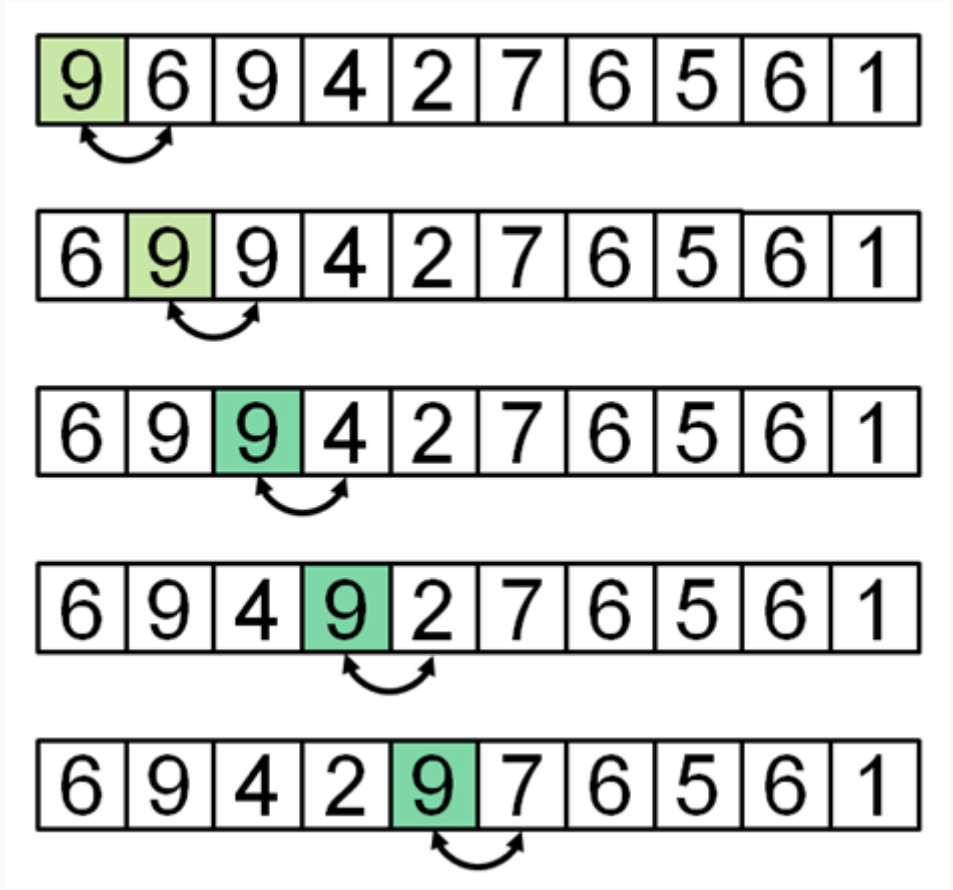


Bubble Sort

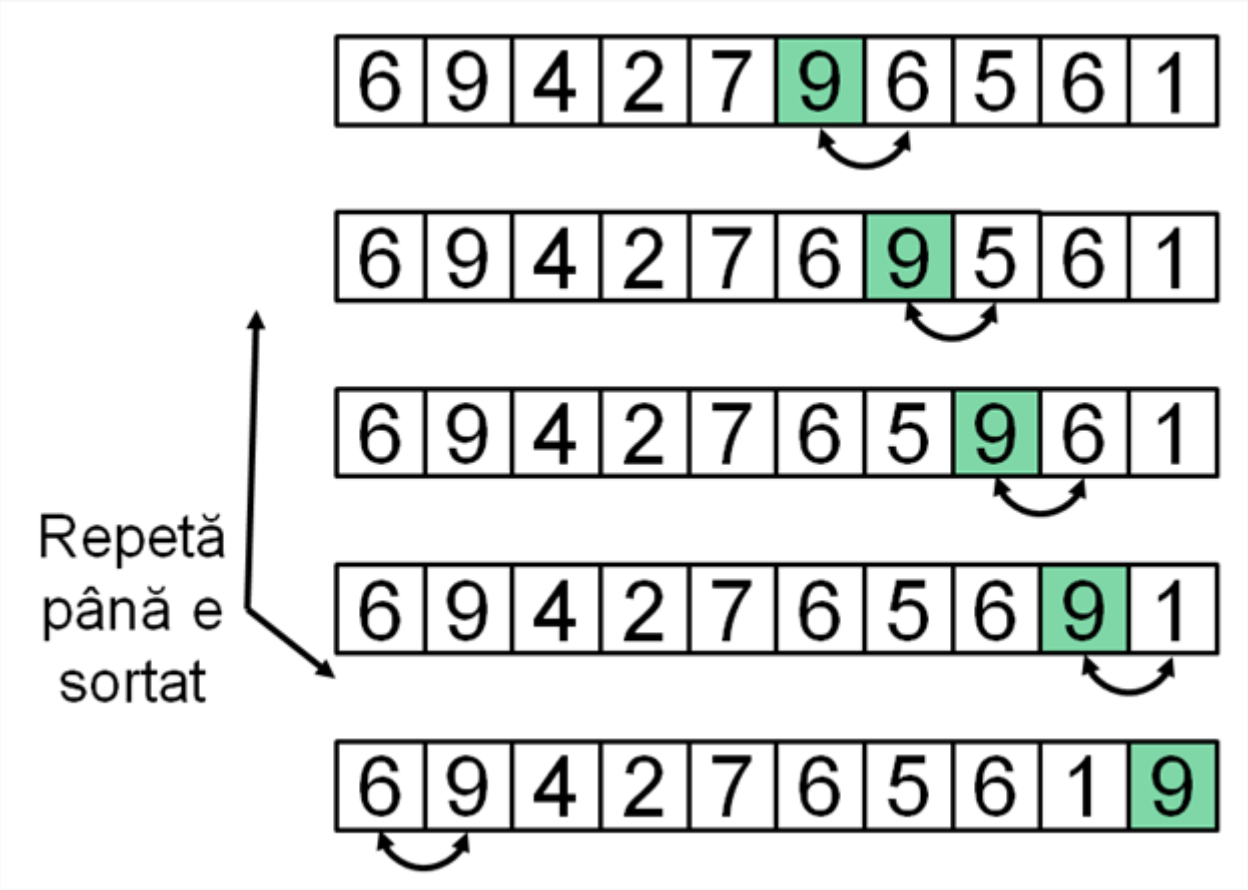
```
function bubbleSort(list) {
  sorted = false;
  while (!sorted) {
    sorted = true;
    for (var i = 0; i < list.length - 1; i++) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

Complexitate temporală: $O(N^2)$, având maxim N parcurgeri ale șirului, cu dimensiune N

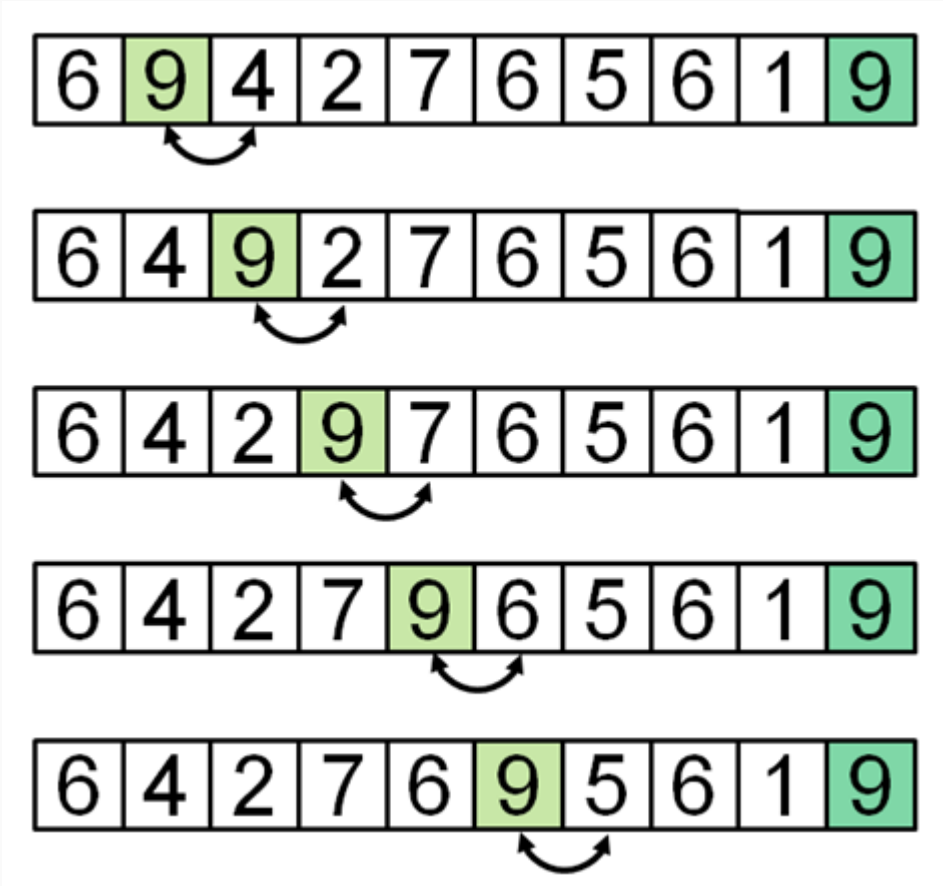
Bubble Sort - rulare



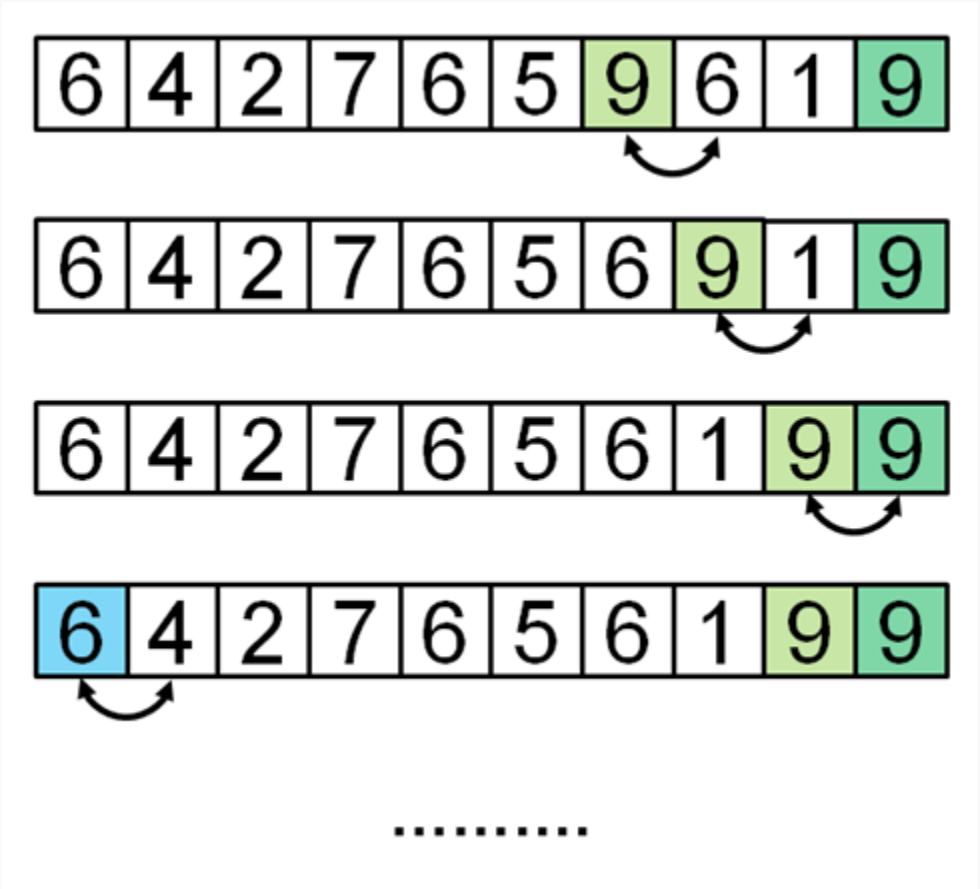
Bubble Sort - rulare



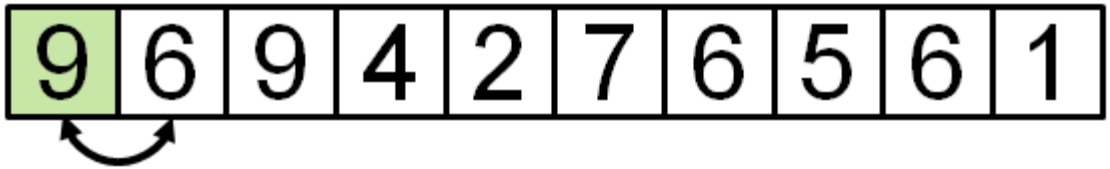
Bubble Sort - rulare



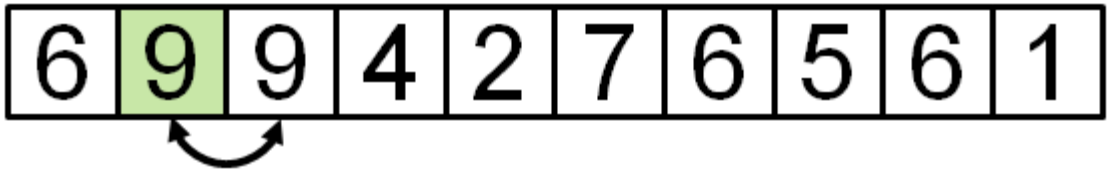
Bubble Sort - rulare



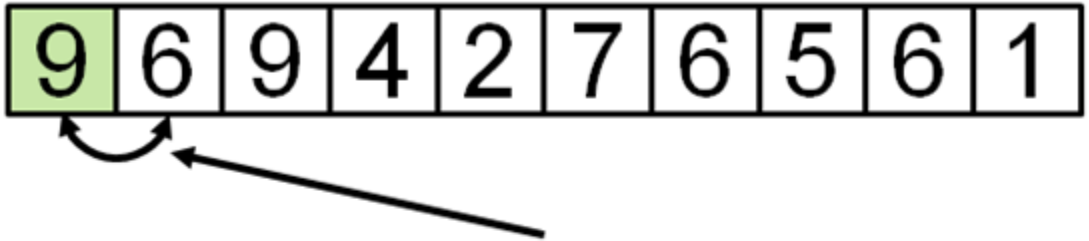
Bubble Sort - tentativă de paralelizare



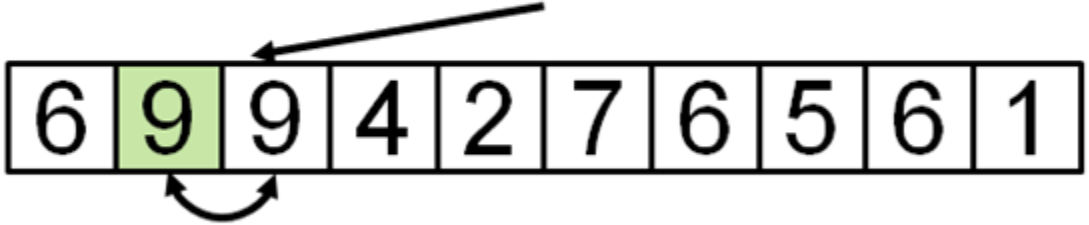
Cum paralelizăm?



Bubble Sort - tentativă de paralelizare



Aceste două operații (și toate perechile similare)
NU pot fi executate în același timp



Solution hint: Nu e necesară execuția operațiilor într-o anumită ordine

Odd-Even Transposition Sort - Bubble Sort paralelizat

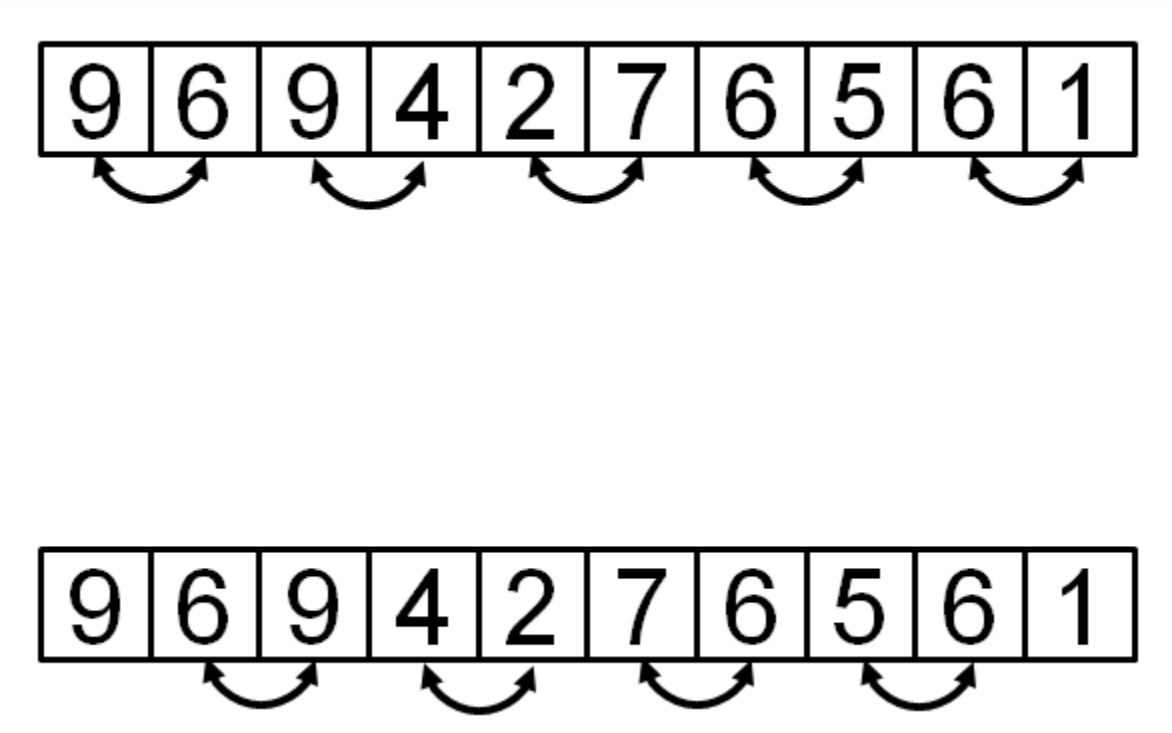
```
function oddEvenSort(list) {
  for (var k = 0; k < list.length; k++) {
    for (i = 0; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
    for (i = 1; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

Această versiune paralelizată de Bubble Sort funcționează în două faze:

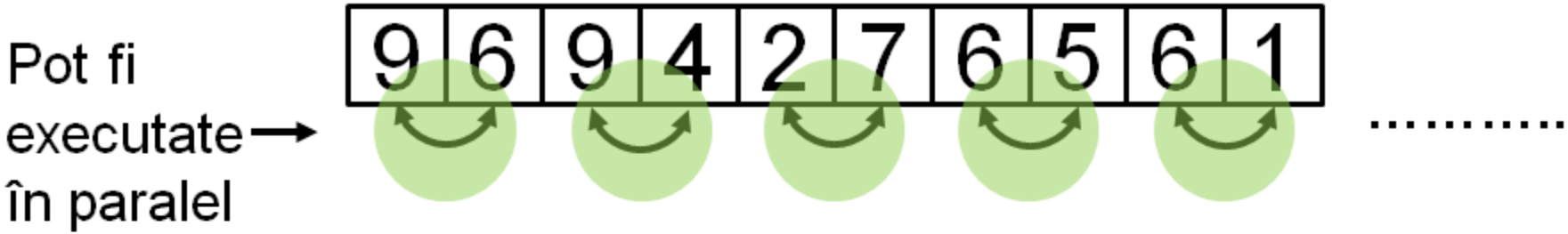
- faza pară - elementele de pe pozițiile **pare** sunt comparate cu vecinii de pe poziția următoare (i cu i + 1) și eventual interschimbate
- faza impară - ea începe după terminarea fazei impare. Elementele de pe pozițiile **impare** sunt comparate cu vecinii de pe poziția următoare (i cu i + 1) și eventual interschimbate

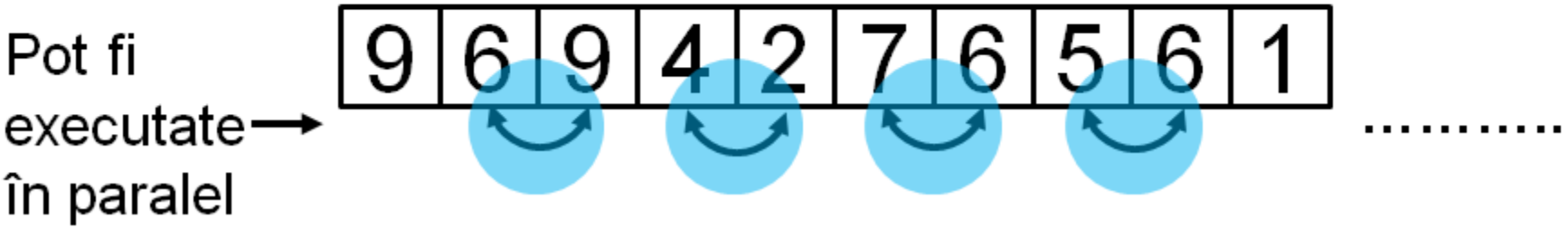
Complexitate temporală: $O(N^2 / P)$, P = numărul de thread-uri

Odd-Even Transposition Sort - mod de funcționare

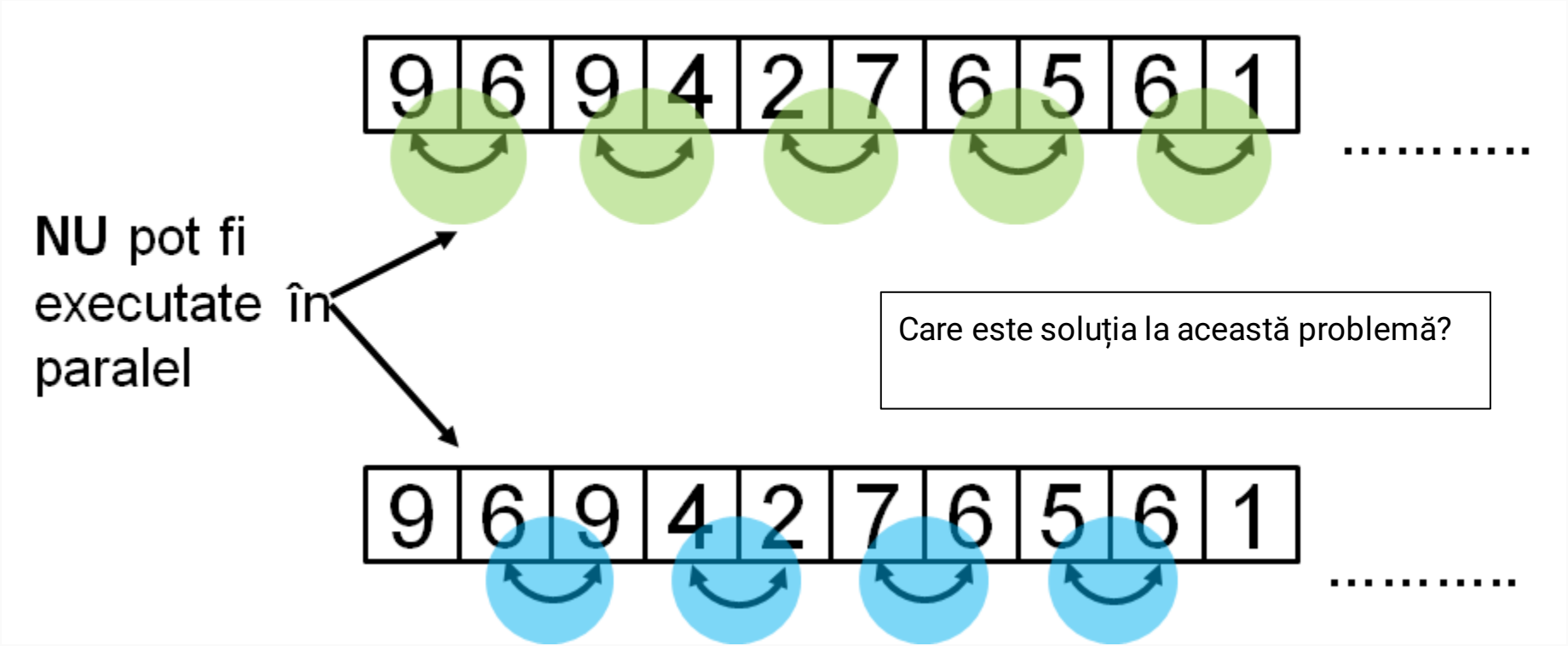


Odd-Even Transposition Sort - mod de funcționare paralelă

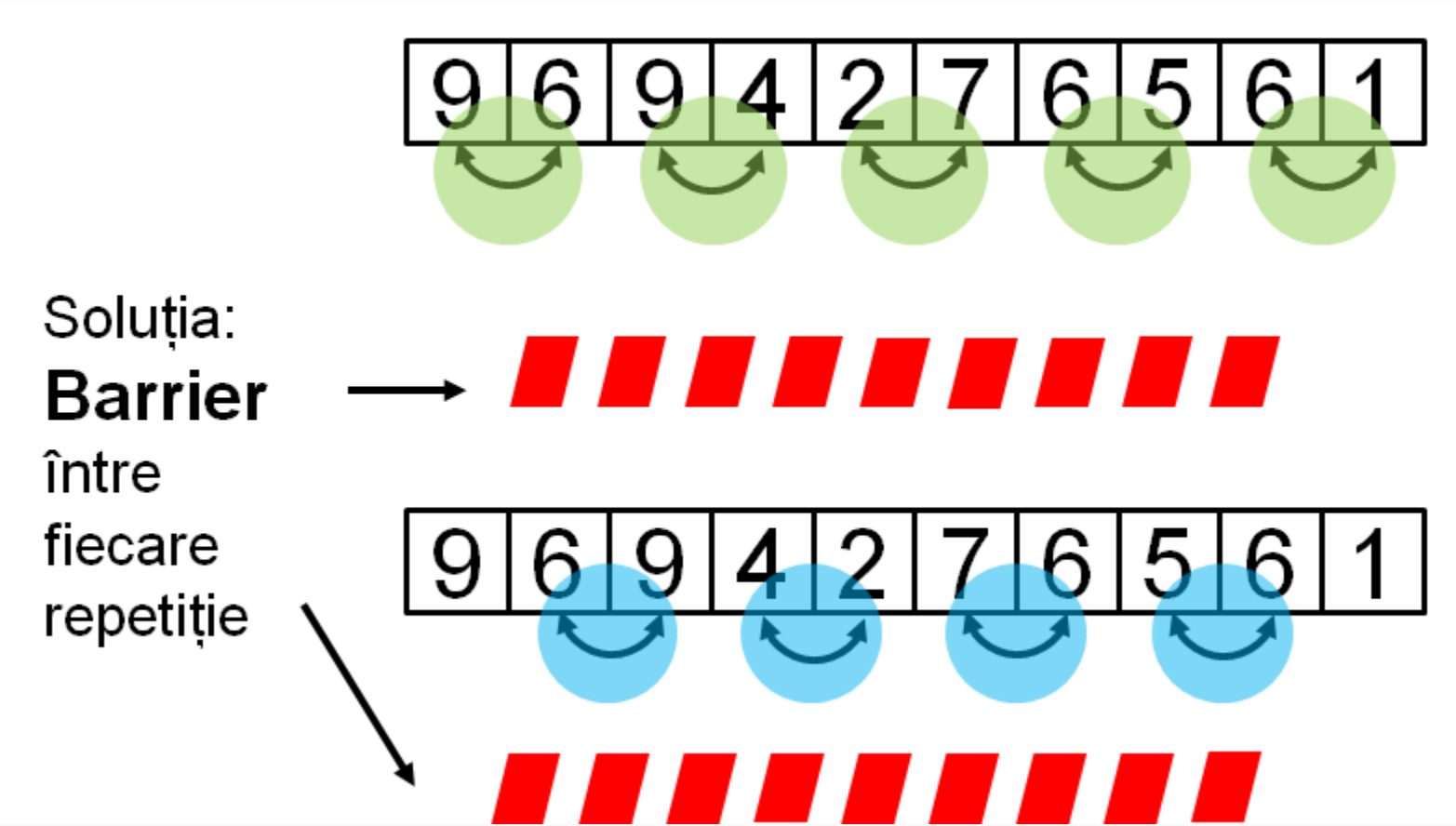




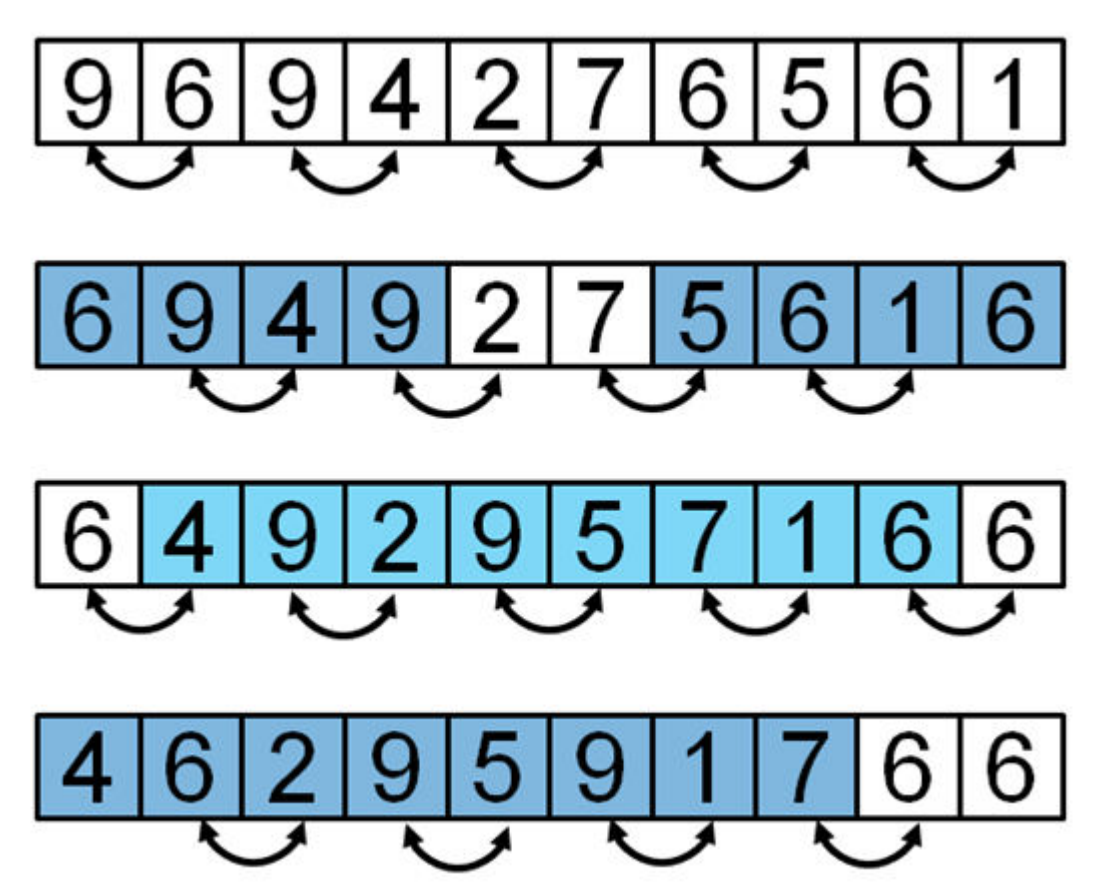
Odd-Even Transposition Sort - probleme de sincronizare



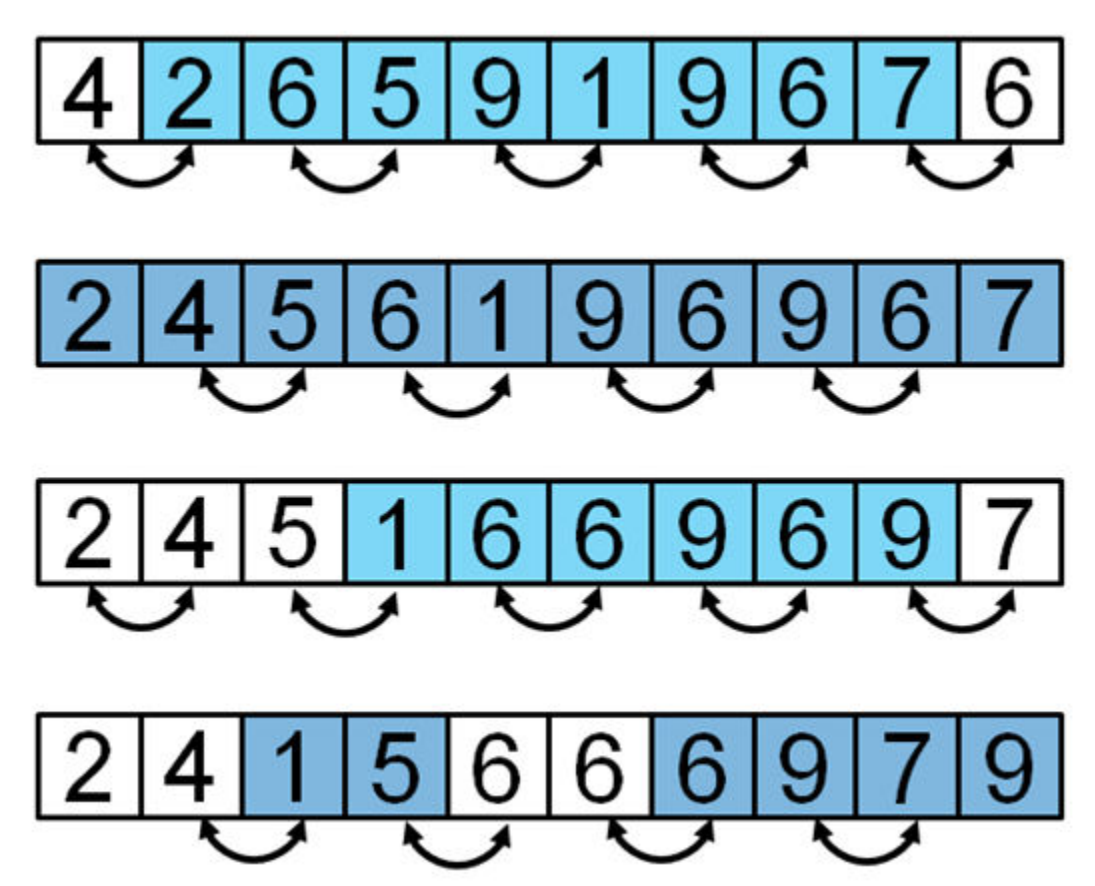
Odd-Even Transposition Sort - probleme de sincronizare - soluție



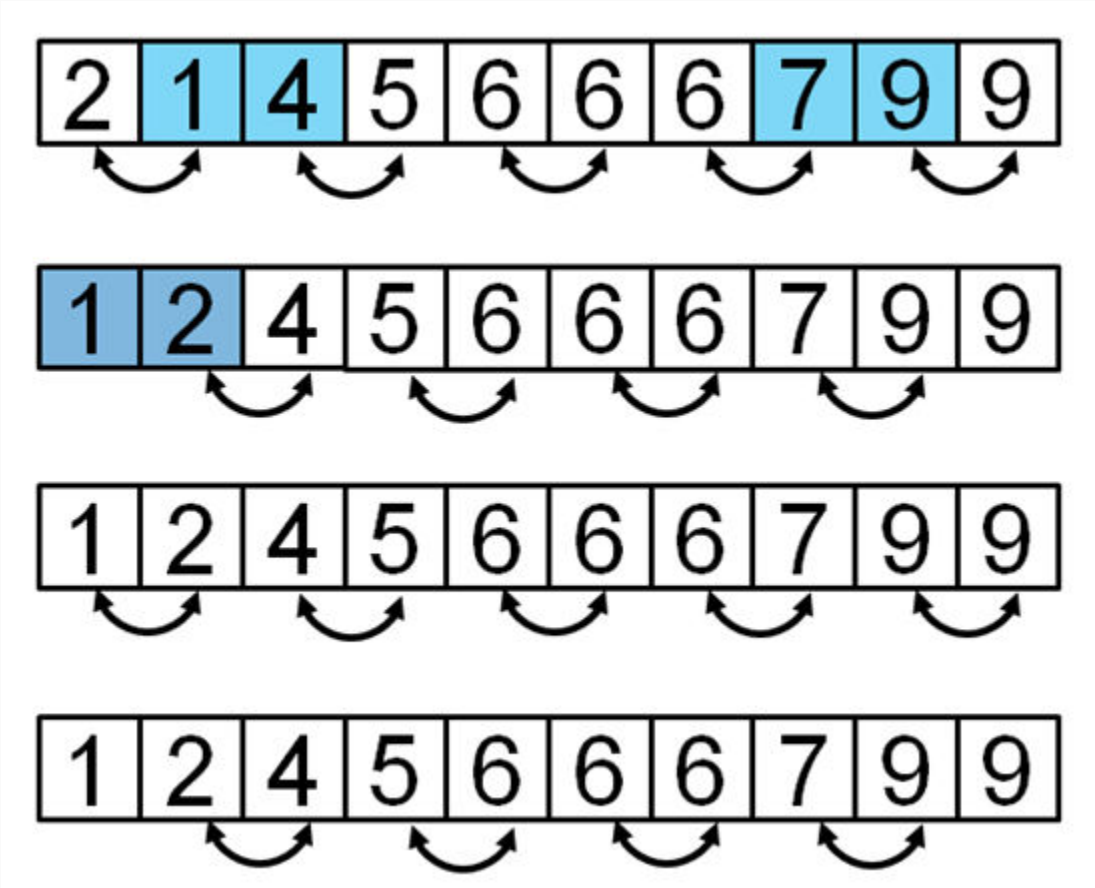
Rulare Odd-Even Transposition Sort



Rulare Odd-Even Transposition Sort



Rulare Odd-Even Transposition Sort



Paralelizare Odd-Even Transposition Sort

- **Atenție**: fiți atenți la indicele de start, mai exact cum îl folosiți în ambele faze ale algoritmului paralelizat, și la end (legat de faptul că ambele faze au for-ul `< N - 1`, N fiind dimensiunea array-ului => `i < min(end, N - 1)`)
- start impar => faza pară începe de la start + 1, faza impară de la start
- start par => faza pară începe de la start, faza impară de la start + 1

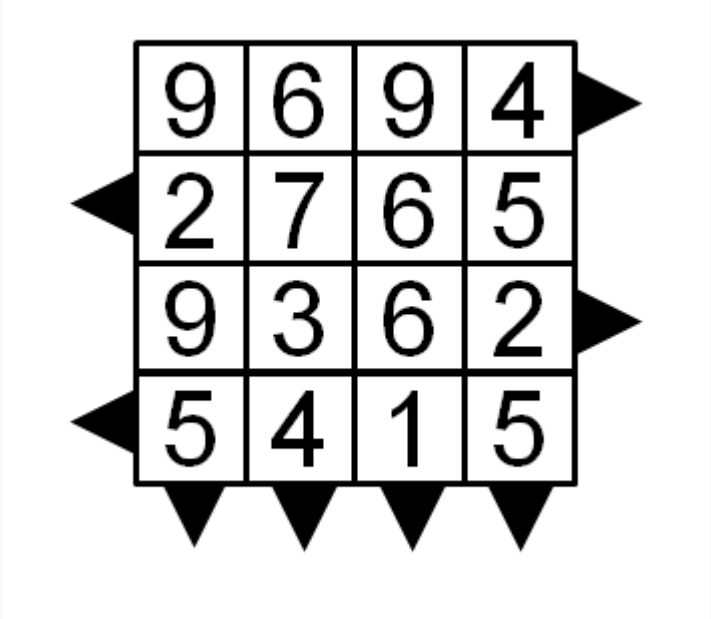
Row Column Sort (Snake Sort / Shear Sort)

- Shear Sort este un algoritm de sortare, conceput special pentru sisteme multi-procesor.
- Acest algoritm funcționează în două faze:
  - se sortează liniile matricei astfel încât randurile pare au valorile ordonate crescător, iar rândurile impare au valorile ordonate descrescător, ca să putem compara cel mai mare element de pe linia i cu cel mai mic de pe linia i + 1
  - se sortează coloanele crescător
- Elementele sunt sortate în maxim $\sup(\log_2 N) + 1$ faze, unde N = numărul de elemente
- Complexitatea: $O(N * \log_2 N)$

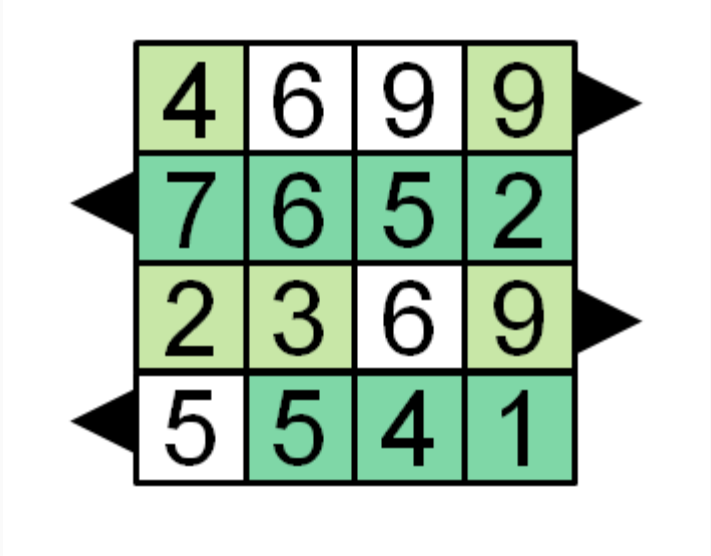
Shear Sort - pseudocod

```
function shearSort(matrix) {
  for (k = 0; k < ceil(log2(matrix.lines * matrix.columns)) + 1; k += 2) {
    for (i = 0; i < matrix.lines; i += 2) {
      sortAscendingLine(i);
    }
    for (i = 1; i < matrix.lines; i += 2) {
      sortDescendingLine(i);
    }
    for (var i = 0; i < matrix.columns; i++) {
      sortAscendingColumn(i);
    }
  }
}
```

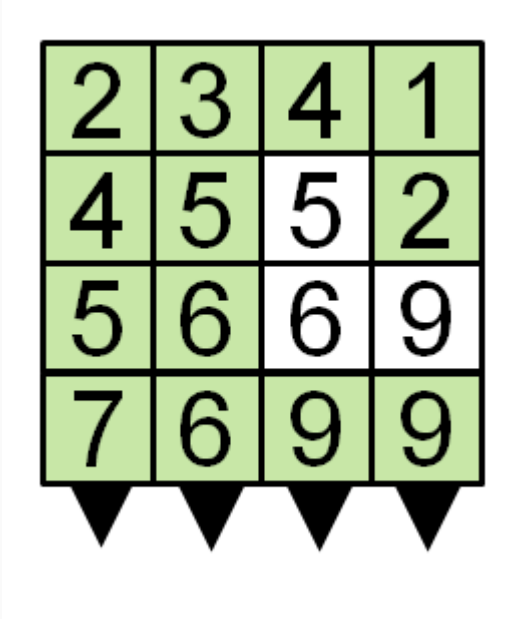
Rulare Shear Short - configurație inițială - sortăm liniile



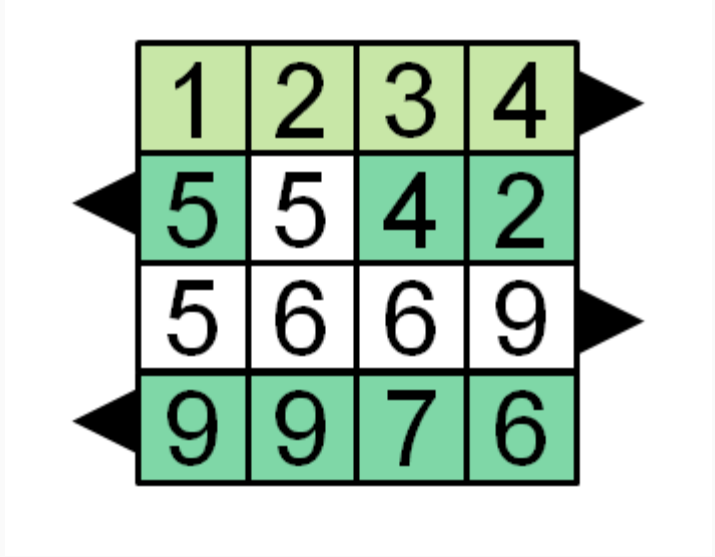
Rulare Shear Short - sortăm coloanele



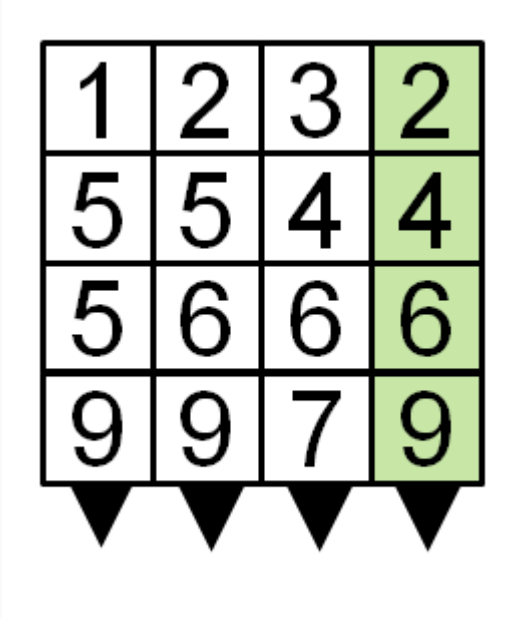
Rulare Shear Short - sortăm liniile



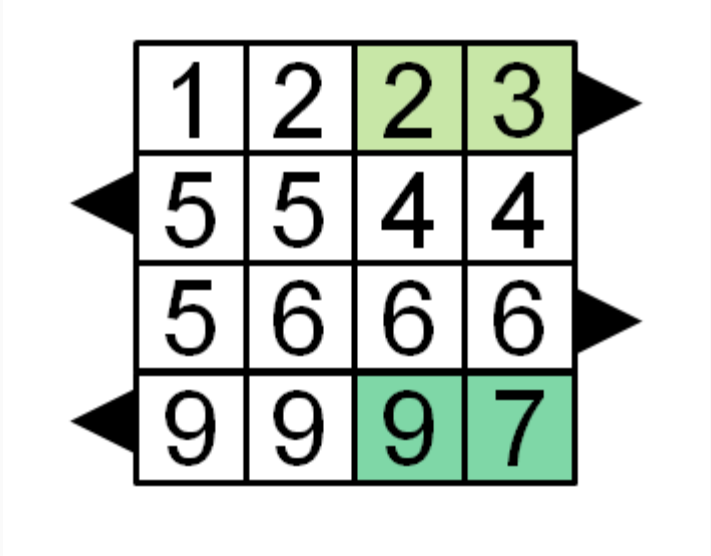
Rulare Shear Short - sortăm coloanele



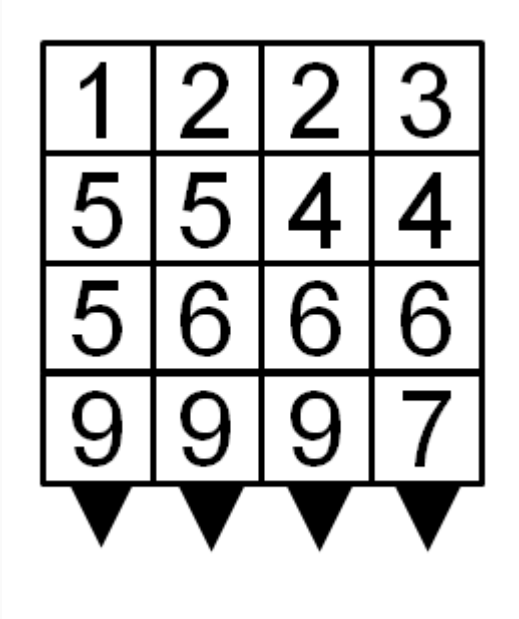
Rulare Shear Short - sortăm liniile



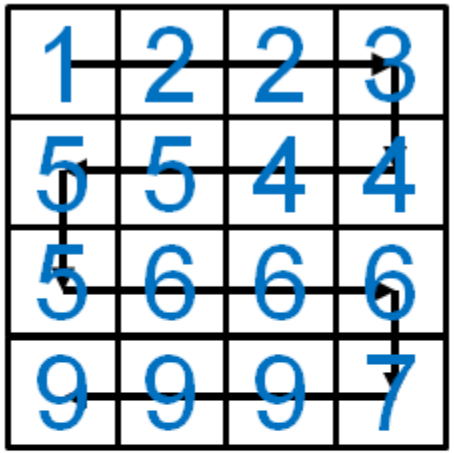
Rulare Shear Short - sortăm coloanele



Rulare Shear Short - gata - linii sortate, coloane sortate



Rulare Shear Short - rezultatul final

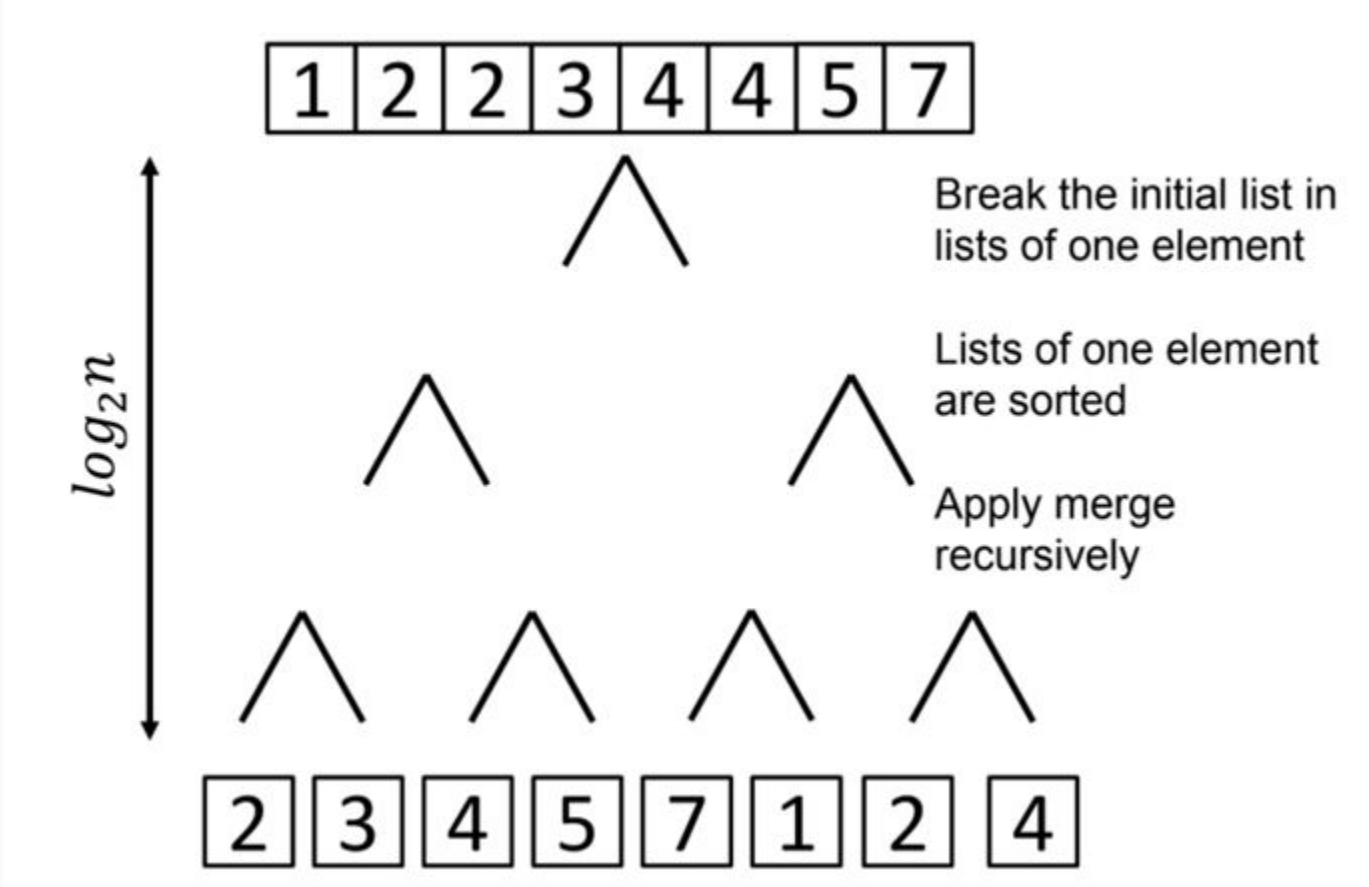


1	2	2	3	4	4	5	5	5	6	6	6	7	9	9	9

Merge Sort

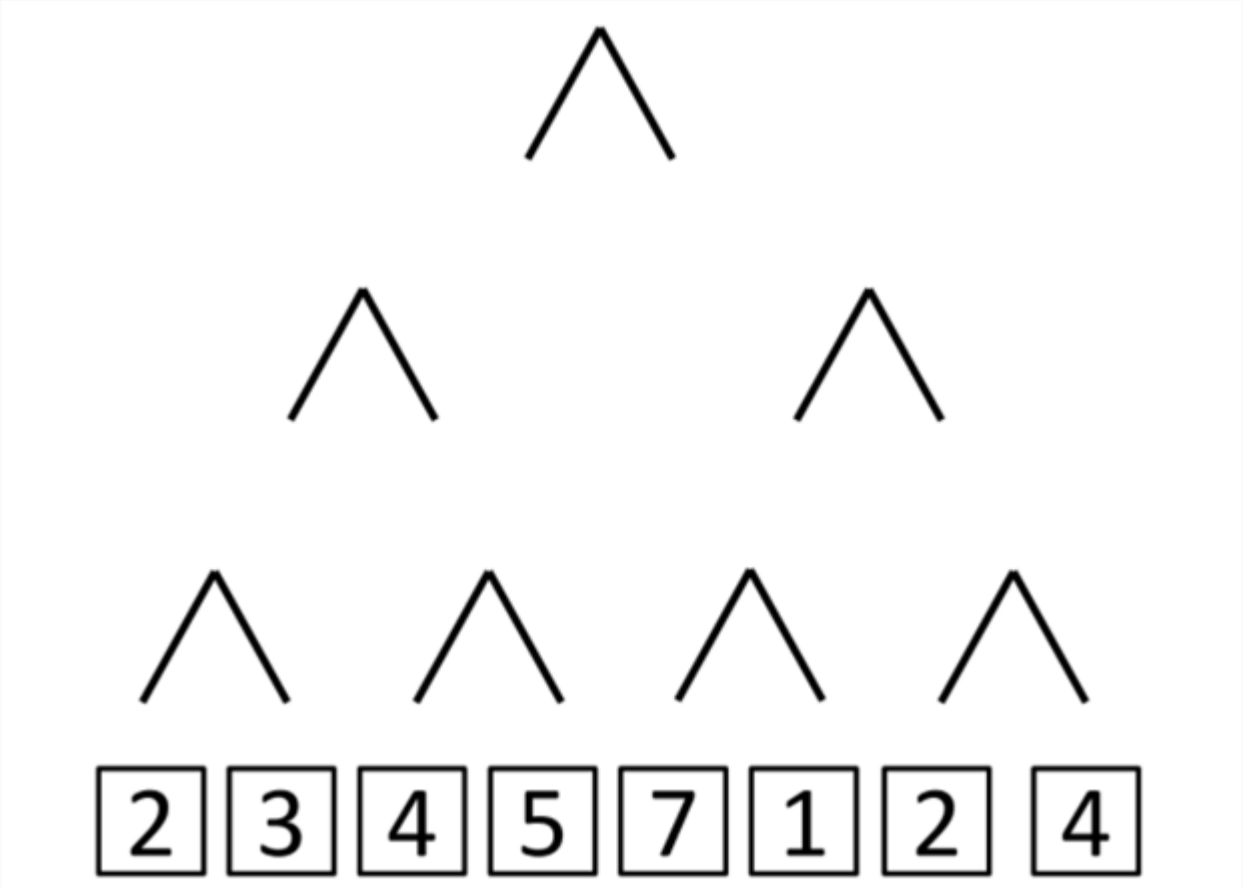
- Merge Sort este un algoritm de sortare, bazat pe tehnica divide et impera, care este format din următorii pași:
  - se împarte șirul de N elemente de sortat în N șiruri de lungime 1
  - se aplica operația de interclasare („merge”) între câte două astfel de șiruri de lungime 1, rezultând N/2 șiruri sortate de lungime 2
  - se repetă pașii de mai sus realizând interclasări între șiruri din ce în ce mai mari, până se ajunge la un șir sortat de N elemente.

Parallel Merge Sort

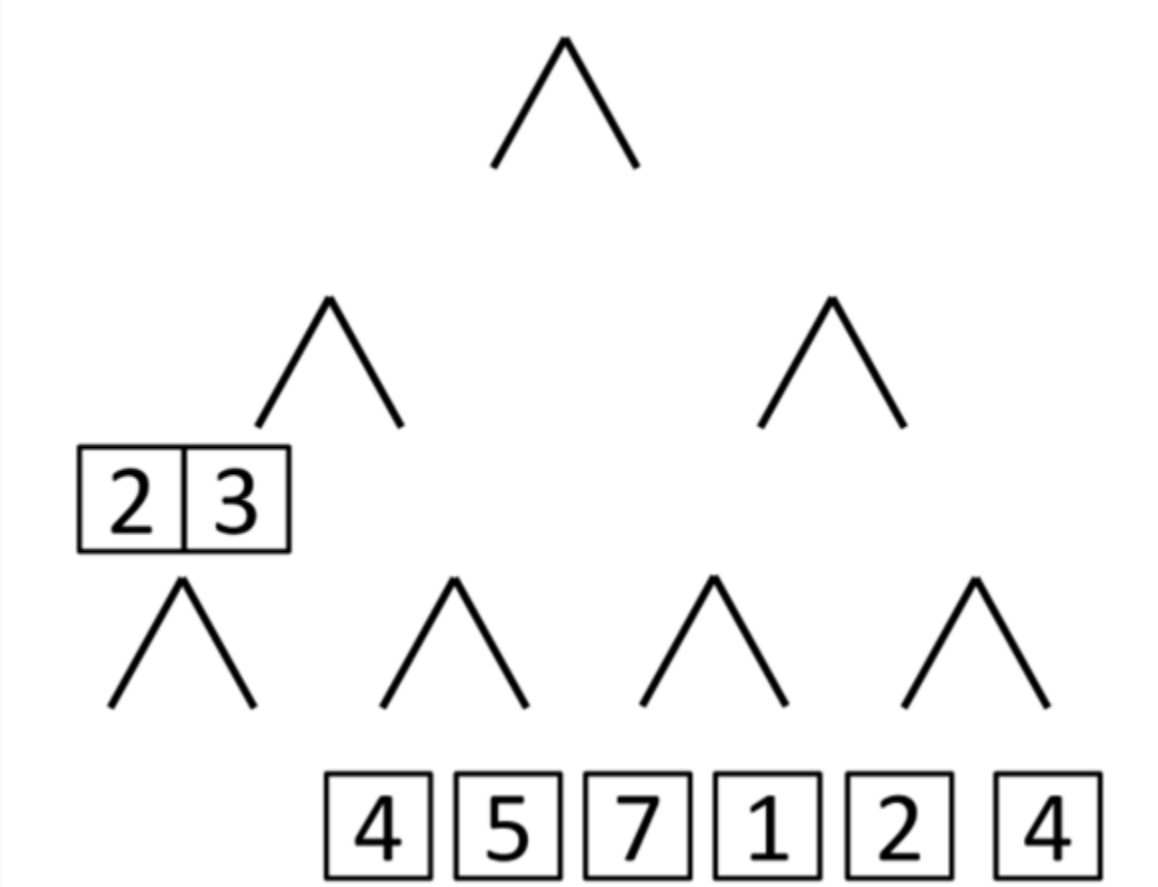


Complexitate temporală (N = P): O(N)

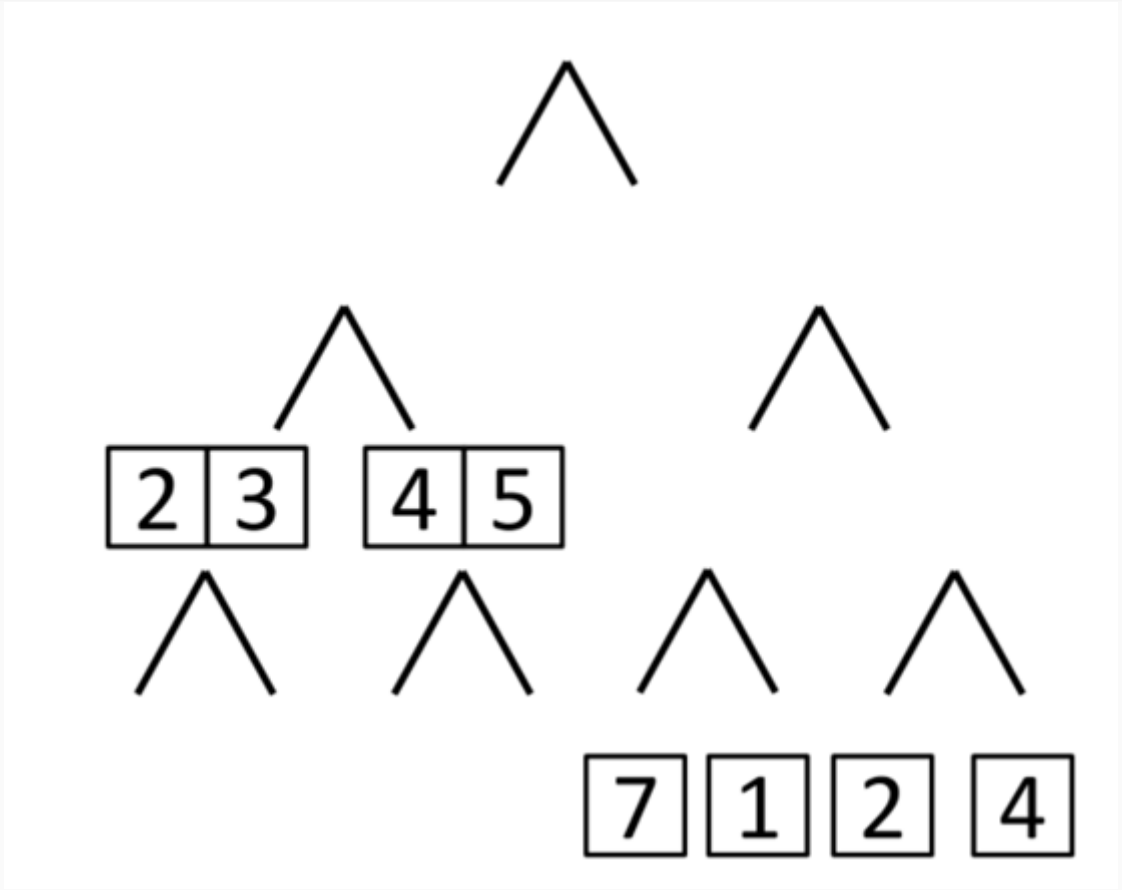
Parallel Merge Sort



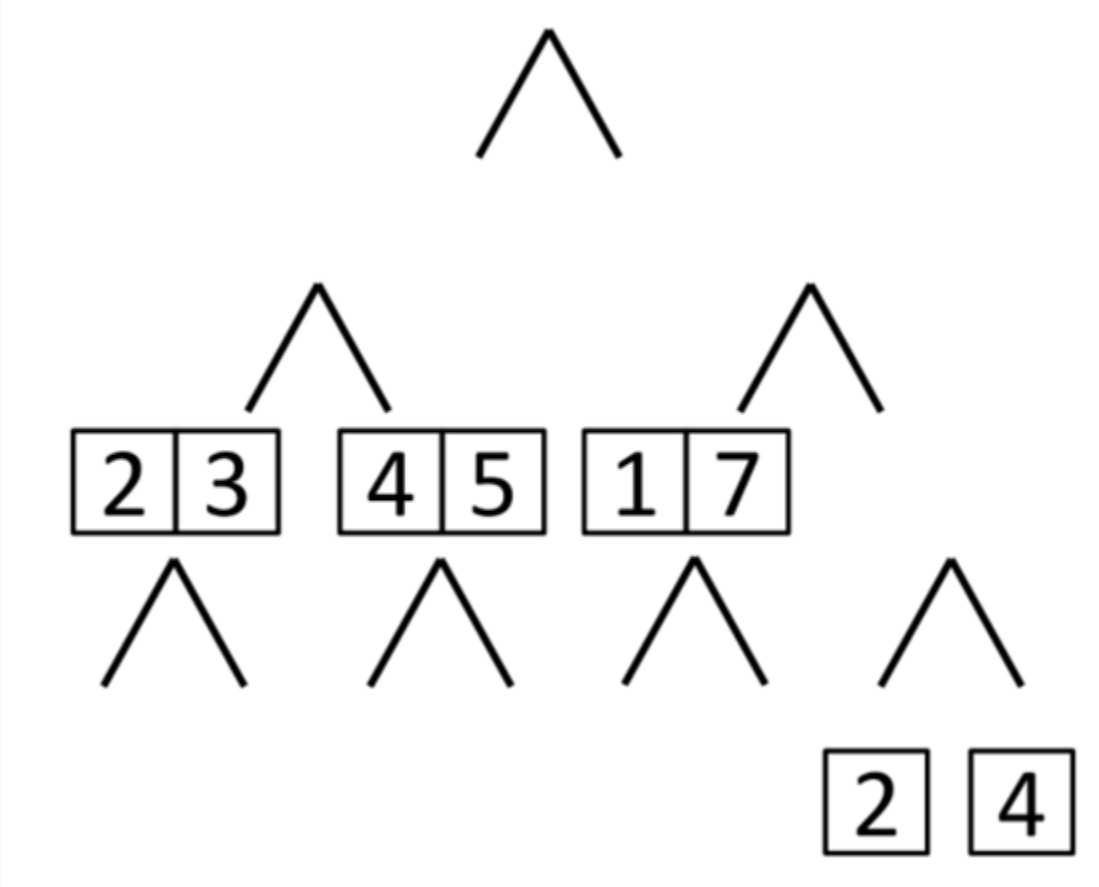
Parallel Merge Sort



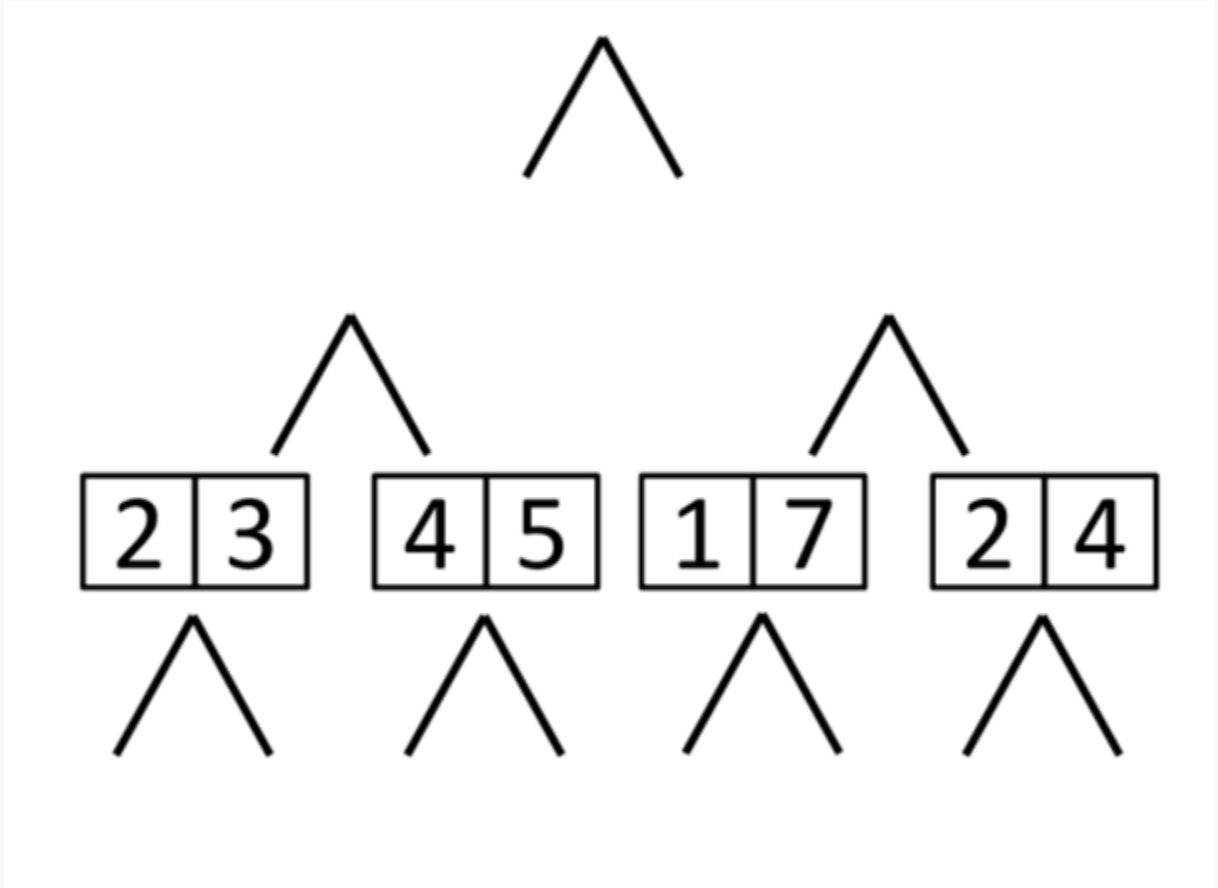
Parallel Merge Sort



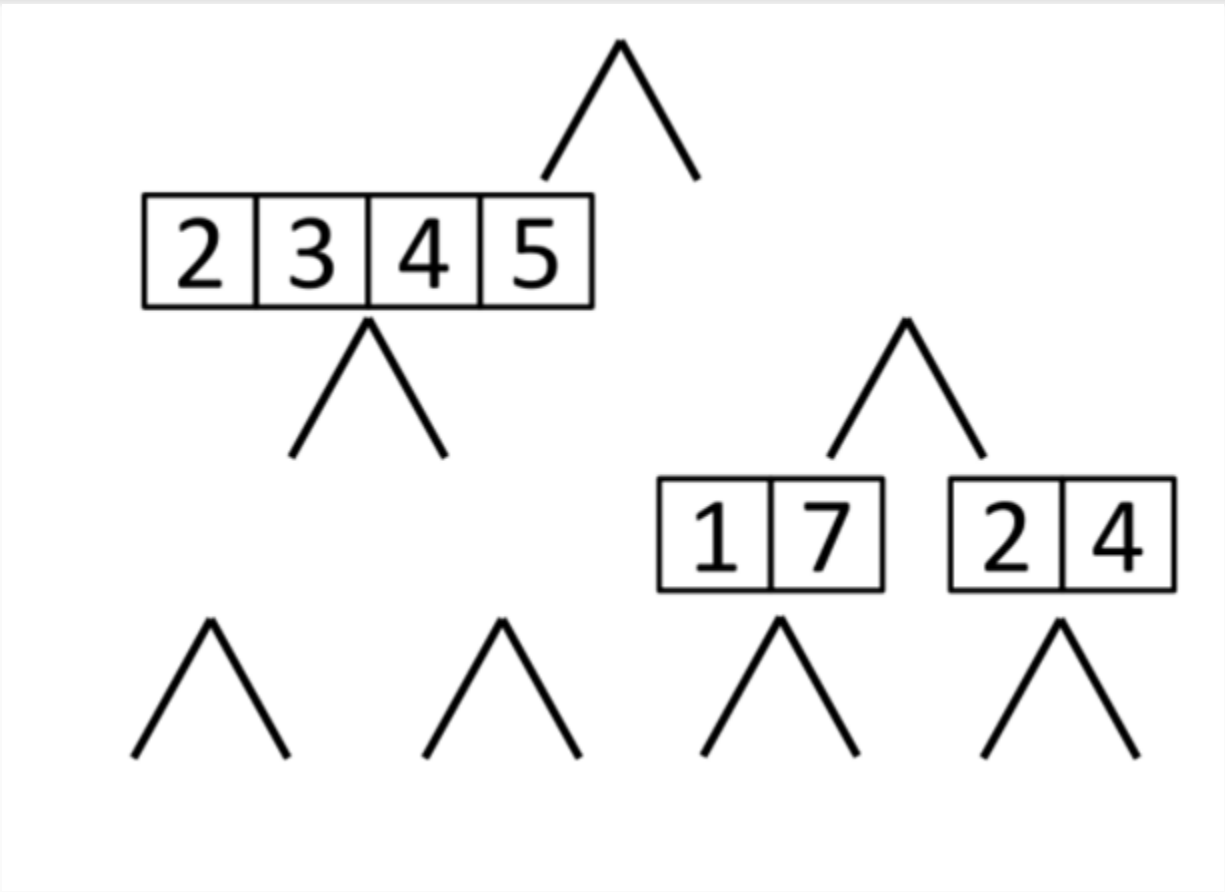
Parallel Merge Sort



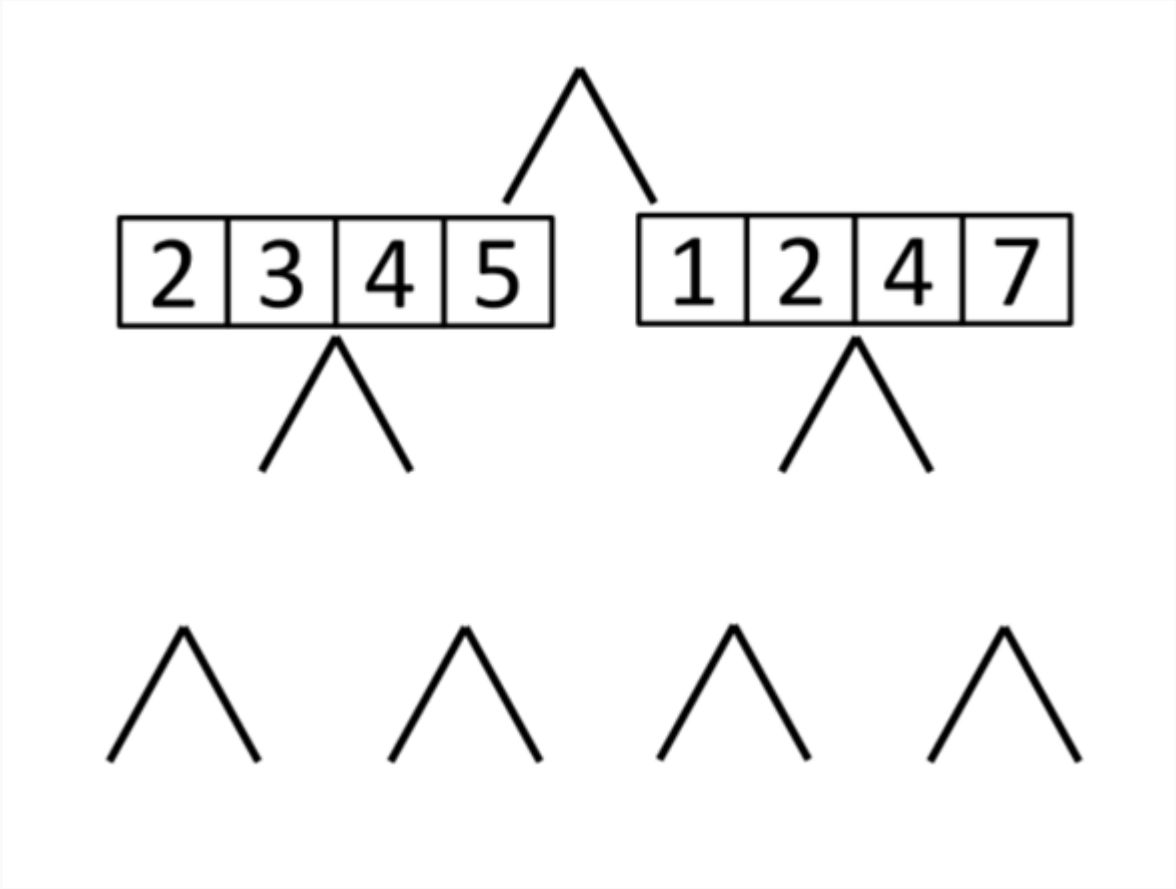
Parallel Merge Sort



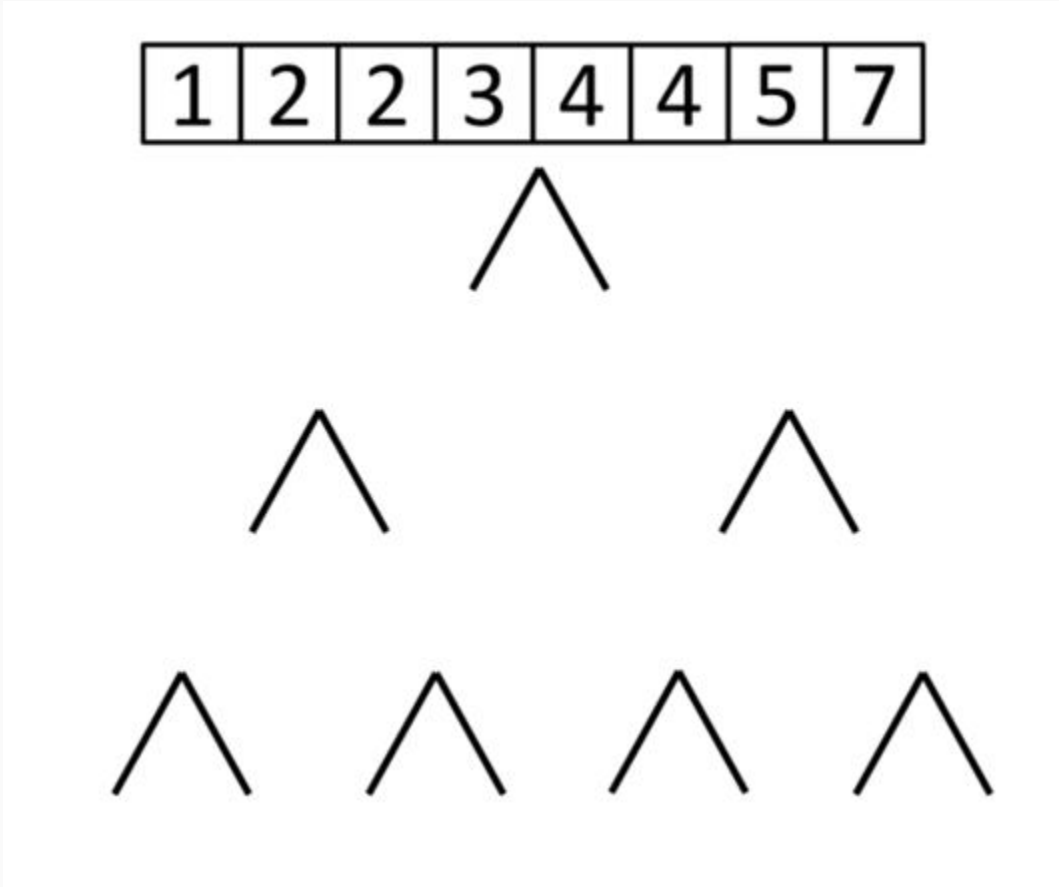
Parallel Merge Sort



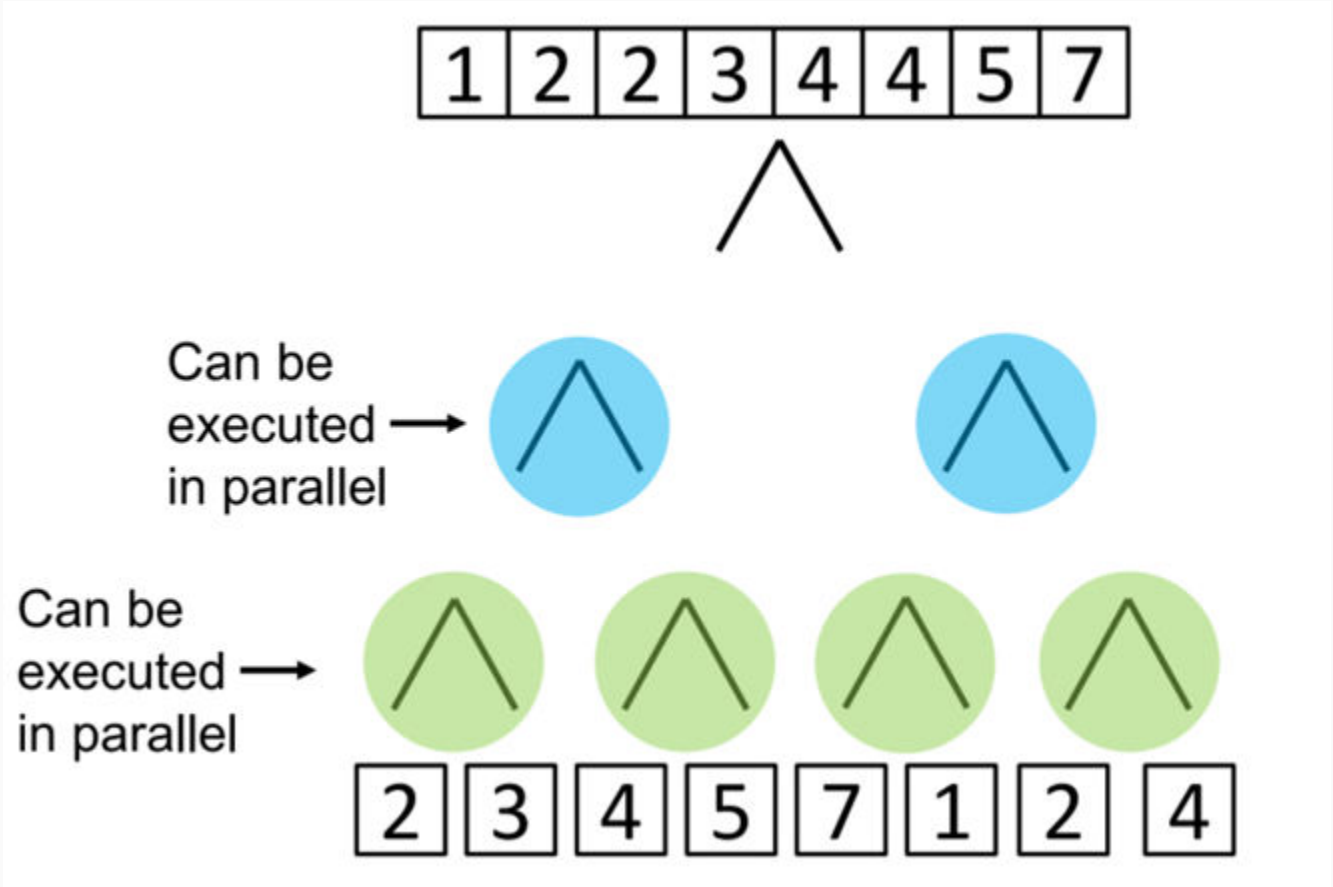
Parallel Merge Sort



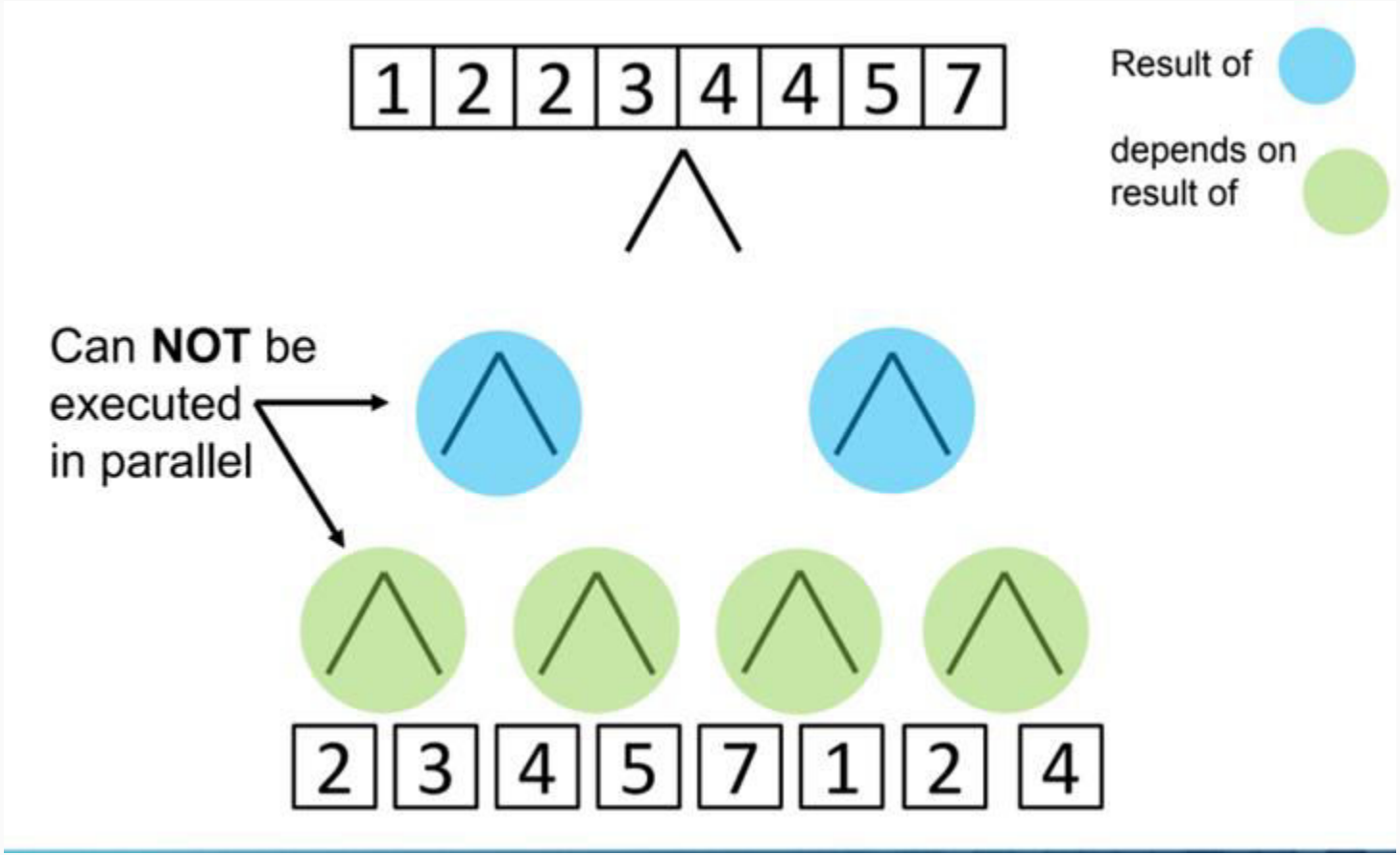
Parallel Merge Sort



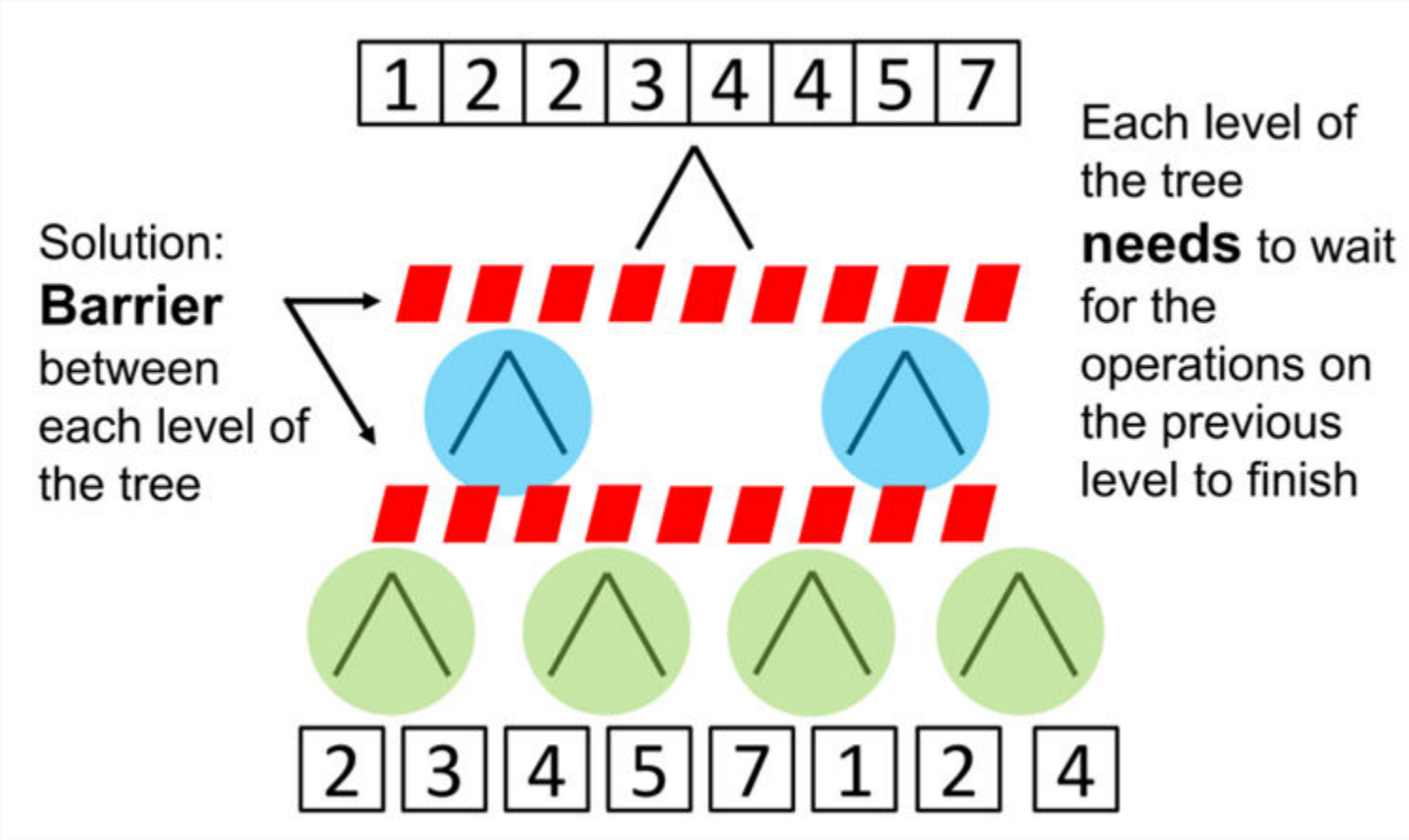
Parallel Merge Sort



Parallel Merge Sort



Parallel Merge Sort



Parallel Merge Sort - probleme

- După cum știți de la primele două laboratoare, folosim formule pentru împărțirea unui array pe thread-uri (**start** și **end**).
- Refreshment:
  - `start_index = id * (double) n / p`
  - `end_index = min(n, (id + 1) * (double) n / p))`, unde `id = id-ul thread-ului`, `n = dimensiunea array-ului`, `p = numărul de threads`

Parallel Merge Sort - probleme

Exemplu de indici pentru merge (N = 8, P = 4, dacă am aplica formulele obișnuite de start și de end):

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 3 - lungimea 4: (0, 2), (2, 4), (4, 6), (6, 8)

GREȘIT

Parallel Merge Sort - probleme

Teoretic, corect ar fi așa:

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 4), (4, 8)
- nivelul 3 - lungimea 4: (0, 8)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0)
- (2, 4) -> (0, 4)
- (4, 6) -> (4, 4)
- (6, 8) -> (4, 8)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): (0 / (2 * 2)) * (2 * 2), (2 / (2 * 2)) * (2 * 2)
- (2, 4) -> (0, 4): (2 / (2 * 2)) * (2 * 2), (4 / (2 * 2)) * (2 * 2)
- (4, 6) -> (4, 4): (4 / (2 * 2)) * (2 * 2), (6 / (2 * 2)) * (2 * 2)
- (6, 8) -> (4, 8): (6 / (2 * 2)) * (2 * 2), (8 / (2 * 2)) * (2 * 2)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): (0 / 4) * 4, (2 / 4) * 4
- (2, 4) -> (0, 4): (2 / 4) * 4, (4 / 4) * 4
- (4, 6) -> (4, 4): (4 / 4) * 4, (6 / 4) * 4
- (6, 8) -> (4, 8): (6 / 4) * 4, (8 / 4) * 4

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): 0 * 4, 0 * 4 = (0, 0)
- (2, 4) -> (0, 4): 0 * 4, 1 * 4 = (0, 4)
- (4, 6) -> (4, 4): 1 * 4, 1 * 4 = (4, 4)
- (6, 8) -> (4, 8): 1 * 4, 2 * 4 = (4, 8)

Formula: (x / (width * 2)) * (width * 2)

Parallel Merge Sort - probleme

Exemplu de indici pentru merge (N = 8, P = 4, dacă am aplicăm formula de pe slide-ul anterior):

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 0), (0, 4), (4, 4), (4, 8)
- nivelul 3 - lungimea 4: (0, 0), (0, 0), (0, 0), (0, 8)

CORECT

Vom avea thread-uri care nu vor face nimic!

Parallel Merge Sort - împărțirea corectă

Formula corectă de calcul (folosind trunchieri - împărțiri de integers):

- `start_index = id * (double) n / p`
- `end_index = (id + 1) * (double) n / p)`, unde `id = id-ul thread-ului, n = dimensiunea array-ului, p = numărul de threads`
- `start_local = (start_index / (2 * width)) * (2 * width)`
- `end_local = min(n, (end_index / (2 * width)) * (2 * width))`, unde `width e lungimea subarray-ului din nivelul curent de recursivitate la merge (2 * width = lungimea subaray-ului următor rezultat din merge)`

Întrebări

- Ce complexitate are Odd Even Transposition Sort pentru n=p (unde n este numărul de elemente de sortat și p este numărul de thread-uri)?

Laboratorul 4
Multithreading în Java



Multithreading în Java

- Mecanismul de multithreading este deja implementat în Java, astfel nu este nevoie să importăm o libr... bibliotecă, ca în C/C++ (`pthreads.h`)
- În Java, putem implementa funcționalitățile thread-urilor (adică să implementăm funcția executată de threads) în două moduri:
  - implementarea interfeței Runnable, cu implementarea metodei run()
  - extinderea clasei Thread și suprascrierea metodei run() (notă: clasa Thread implementează interfața Runnable)

Clasa Thread

```
public class MyThread extends Thread {

        @Override

    public void run() {

         System.out.println("Hello from my new thread!");

    }

}
```

Interfața Runnable

```
public class Task implements Runnable {

        @Override

    public void run() {

         System.out.println("Hello from my new thread!");

     }

}
```

Crearea, rularea și terminarea thread-urilor

- Folosind clasa Thread: https://gist.github.com/florinrm/eda49c9732701410e69abc3dda1b6d44
- Folosind interfața Runnable: https://gist.github.com/florinrm/23a4c2eb7de7ae1412beee9090f3d683

Rulare în linie de comandă

Dacă rulați în linie de comandă:

- compilare (în folderul cu sursele): javac *.java
- rulare (în folderul rădăcină al pachetului): java numePachet.Main (clasa care are metoda main)

run()

- În ambele cazuri (Thread și Runnable), `run()` reprezintă metoda executată de către thread, după ce acesta este lansat.
- Acesta reprezintă echivalentul lui `threadFunction`, trimis ca parametru la funcția `pthread_create` din pthreads.

start()

- Prin metoda `start()`, un thread este creat și lansat în execuție, fiind astfel executată metoda `run()`.
- **Notă**: a NU se confunda `start()` cu `run()`. Dacă am apela `run()` în locul lui `start()`, nu se va crea un nou thread și `run()` va fi executată de către thread-ul curent

join()

- Prin metoda `join()` se așteaptă terminarea execuției thread-urilor.
- Notă: metoda `join()` poate arunca excepții de tipul `InterruptedException`

synchronized

- Keyword-ul synchronized marchează o zonă critică în Java (acesta reprezintă un mutex), mai concret metode și blocuri de cod.
- Când să execute cod synchronized, thread-ul va folosi un monitor (un lock), pentru a rezerva zona critică în care execută instrucțiuni
- Putem avea:
  - metode sincronizate
  - blocuri de cod sincronizate
- **Important**: Metodele și blocurile de cod synchronized din Java sunt reentrante. Dacă un thread a obținut monitorul unui obiect, atunci el va putea intra în orice alt bloc și metodă sincronizate ce sunt asociate cu acel obiect (implicit cu acel monitor).

Metode sincronizate

```
public synchronized T get(int index) {

        if (index < content.length) {

            return content[index];

        }

        throw new IndexOutOfBoundsException(index + " is out of bounds for
MyConcurrentArray of size " + content.length);

}
```

Metode sincronizate

- Când un thread va apela metoda sincronizată get(), acesta va trebui să obțină monitorul (lock-ul) pentru a putea executa metoda respectivă.
- Dacă monitorul este liber, thread-ul va executa metoda.
- Altfel, dacă monitorul e folosit de alt thread, thread-ul curent va aștepta până când monitorul (lock-ul) va fi eliberat.

Blocuri sincronizate

```
public void set(int index, T newT) {

        //Bloc de cod sincronizat ce folosește instanța curentă (this) pe post de zăvor.

        synchronized(this) {

            if (index < content.length) {

                content[index] = newT;

            }

            throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);

        }

    }
```

Blocuri sincronizate

În cadrul metodei set(), avem o zonă de cod sincronizată.

În acest caz, folosim instanța curentă a clasei (this) drept zăvor / lock pentru accesul în zona critică

Sincronizare cod static

```
private static int numberOfInstances = 0;

public static synchronized int getNumberOfInstances(){

        return numberOfInstances;

}
```

Sincronizare cod static

```
public MyConcurrentArray() {
        synchronized(MyConcurrentArray.class) {
            ++numberOfInstances;
        }
    }
```

Sincronizare cod static

Când sincronizăm cod static (metode statice, câmpuri statice), monitorul / lock-ul folosit este unul asociat clasei (MyConcurrentArray.class), nu asociat instanței

Concluzie - cum folosim synchronized

- synchronized pe un obiect → se folosește monitor lock-ul acelui obiect
- synchronized pe o functie ne-statică → se folosește monitor lock-ul obiectului de care aparține funcția
- synchronized pe o clasă → se folosește monitor lock-ul clasei (se face locking pe orice instanță a clasei respective)
- synchronized pe o funcție statică → se folosește monitor lock-ul clasei de care aparține funcția

CyclicBarrier

- Bariera ciclică este un tip de barieră folosit în structuri iterative (mai precis în structuri repetitive de tip for și while), prin care thread-urile de resincronizează la fiecare iterație, astfel o barieră ciclică este una reutilizabilă (precum cea din pthreads).
- În Java, mecanismul de barieră reutilizabilă este reprezentat de CyclicBarrier.

CyclicBarrier

```
CyclicBarrier barrier = new CyclicBarrier(NUMBER_OF_THREADS); // in main

// in run()

try {

      barrier.await();

      // cod

} catch (BrokenBarrierException | InterruptedException e) {

      e.printStackTrace();

}
```

volatile

- `volatile` este un keyword asociat unei variabile, care specifică faptul că respectiva variabilă nu este cached (în CPU memory) și că citirea și scrierea ei se fac direct din memoria RAM, astfel se garantează faptul că o schimbare efectuată de un thread asupra variabilei respective este imediat vizibilă celorlaltor thread-uri
- Exemplu: `public static volatile int counter = 0;`

Operații atomice

```
AtomicInteger a = new AtomicInteger(0);

int x = a.get();

a.set(5);

x = a.getAndSet(6); // intoarce 5, seteaza 6

boolean b = a.compareAndSet(6, 5); // compara valoarea curenta cu 6 si,
                                   // daca este egala, modifica la 5
```

Întrebări de interviu

- https://dzone.com/articles/top-15-java-multithreading-concurrency-interview-q
- https://www.indeed.com/career-advice/interviewing/multithreading-interview-questions
- https://javarevisited.blogspot.com/2014/07/top-50-java-multithreading-interview-questions-answers.html

Temă de gândire (problemă de interviu)

Aveți următorul cod serial: link

Întrebări:

1) Ce reprezintă acest cod?
2) Ce probleme poate aduce dacă îl folosim într-o soluție paralelă și cum pot fi rezolvate?

Laboratorul 5
Probleme de sincronizare



Problemă

```
function_thread_0:
  noua_valoare =
calculeaza_valoare_a()
  mutex_a.lock()
      a = noua_valoare
  mutex_a.unlock()
```

```
function_thread_1:
  while(a nu s-a actualizat): // busy waiting
      mutex_a.lock()
      if (a s-a actualizat):
          afiseaza(a)
          a = -1 //  Resetam valoarea lui a.
      mutex_a.unlock()
```

- thread-ul 0 calculează o valoarea și o scrie în variabila ”a”
- thread-ul 1 verifică dacă valoarea lui ”a” s-a actualizat și o afișează

Problemă

- thread-ul 1 va cicla de mai multe ori, chiar dacă thread-ul 0 nu actualizează variabila ”a”, astfel se pierde timp inutil
- această abordare se numește **busy waiting**, care aici nu este o idee rea, însă avem dezavantajul că se irosesc resurse

wait()

- forțează thread-ul curent să între în starea de waiting (să aștepte), până când alt thread apelează `notify()` sau `notifyAll()` pe același obiect, pe care thread-ul curent trebuie să-l dețină, situație când:
  - se execută o metodă sau un bloc synchronized pe obiectul respectiv
  - se execută o metodă statică synchronized pe clasa din care face parte obiectul respectiv
- atenție: nu este recomandat să folosiți wait() pe obiecte statice sau pe string-uri constante (final), căci JVM optimizează accesul la aceste variabile

notify() + notifyAll()

- `notify()` - este ”trezit” din waiting un thread aleatoriu și pus în running
- `notifyAll()` - sunt ”trezite” toate thread-urile din waiting și puse în running

T0	T1

`synchronized(object) { // lock`
`  while (!condition) {`
`  object.wait();`
`  }`
`}`
	`[...]`
	`synchronized(object) { lock`
	`  condition = true;`
	`  object.notify();`
	`  // object.notifyAll();`
	`}`
	`[...]`

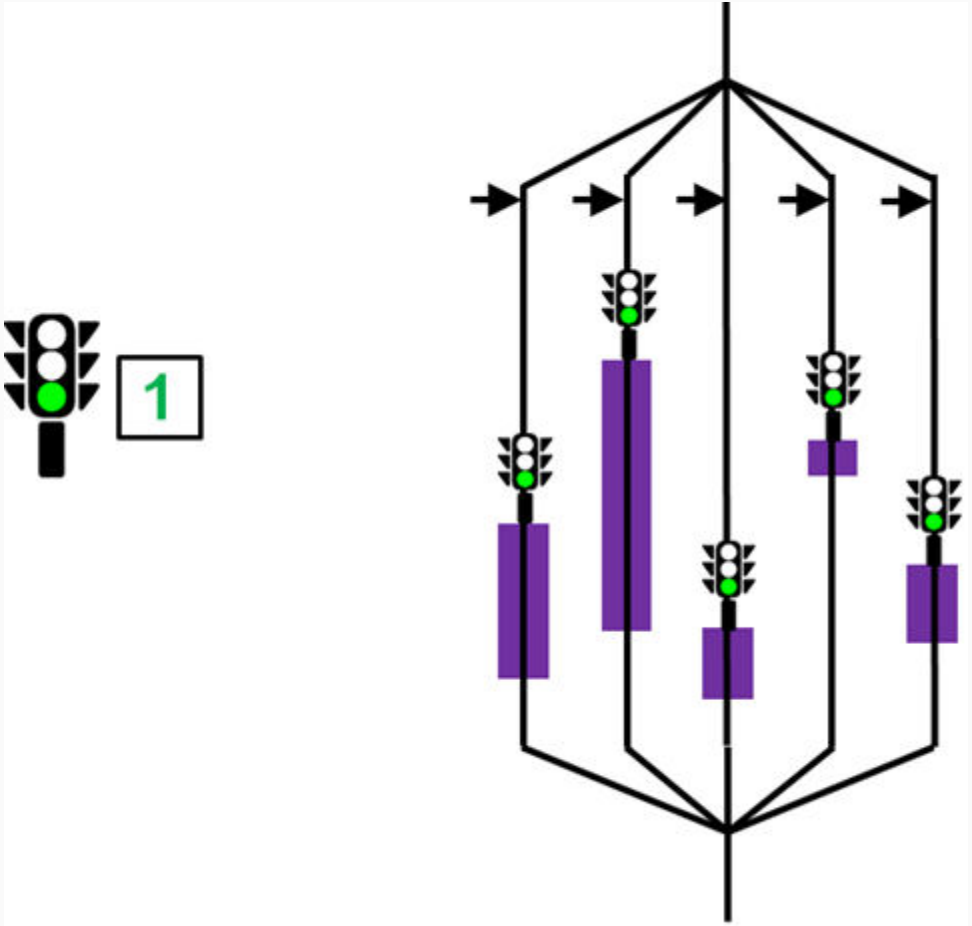
Important! Cand apelam wait() si notify(), trebuie sa detinem monitor lock-ul obiectului pe care apelam functiile.

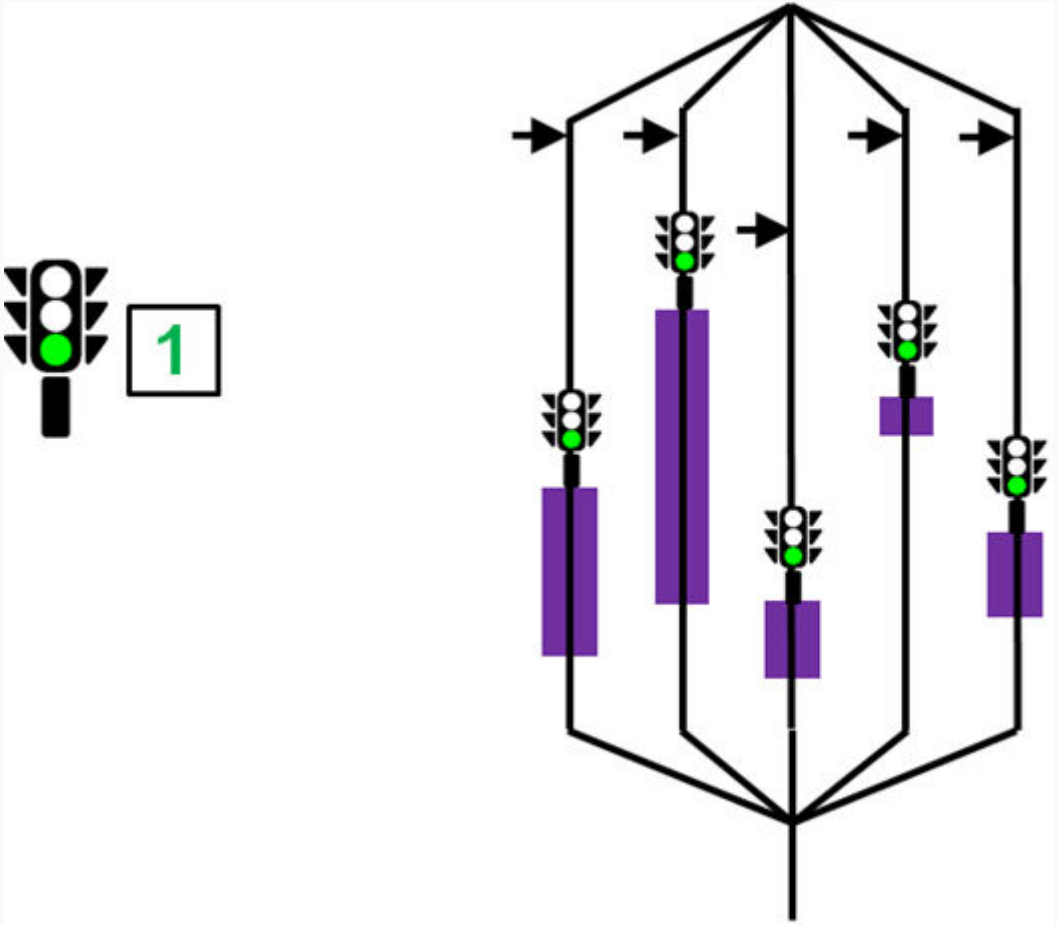
Semafor

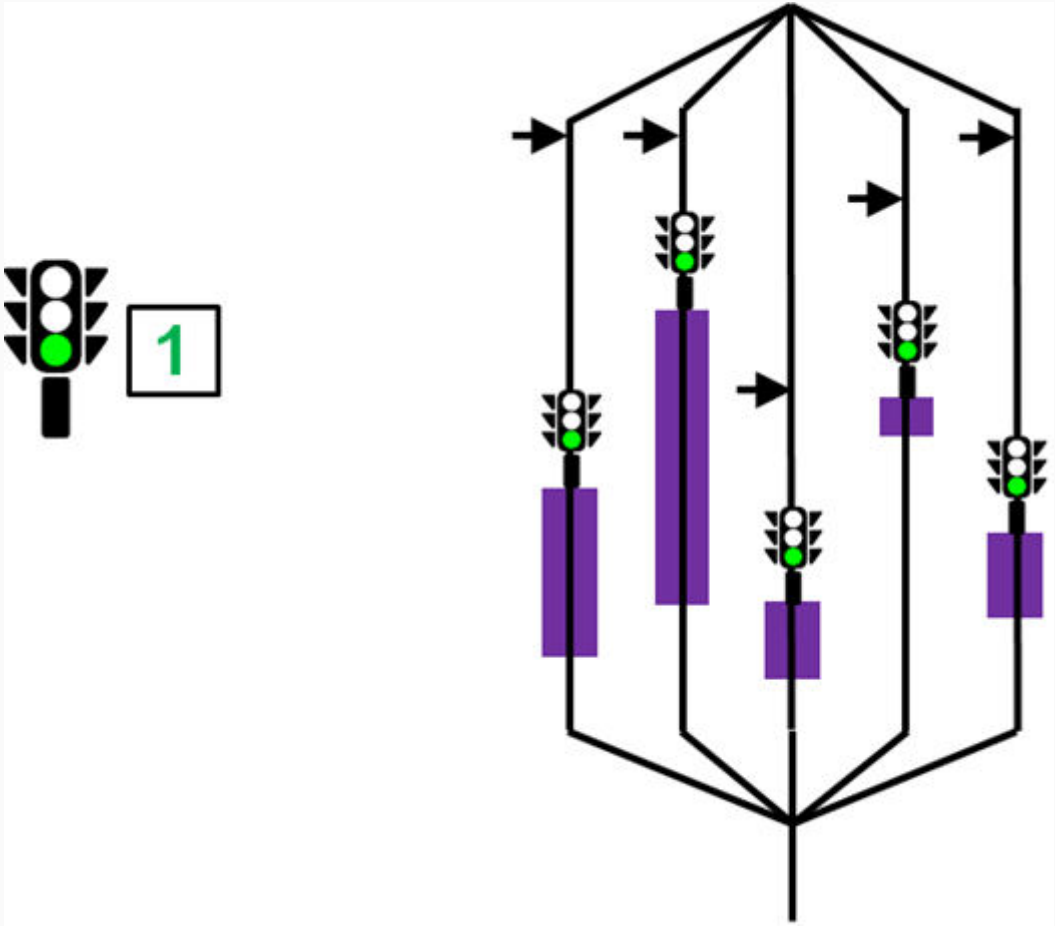
- semaforul este o generalizare a unui mutex, unde mai multe thread-uri pot intra într-o zona critică la un moment dat
- acesta are un contor care determină câte thread-uri mai pot intra în zona critică
- un thread este lăsat să intre în zona critică doar dacă contorul curent e mai mic decât numărul maxim de thread-uri permise în zona critică

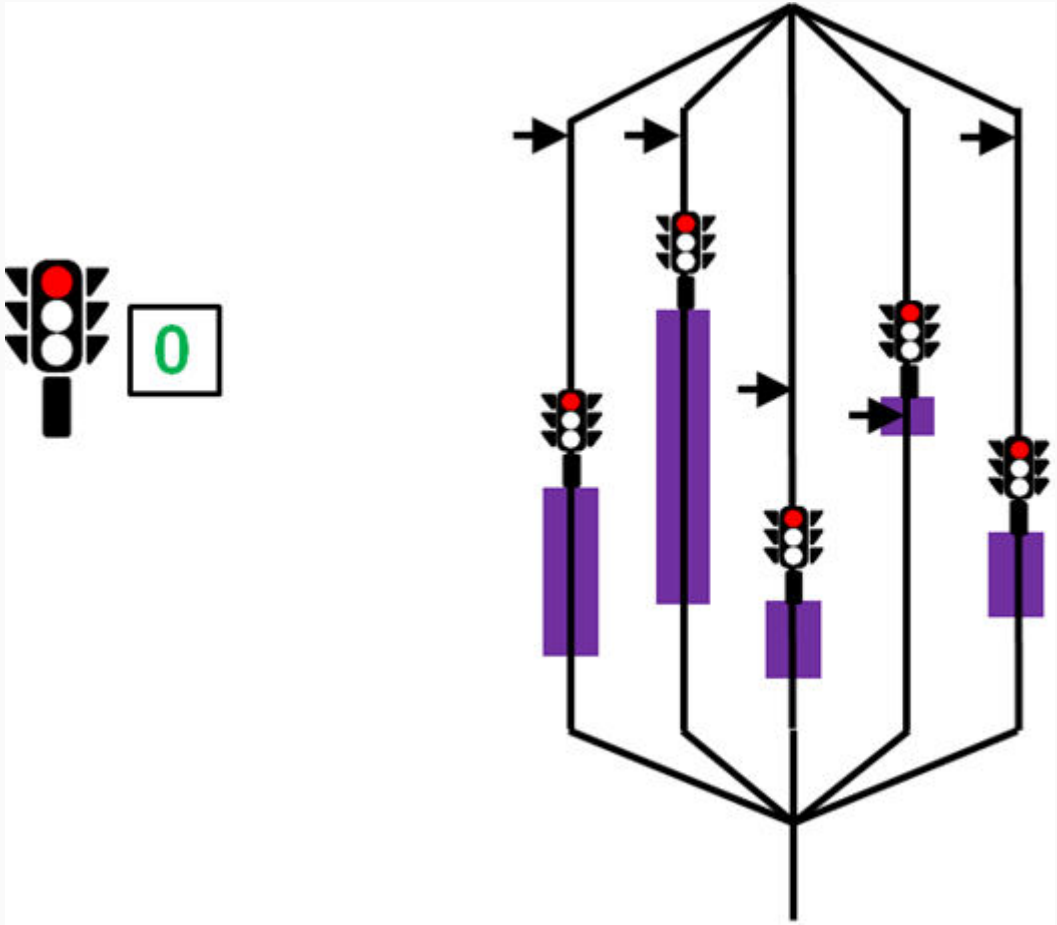
Semafor - funcții

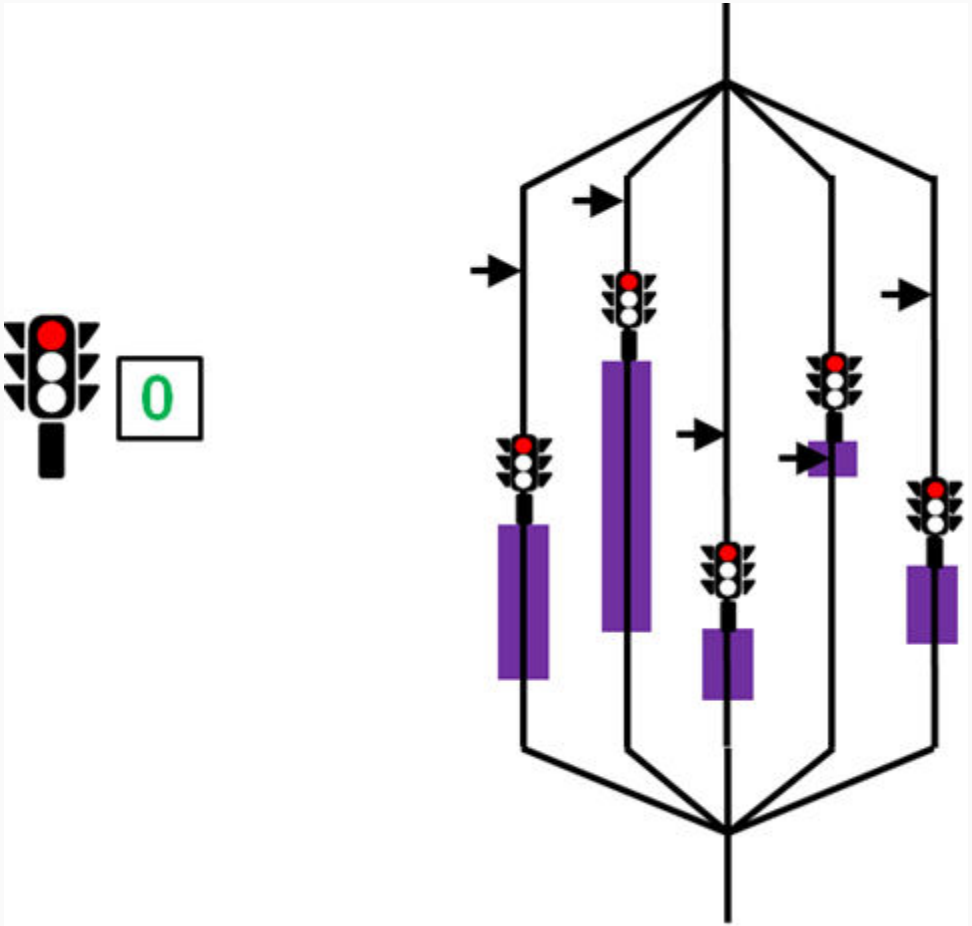
- acquire - se încearcă trecerea de semafor și intrarea în zona critică. Dacă contorul curent e mai mic decât numărul maxim de thread-uri permise, thread-ul intră și contorul este decrementat
- release - un thread iese din zona critică, contorul este incrementat

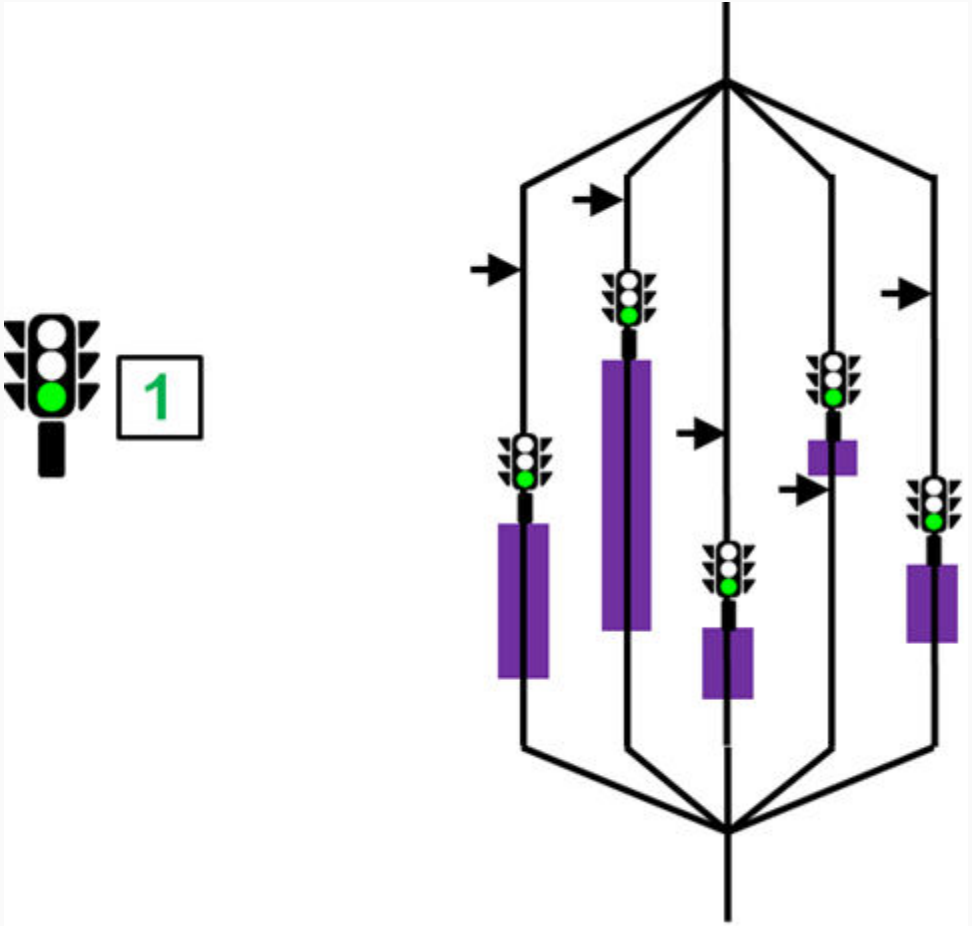

1

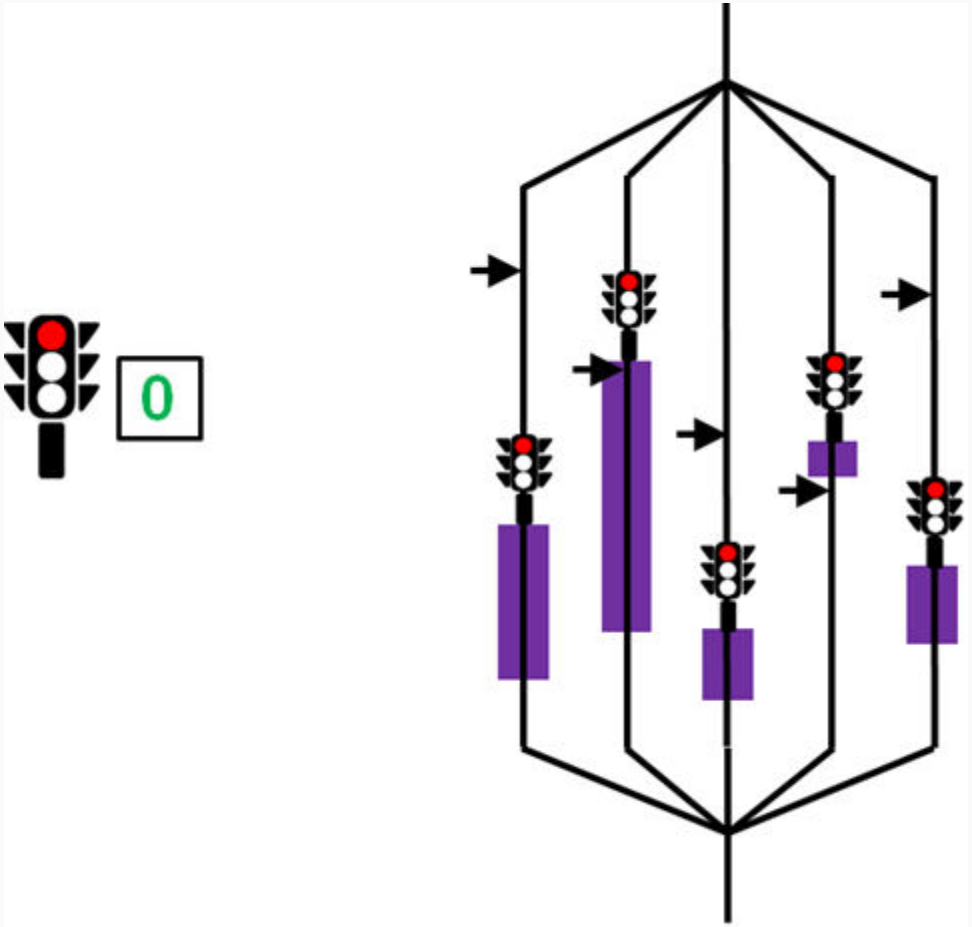

1

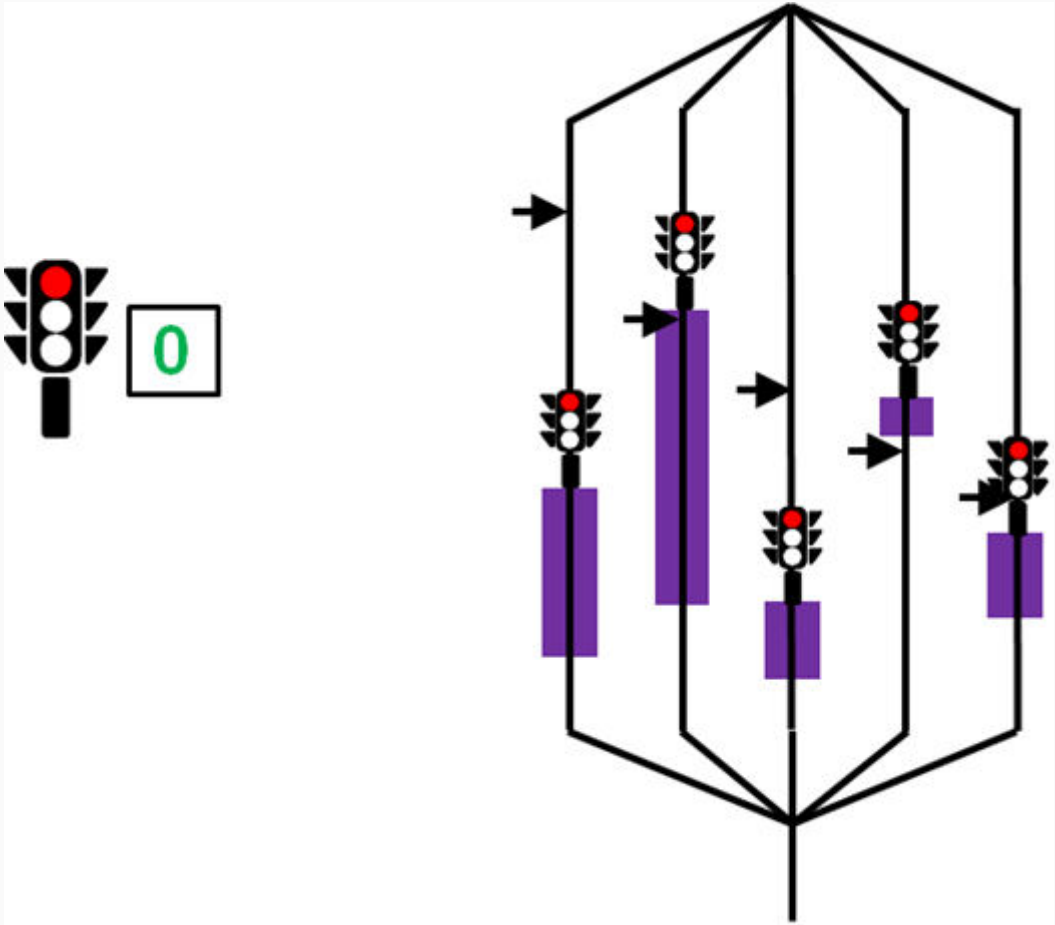

1

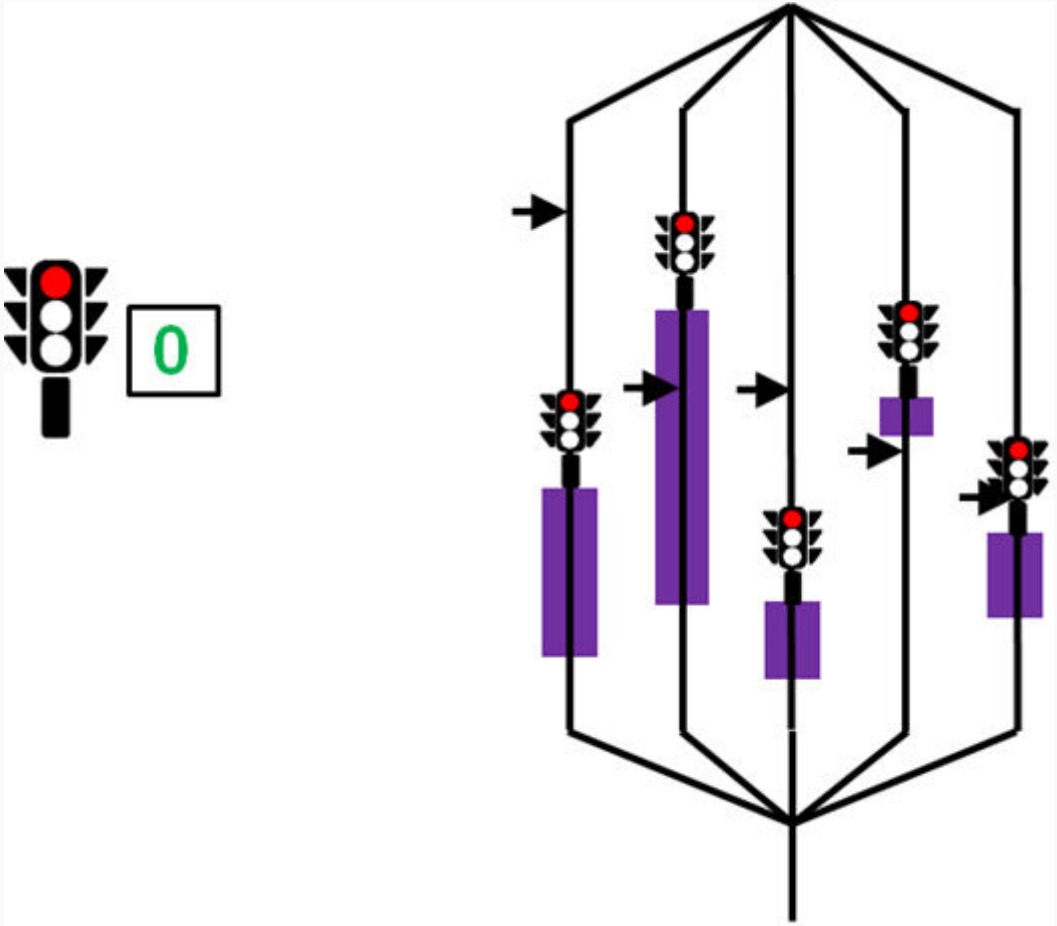

0

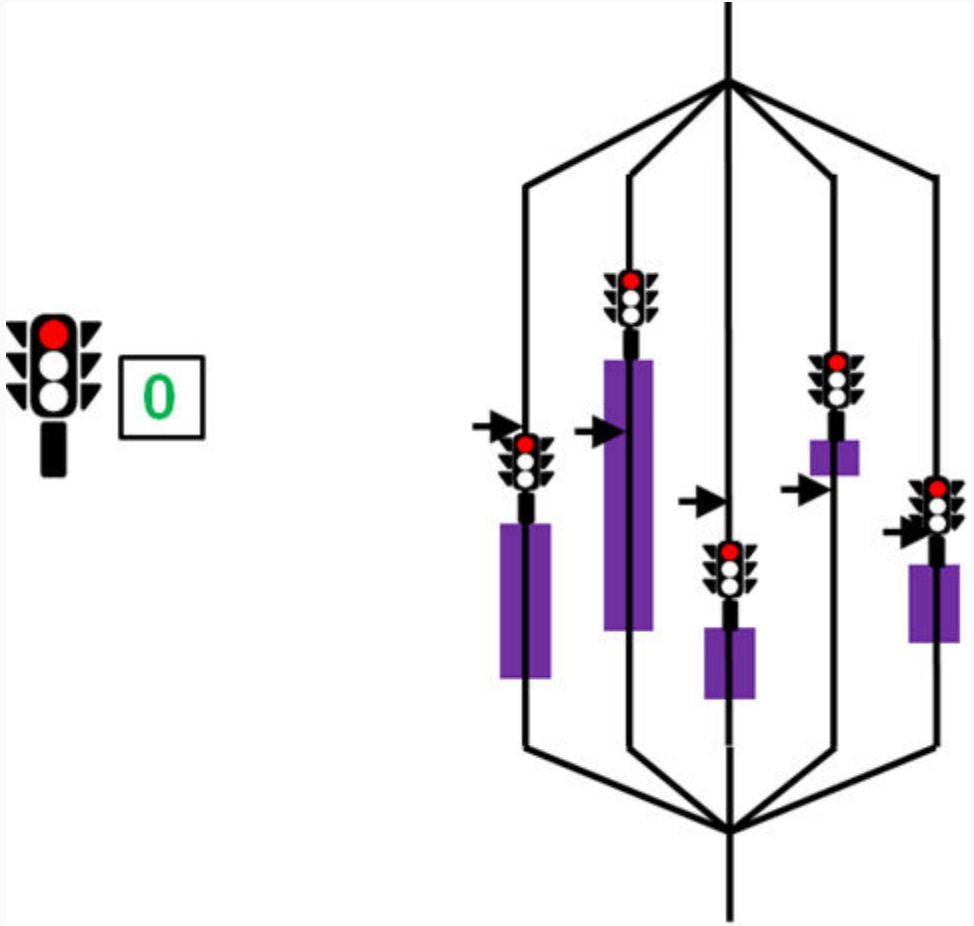

0

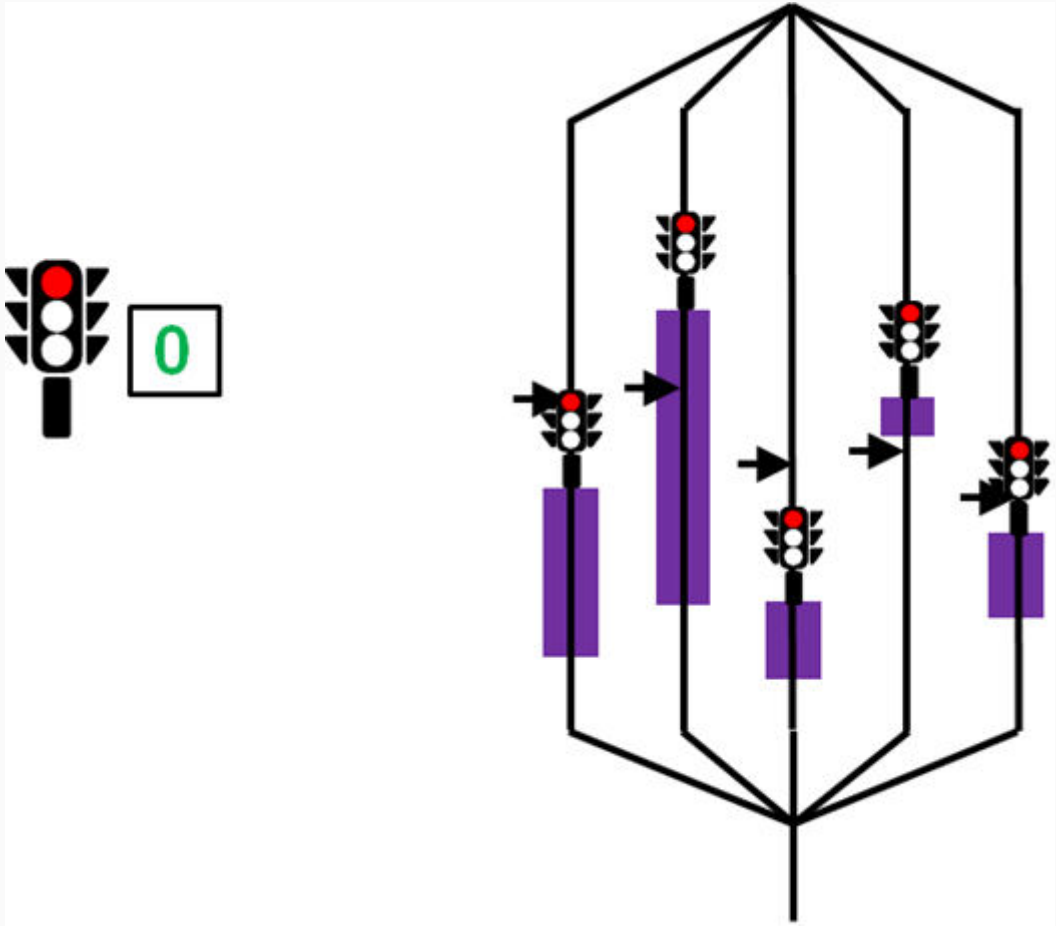

1

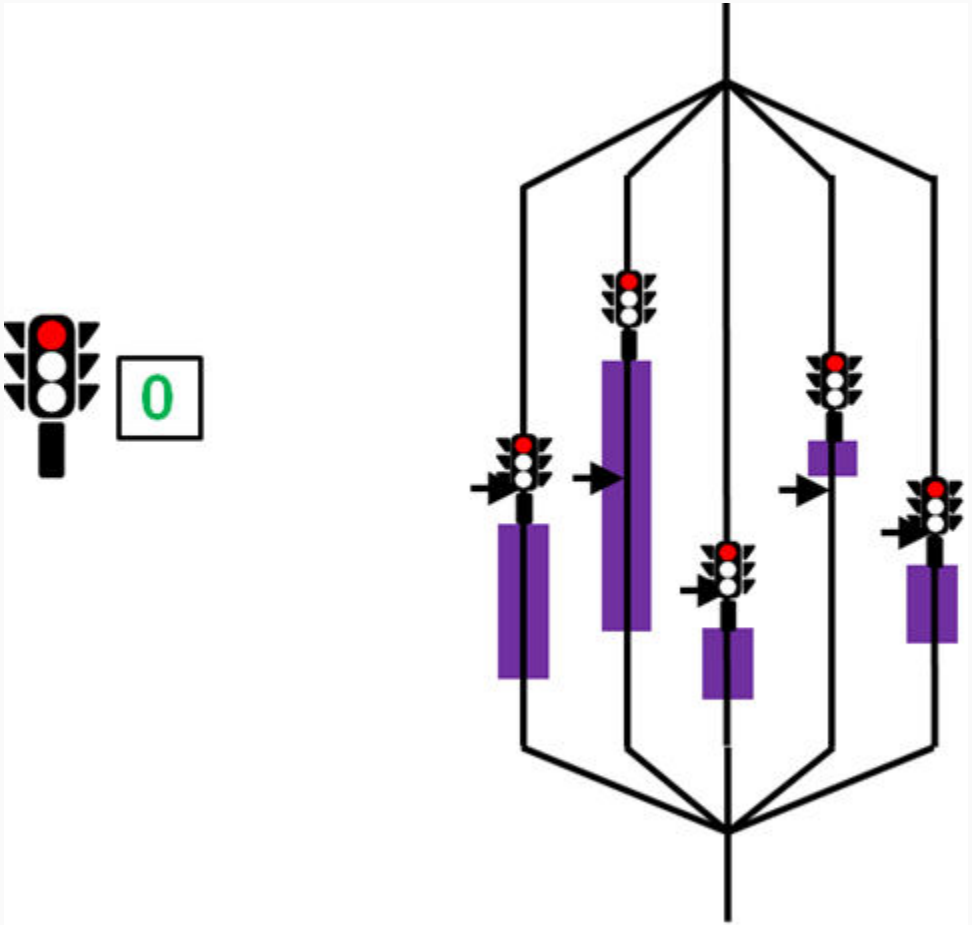

0

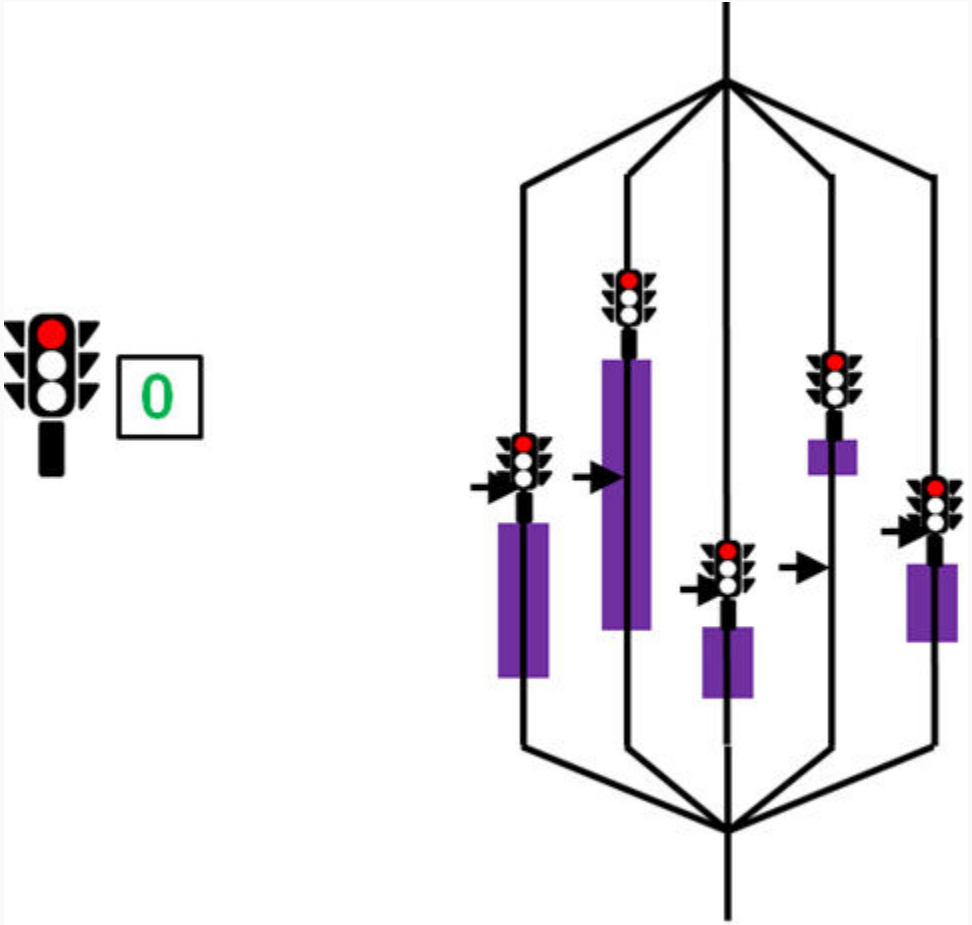

0

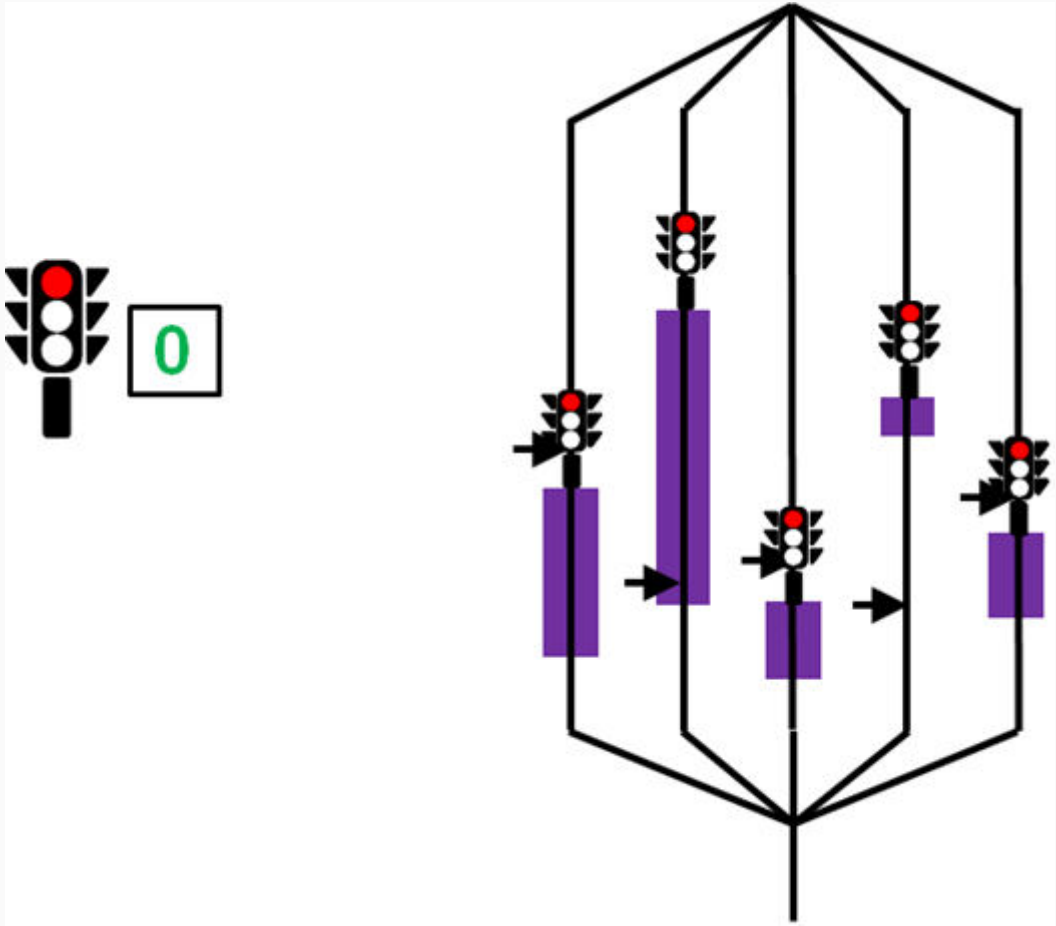

0

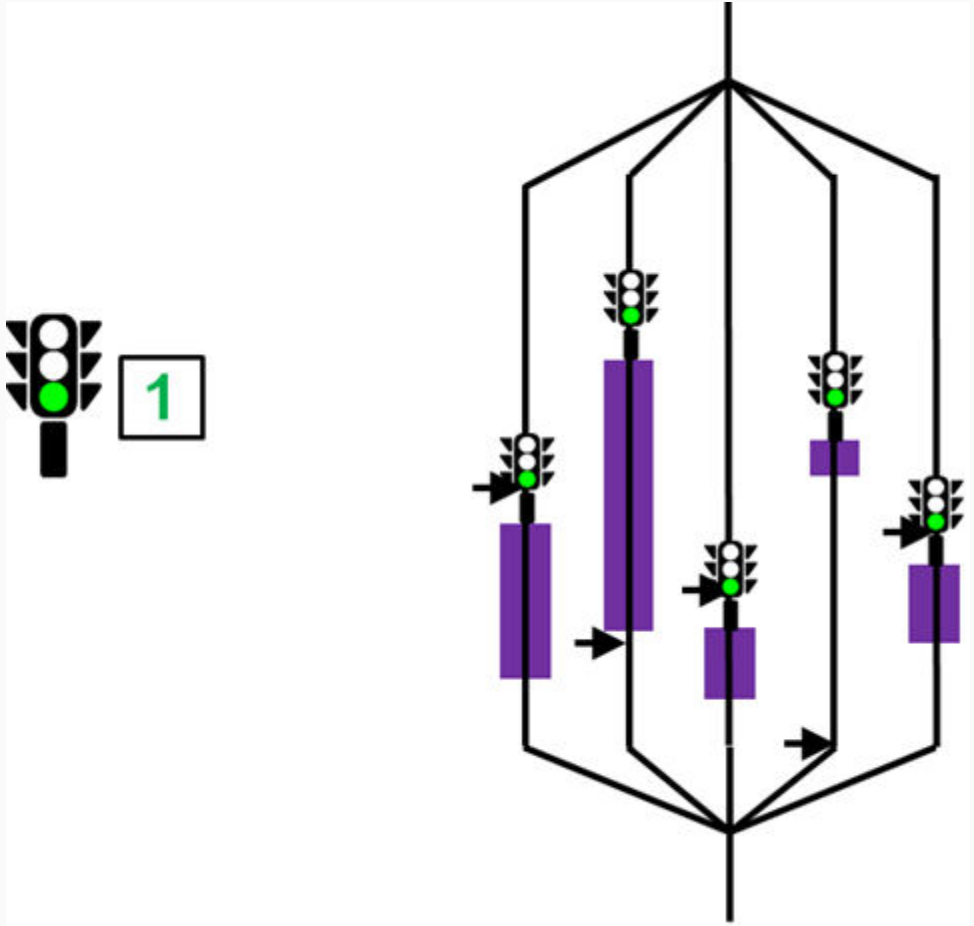

0

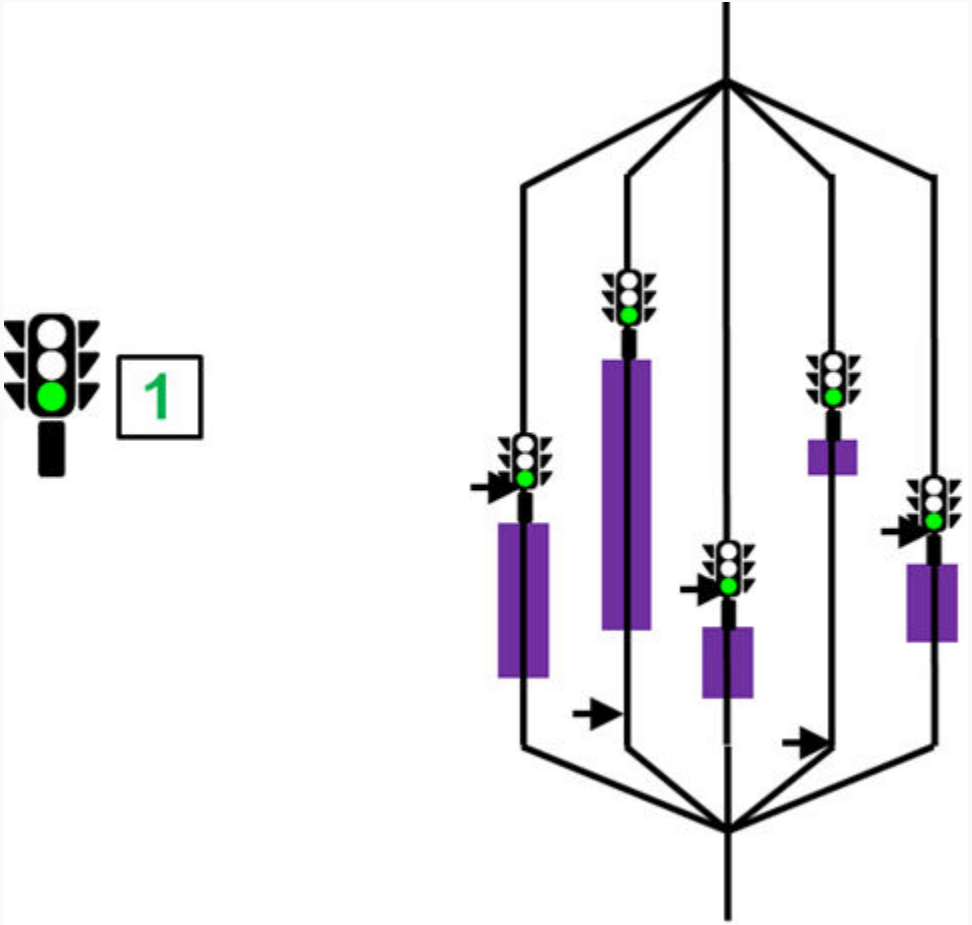

0

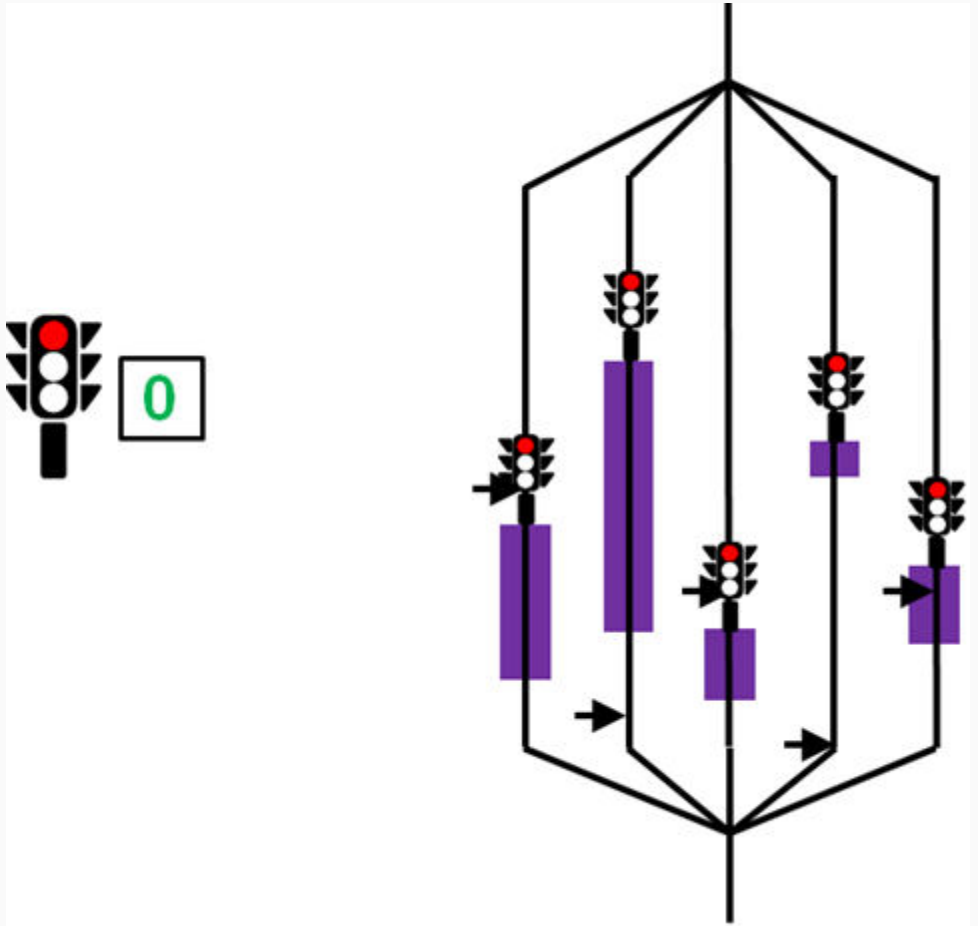

0


0


0


1


1


0

Producer consumer

- avem două tipuri de thread-uri:
  - producător - inserează date într-un buffer
  - consumator - extrage date din acel buffer, care are o dimensiune stabilită
- condiționări:
  - producătorul nu poate pune date într-un buffer plin
  - consumatorul nu poate extrage date dintr-un buffer gol
  - producătorul și consumatorul nu pot acționa simultan asupra bufferului -> mutex / synchronized

Producer

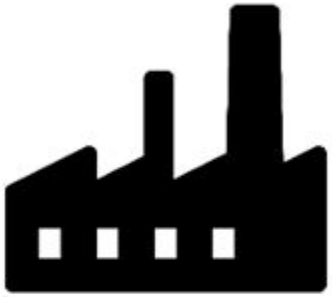
Buffer

Consumer



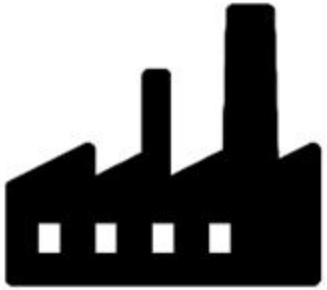








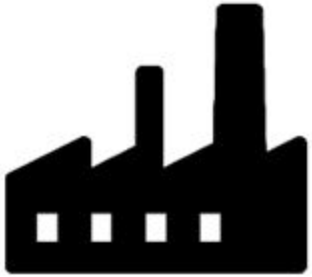








Can**not** produce
while buffer **full**
(must wait)







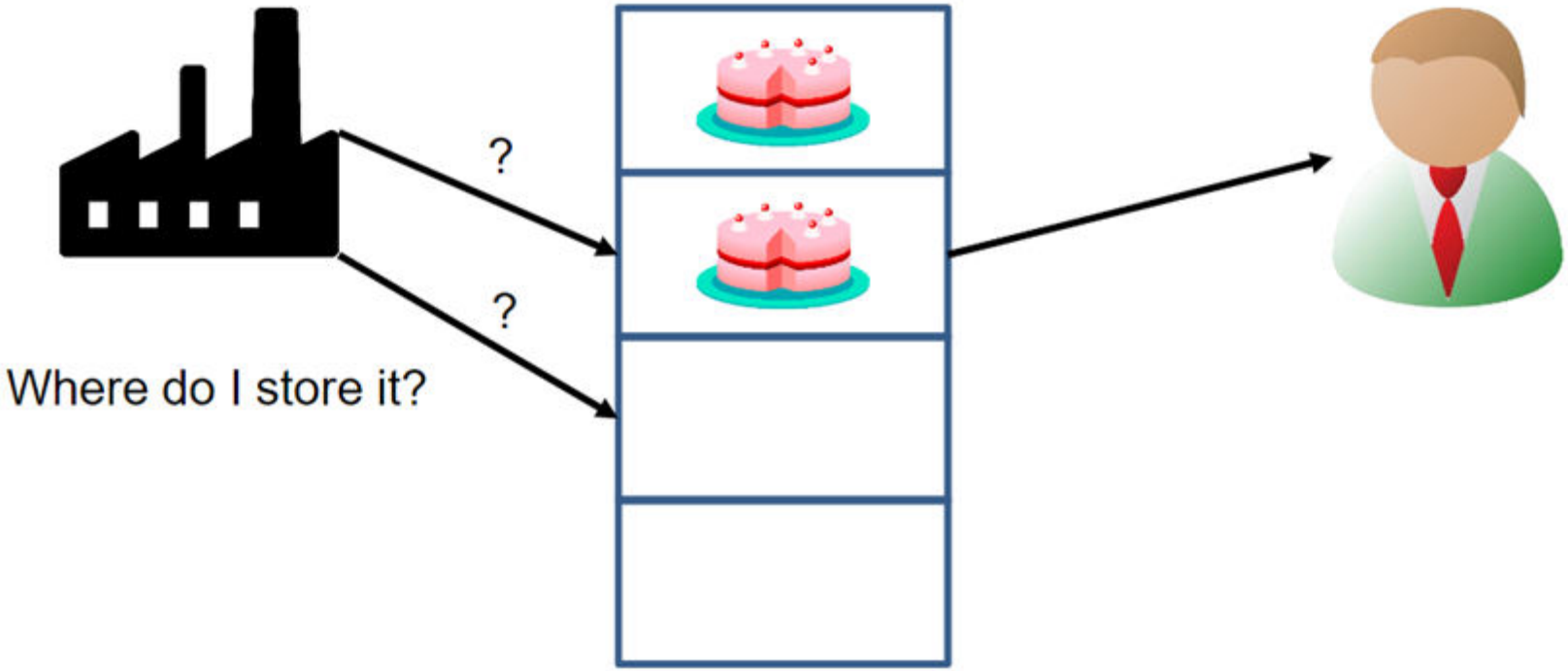




Can**not** consume while buffer **empty** (must wait)



Cannot produce while consuming

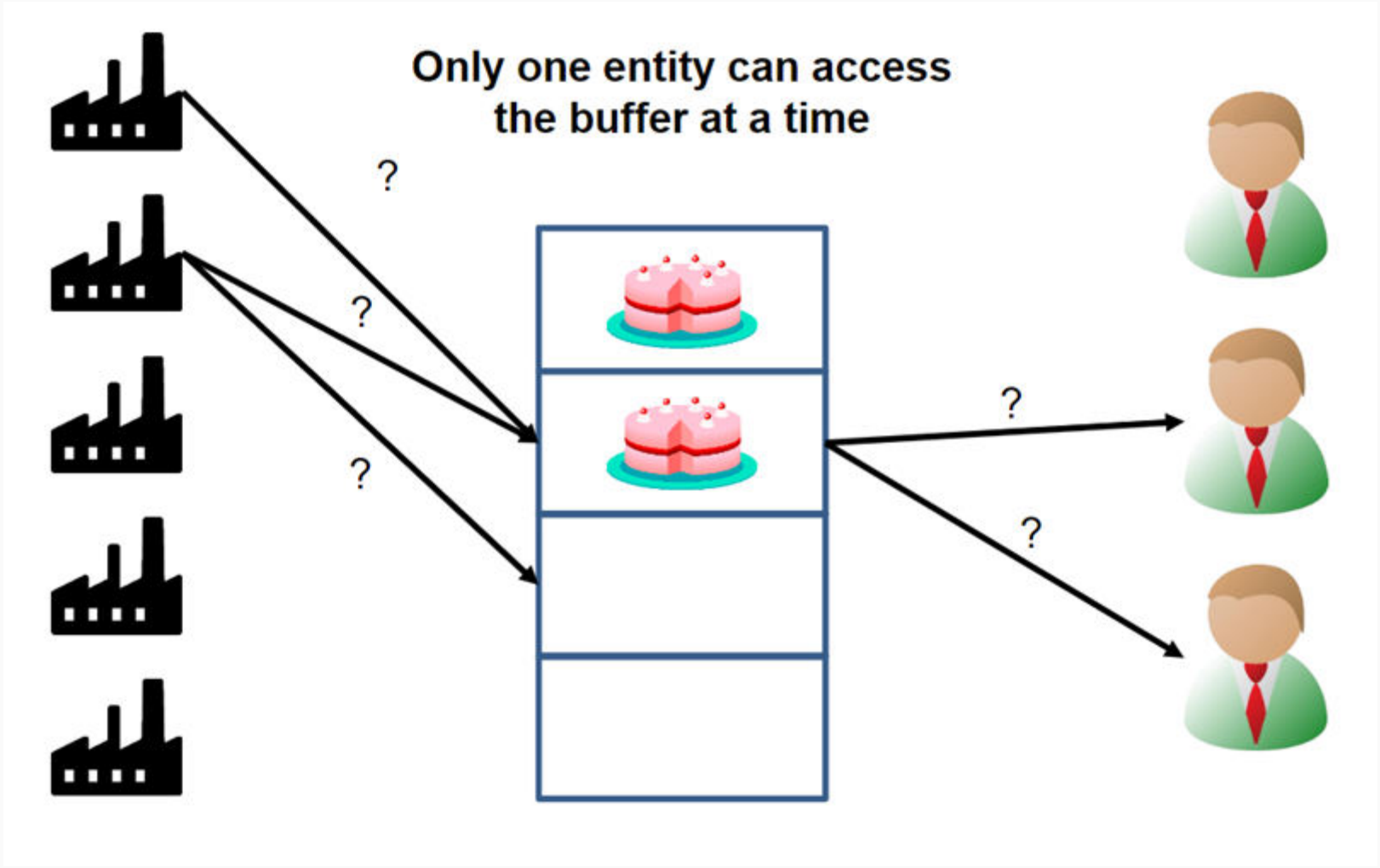


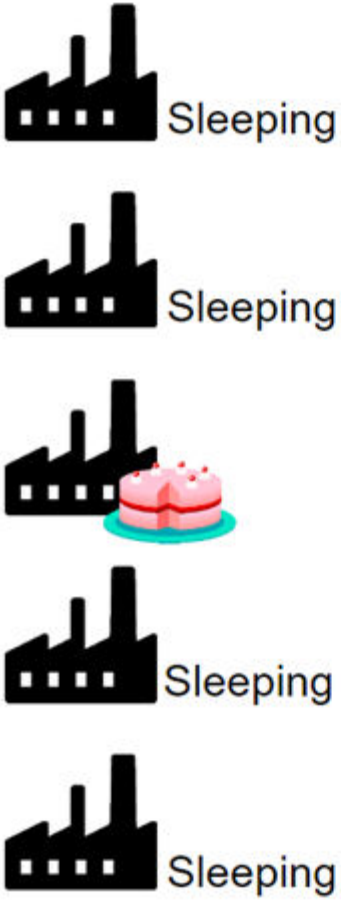
Does not known when the other works
(asynchronous)








Only one entity can access
the buffer at a time
?
?
?
?
?
?



If using a signaling system make sure you signal the right entity






Sleeping


Sleeping




Sleeping


Sleeping

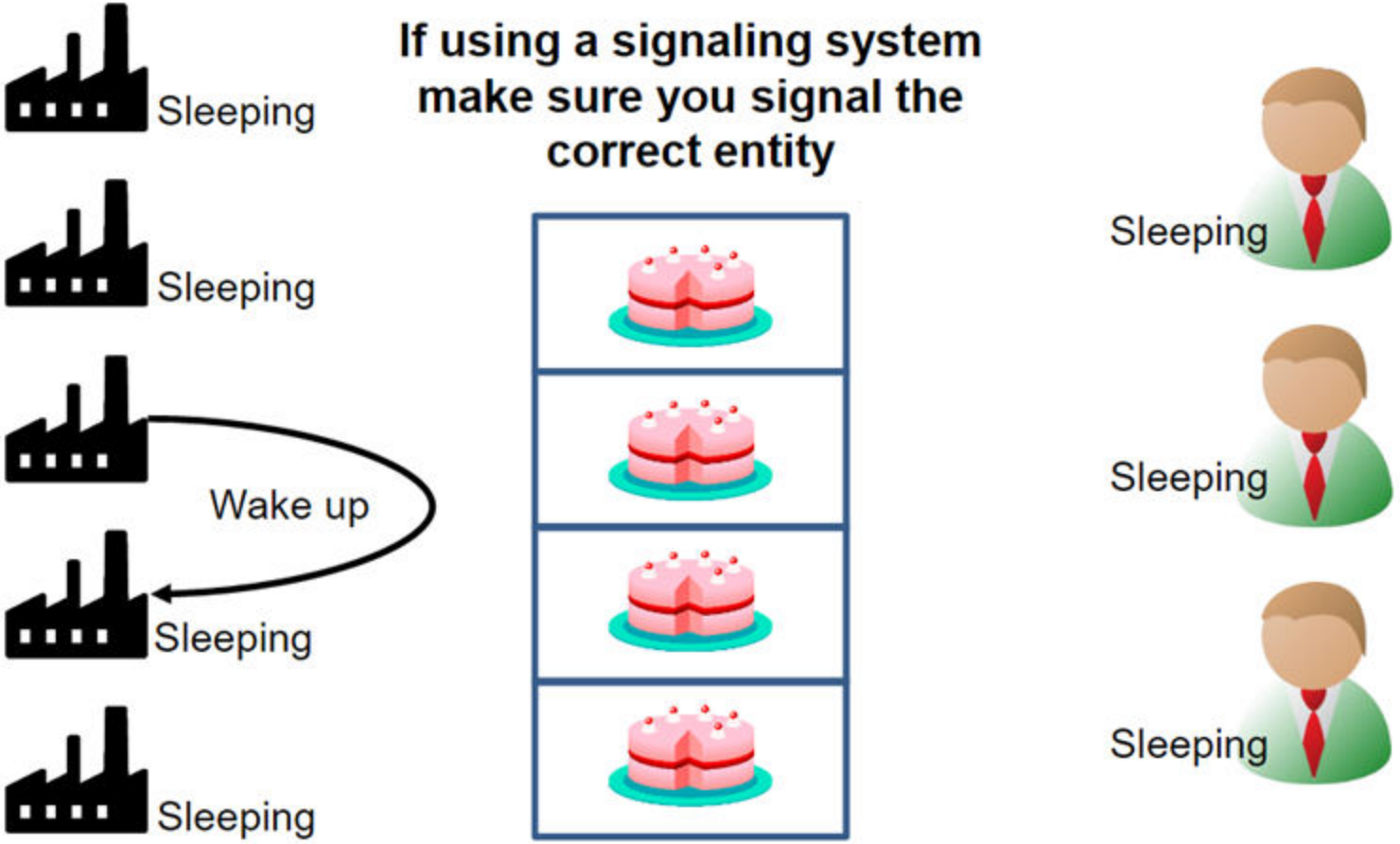
If using a signaling system make sure you signal the correct entity




Sleeping


Sleeping


Sleeping


Sleeping
Sleeping
Wake up
Sleeping
Sleeping
If using a signaling system make sure you signal the correct entity
Sleeping
Sleeping
Sleeping


Sleeping

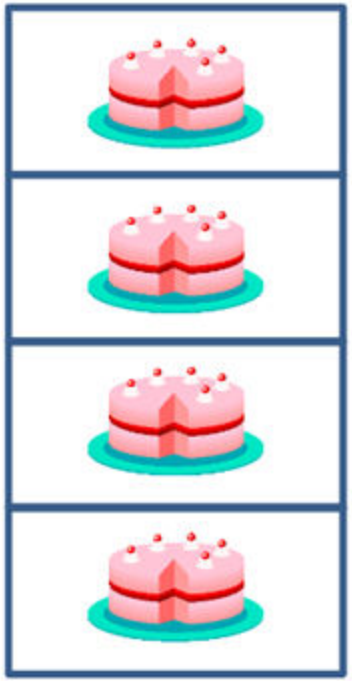

Sleeping


Sleeping




Sleeping

If using a signaling system make sure you signal the correct entity




Sleeping


Sleeping


Sleeping











If using a signaling system make sure you signal the correct entity





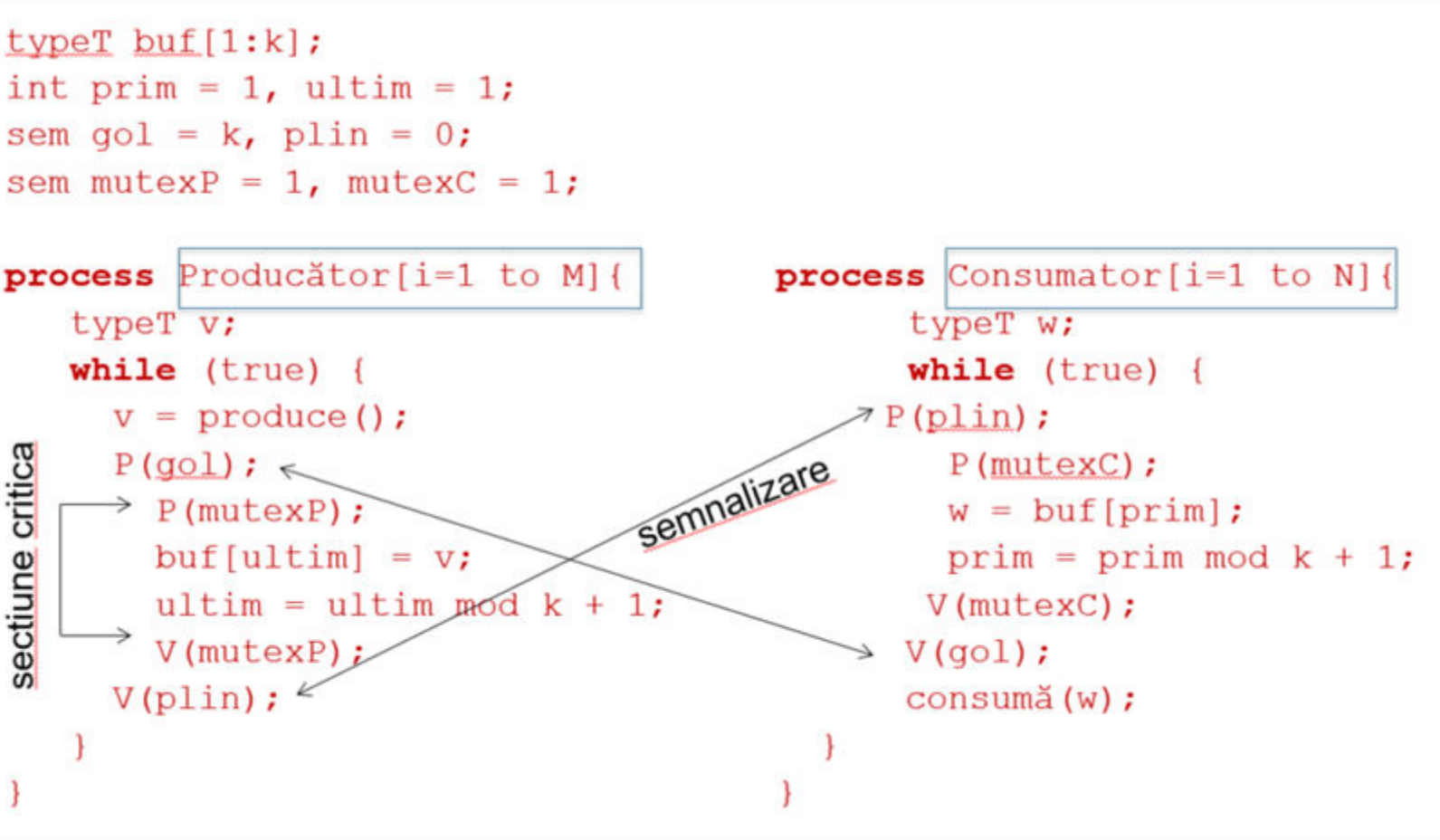




P	C

while (B.isFull()) {
    wait();
}
produce();
notify();
	while (B.isEmpty()) {
	    wait();
	}
	consume();
	notify();

Implementare cu wait-notify - one producer one consumer - se aplică și în cazul general, cu notifyAll() în loc de notify()

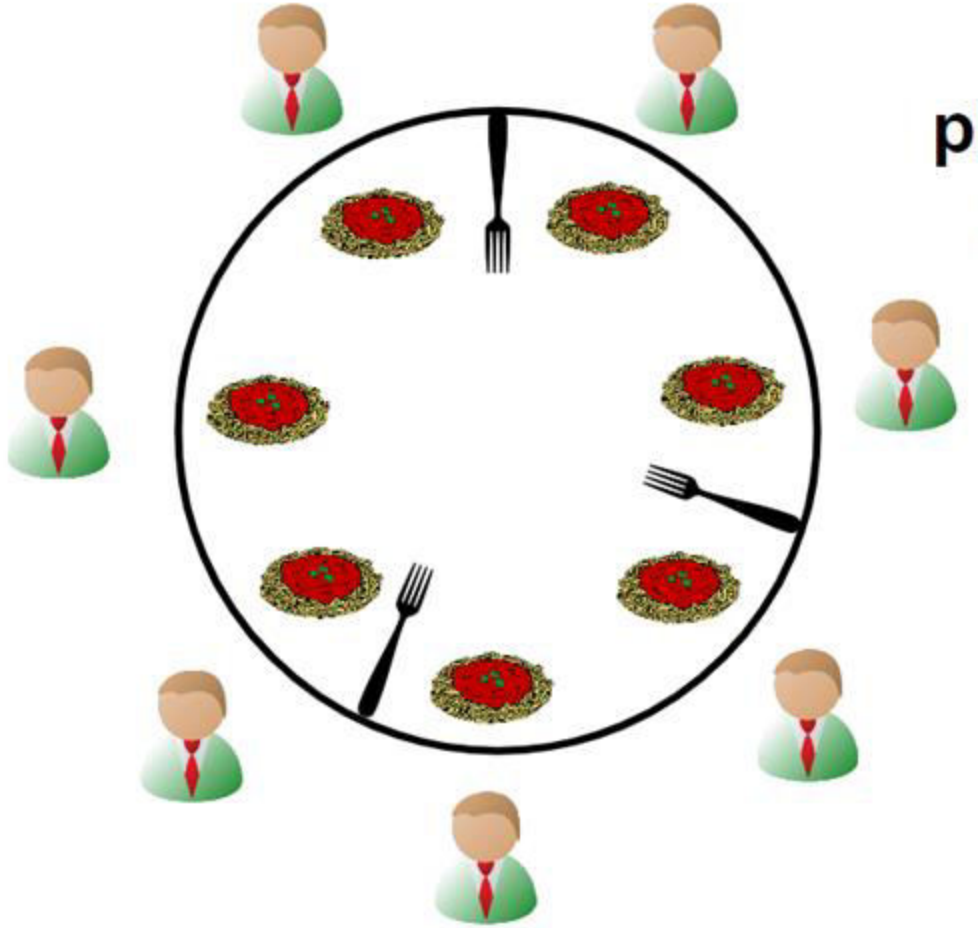


Implementare cu semafoare - multiple producers multiple consumers

Problema filosofilor

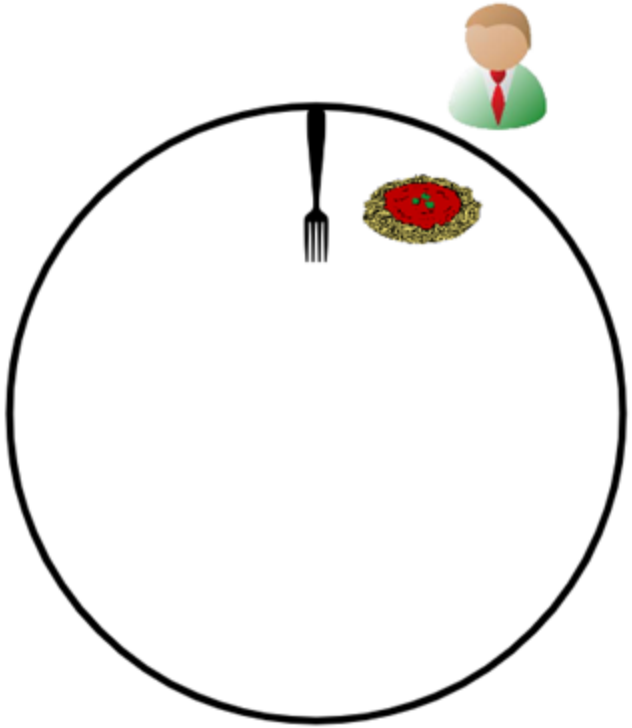
- avem n filosofi (threads) puși la o masă circulară, unde sunt puse n farfurii și n furculițe (locks), astfel încât fiecare filosof are două tacâmuri (unul în stânga, altul în dreapta) pentru a mânca
- soluție: n - 1 filosofi iau mai întâi furculița dreaptă, apoi pe cea stânga, iar al n-lea filosof va lua invers (mai întâi stânga, apoi dreapta)

The forks are locks



The philosophers are threads

The forks are locks

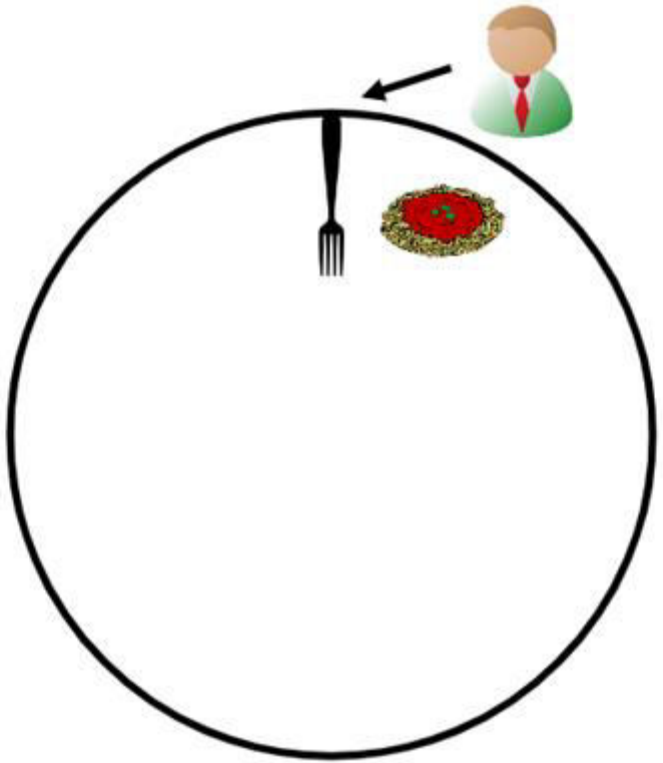


A philosopher wants to eat.

Eat is a task that required both forks (**locks**).

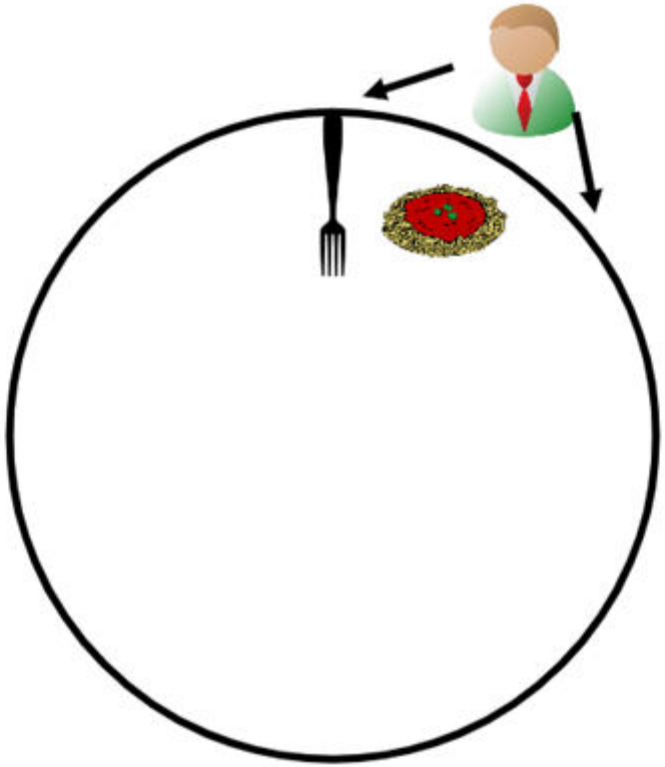
Eat could simply mean displaying a message.

The forks are locks



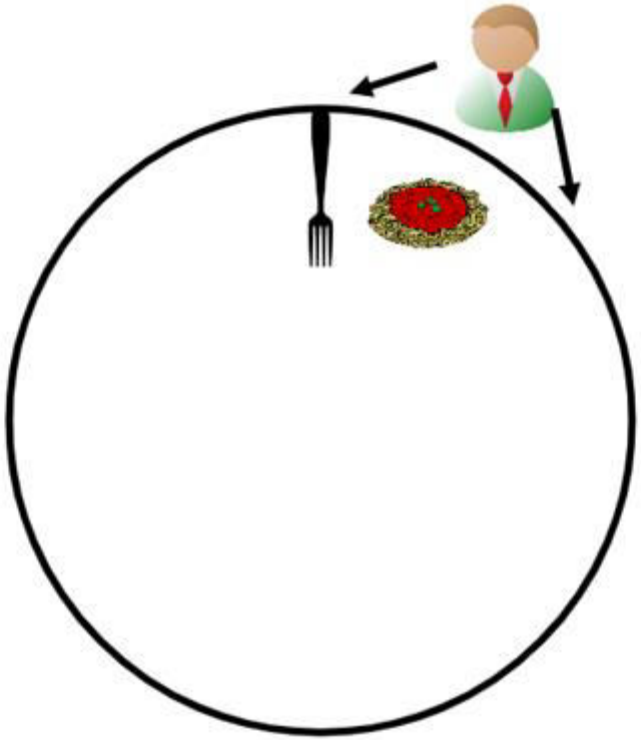
Take right fork (lock)

The forks are locks



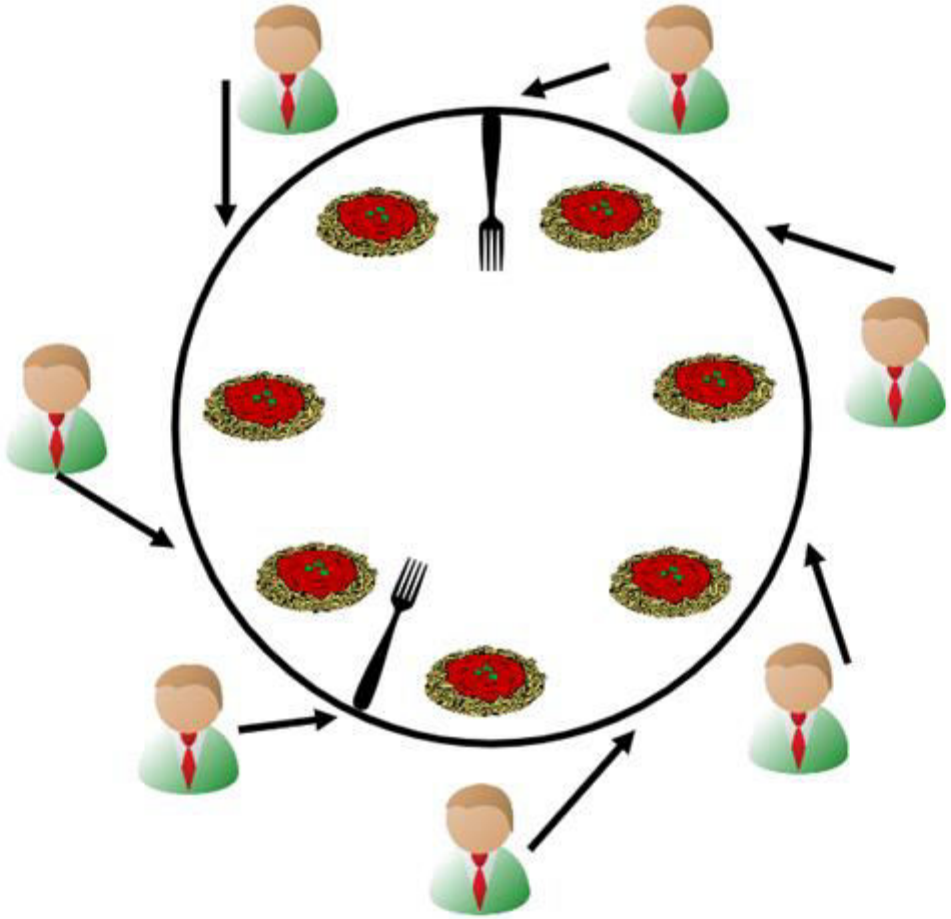
While holding right fork (lock) Take the left fork (lock)

The forks are locks

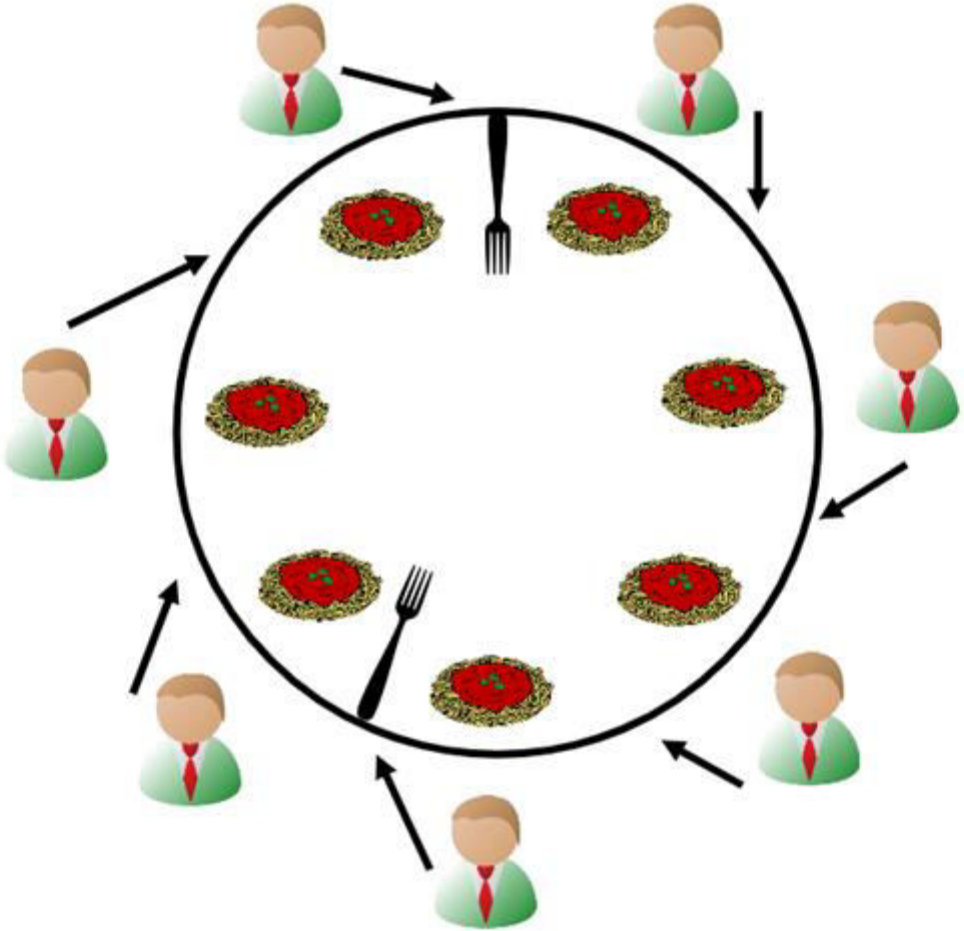


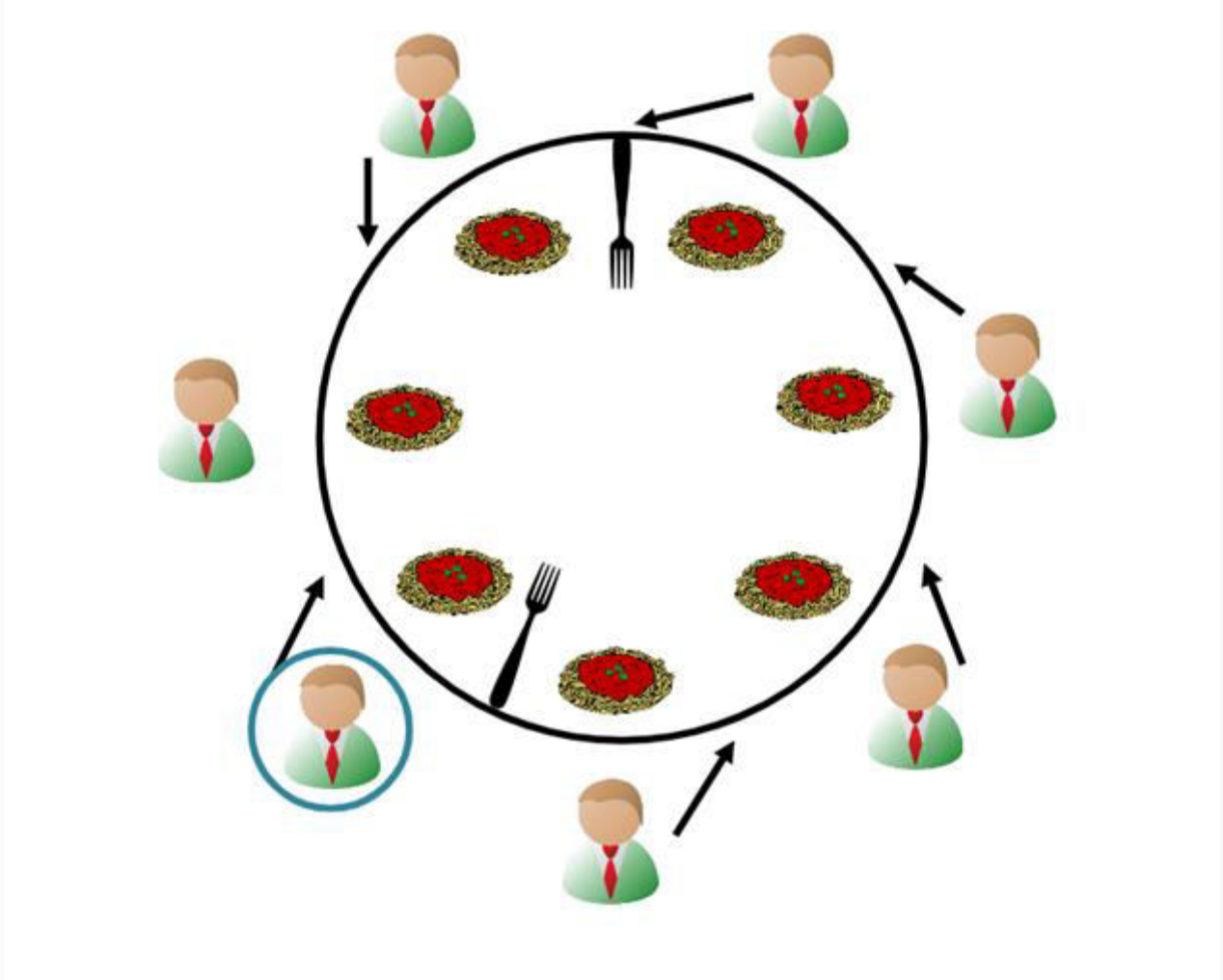
Now he can eat. Then he will let go of the forks (locks). This means someone else can take them.

If they all get the right fork (lock) first, they can all get stuck (we have a dead-lock)

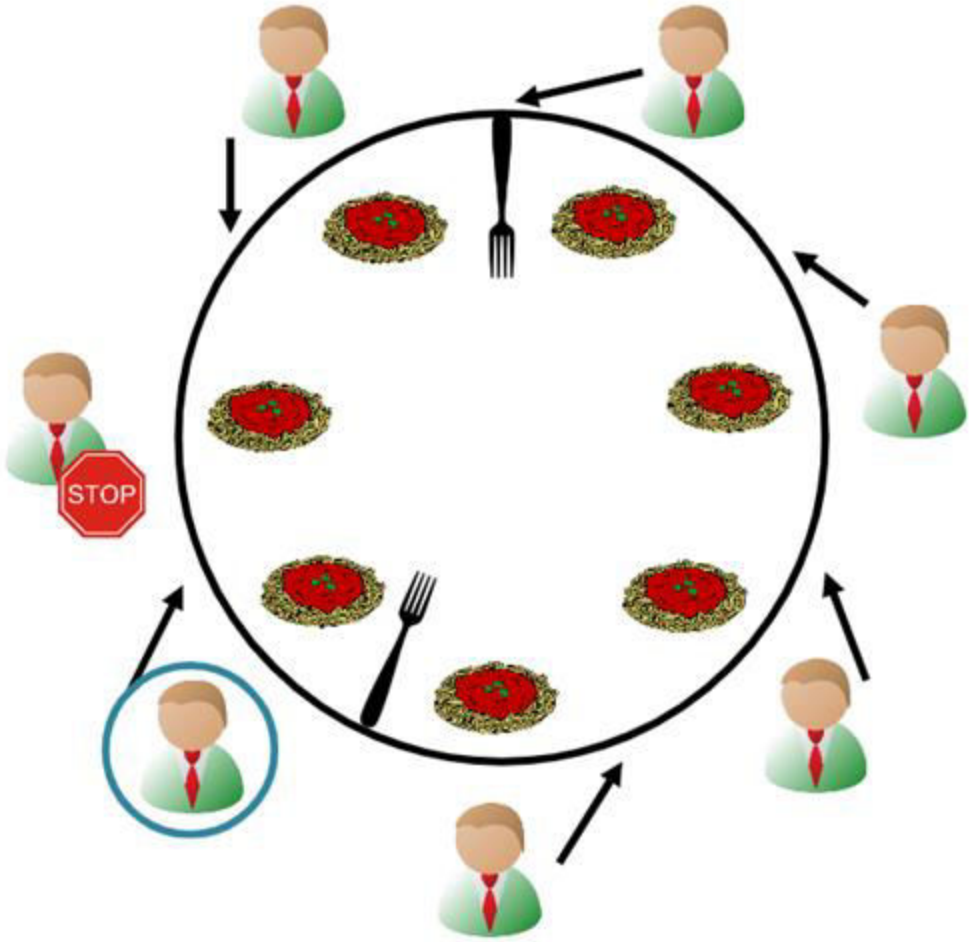


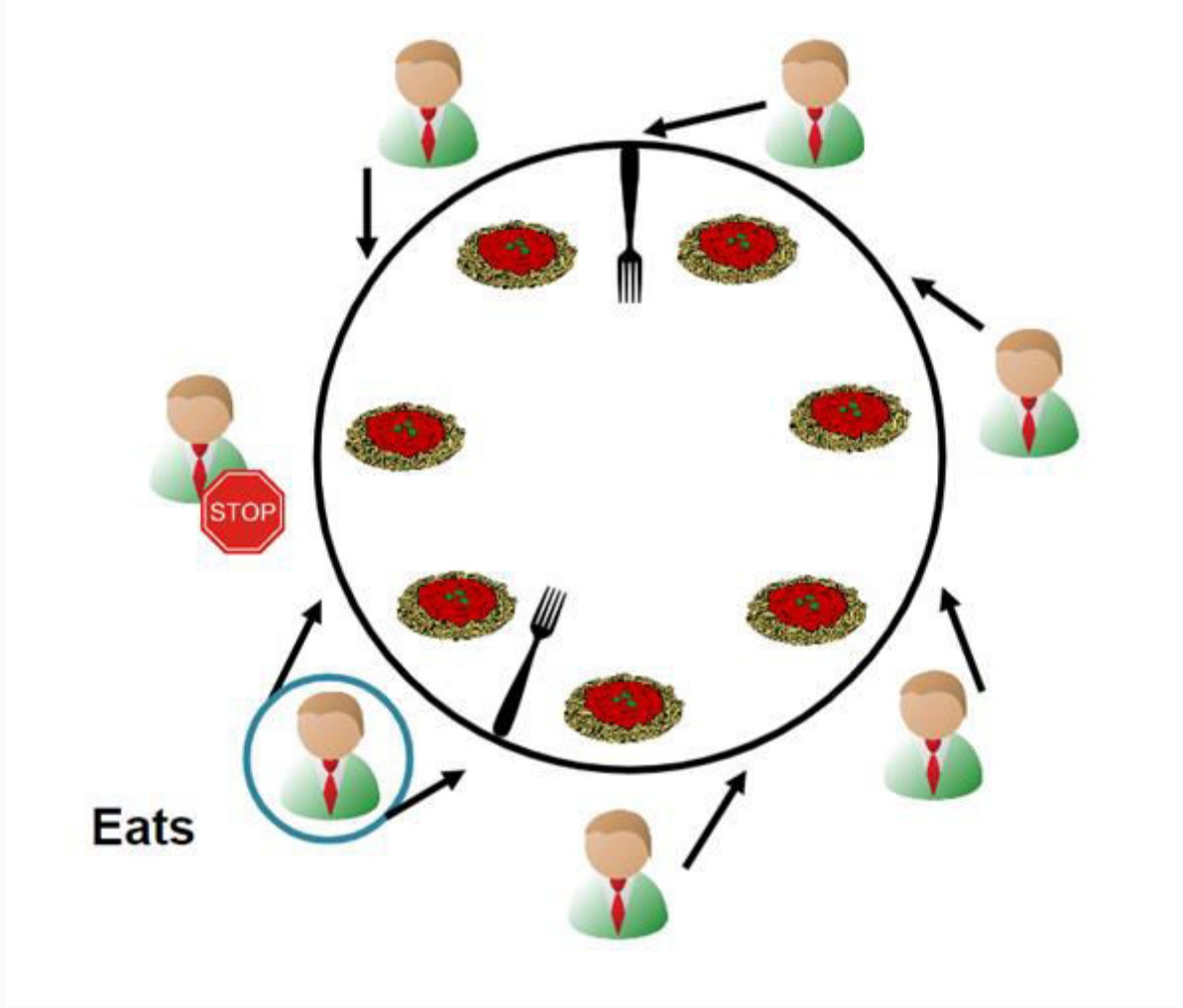
If they all get the left fork (lock) first, they can all get stuck (we have a dead-lock)

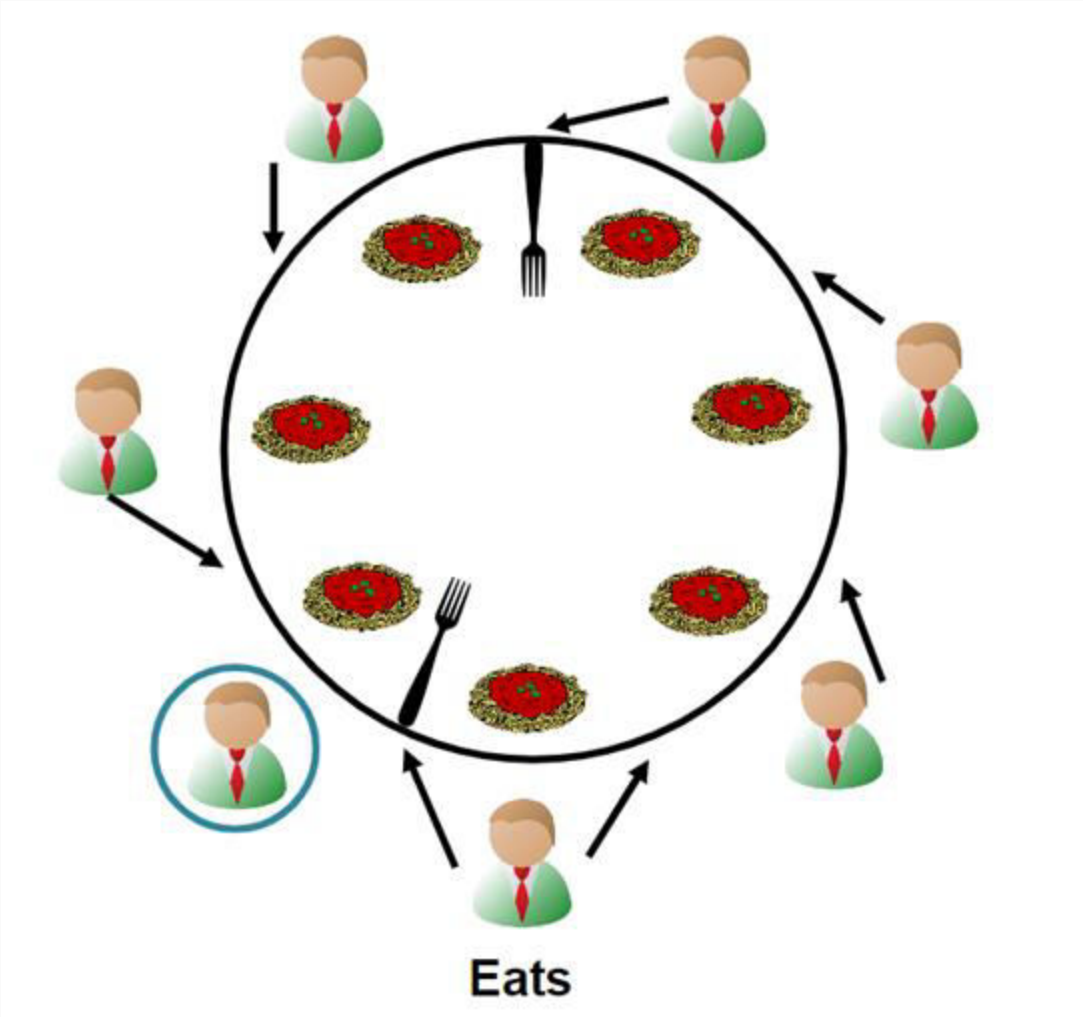


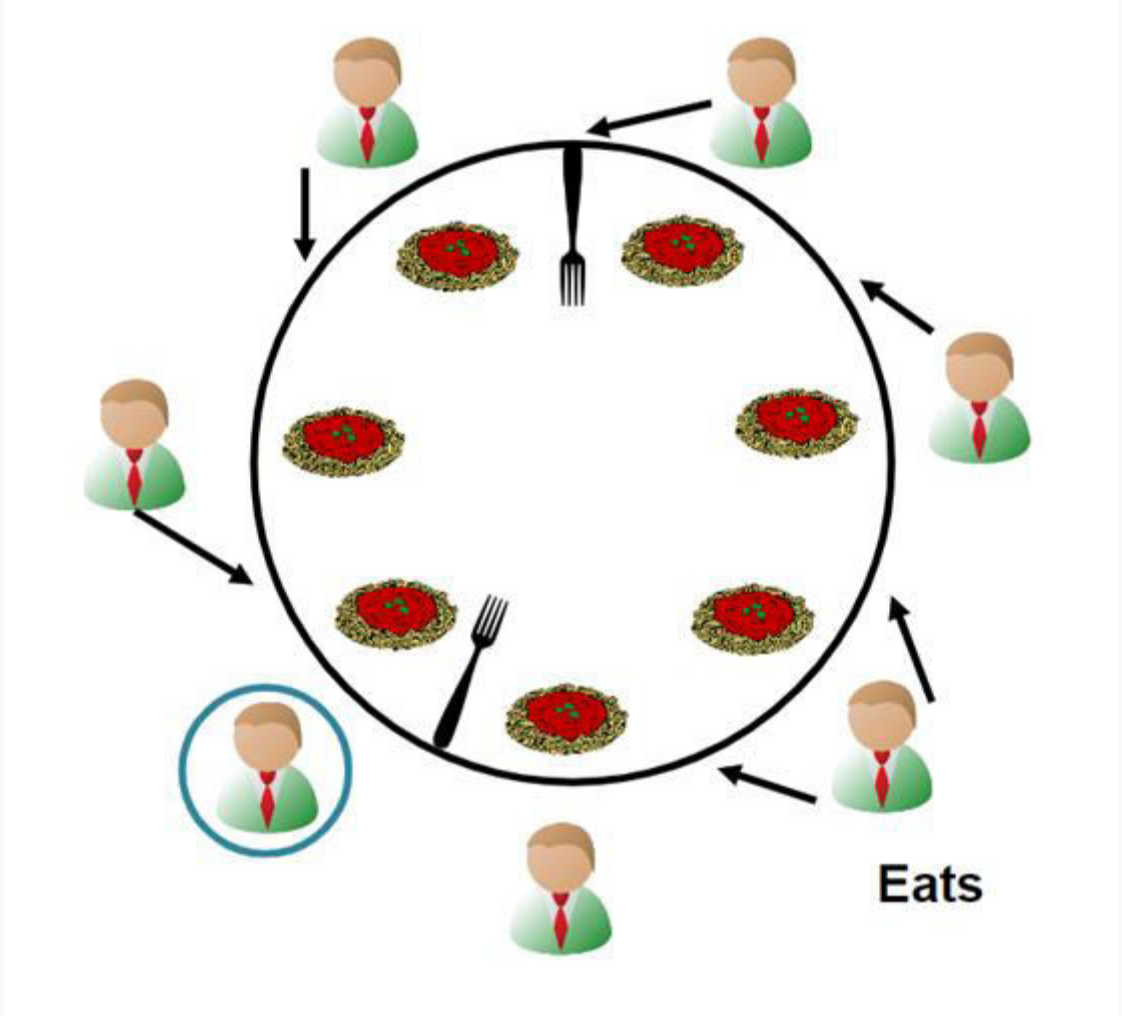


Can not take
any fork (lock)




STOP
Eats

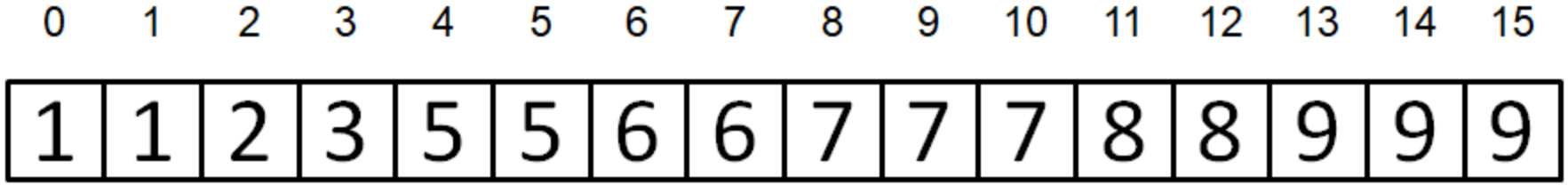

Eats


Eats

Căutare binară paralelă

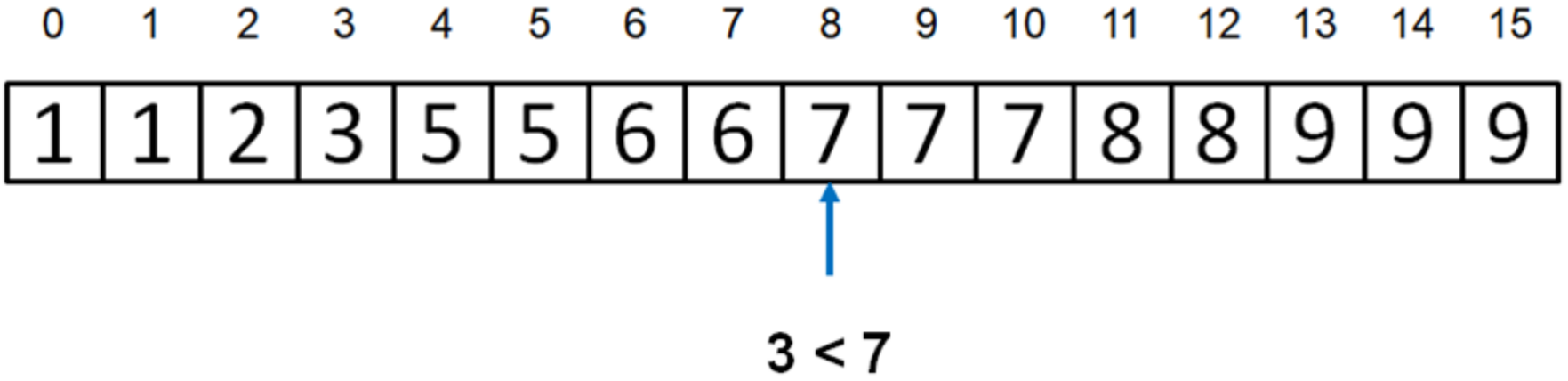
- șirul în care se caută dacă un element există trebuie să fie sortat
- se împarte șirul în P bucăți egale (aproximativ)
- fiecare thread verifică dacă elementul căutat se găsește între capetele bucății sale
- la pasul următor, se merge doar pe subșirul unde se găsește (eventual) elementul căutat
- pseudocod

Searching for 3



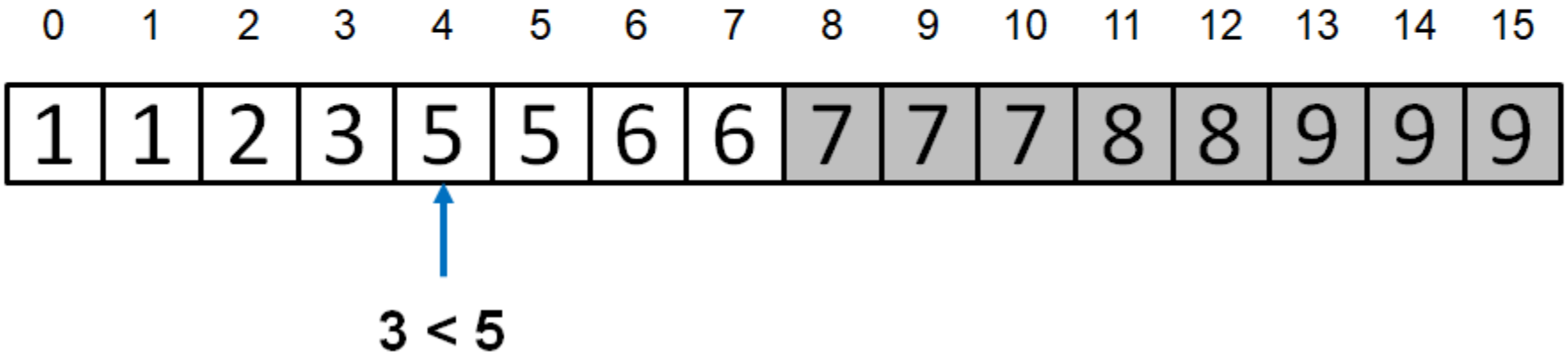
Searching for 3

Interest area 0 15



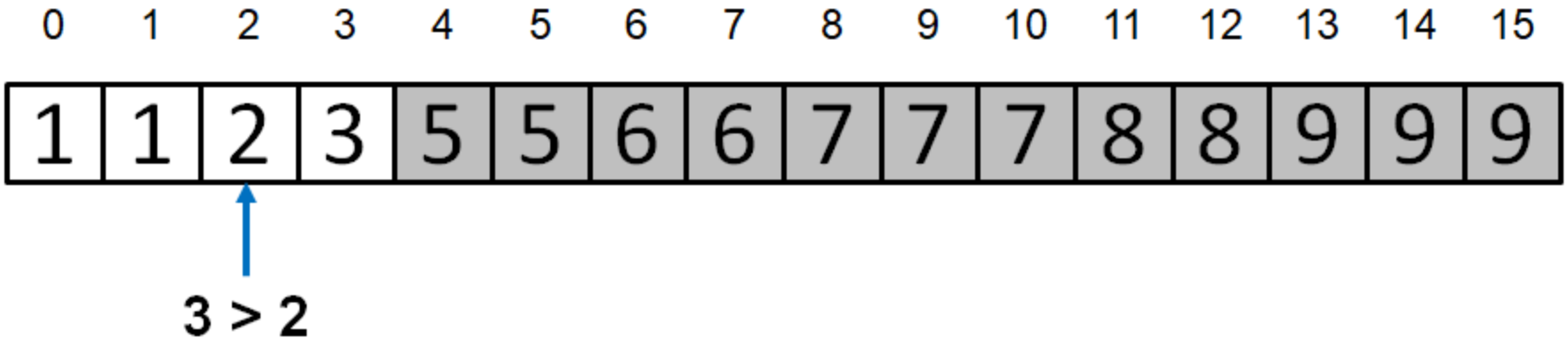
Searching for 3

Interest area 0 7



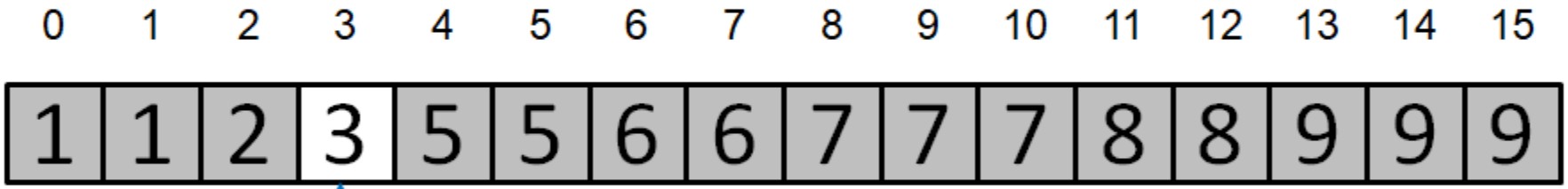
Searching for | 3 |

Interest area | 0 | 3 |



Searching for 3

Interest area 3 3

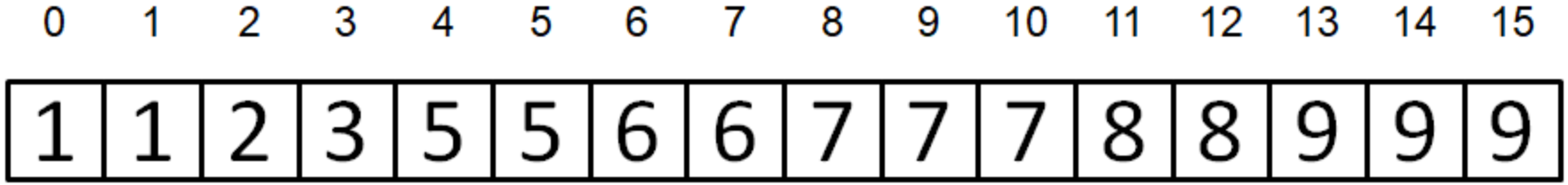


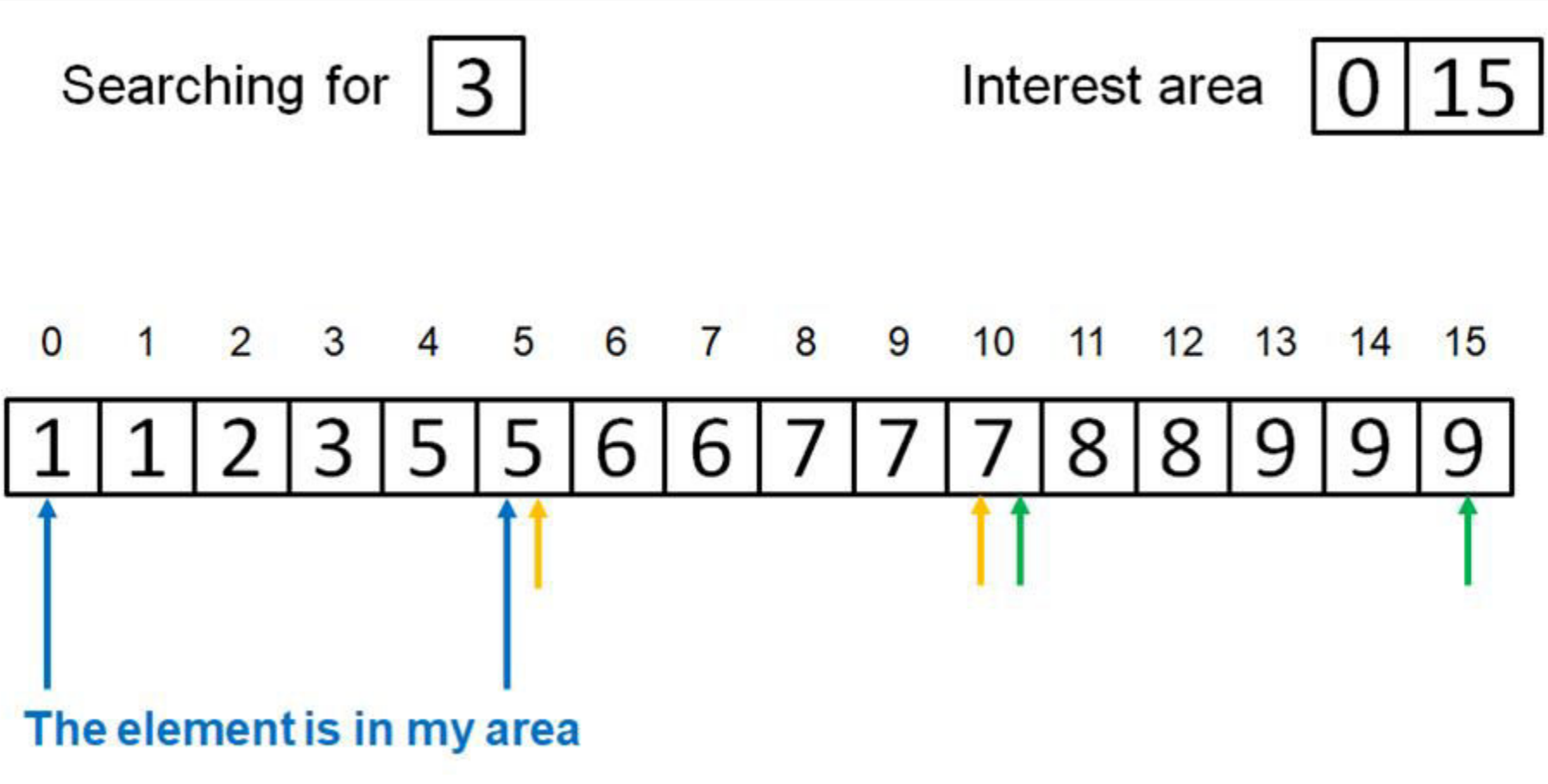
3 = 3

end

O($log_2(n)$) time

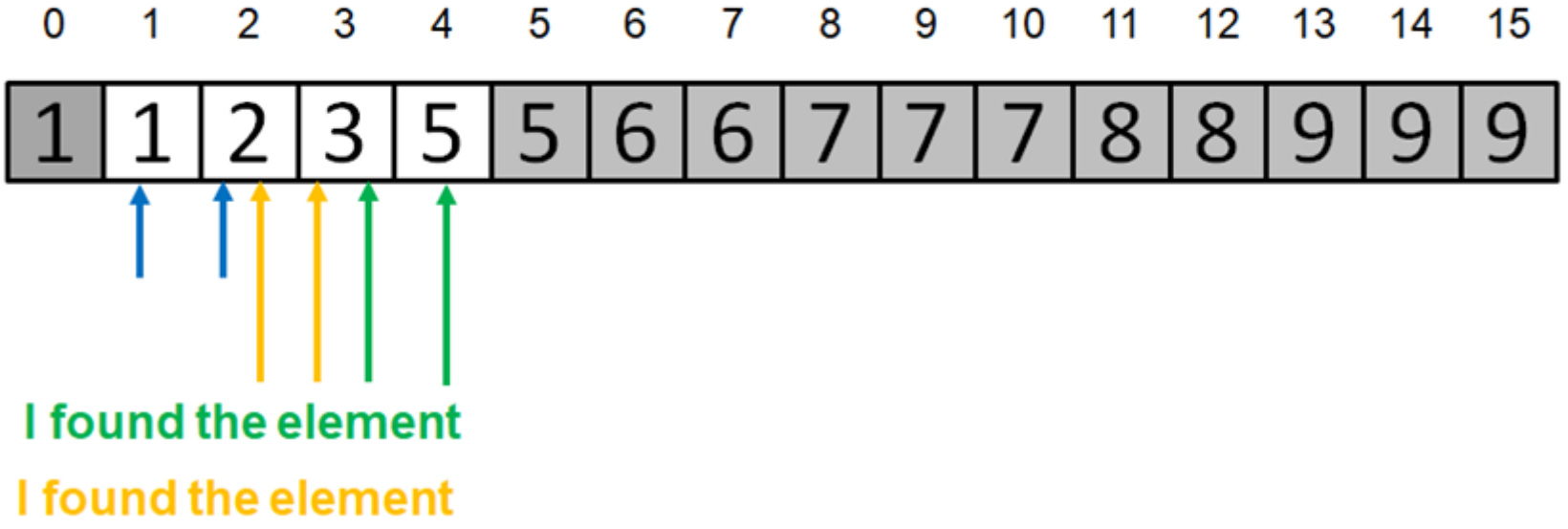
Searching for 3

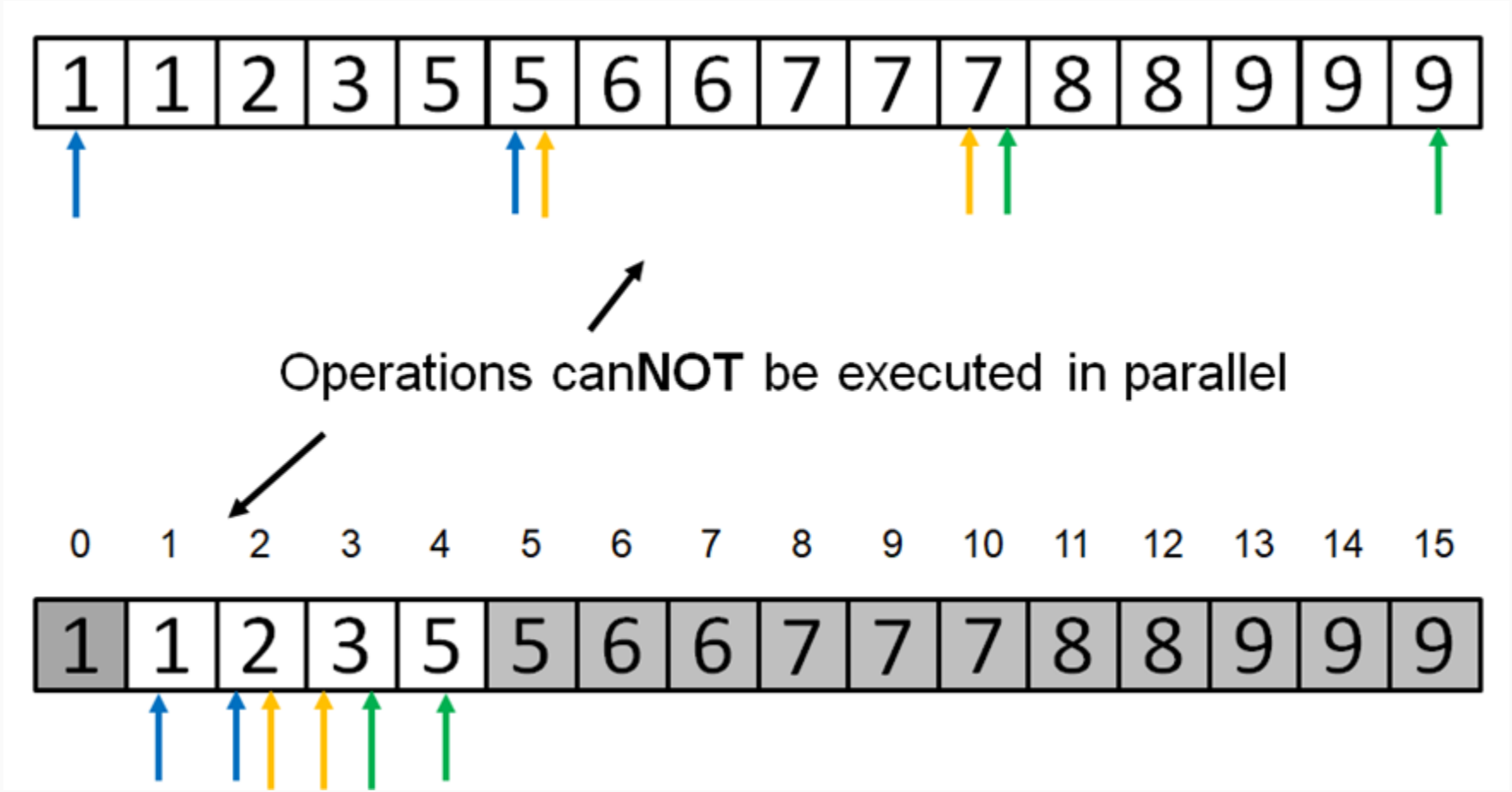


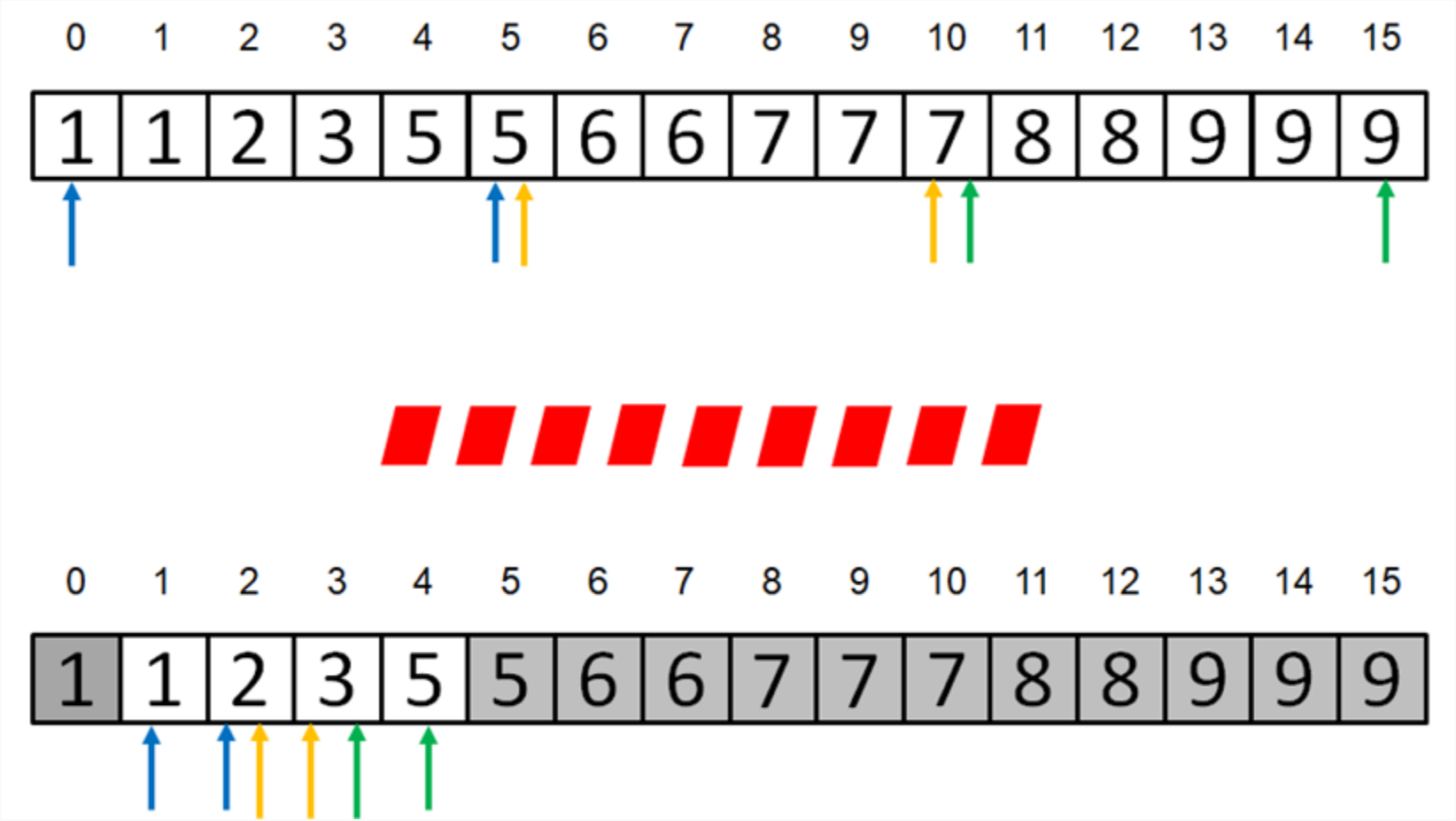

Searching for 3
Interest area 0 15
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
The element is in my area

Searching for 3

Interest area 1 4

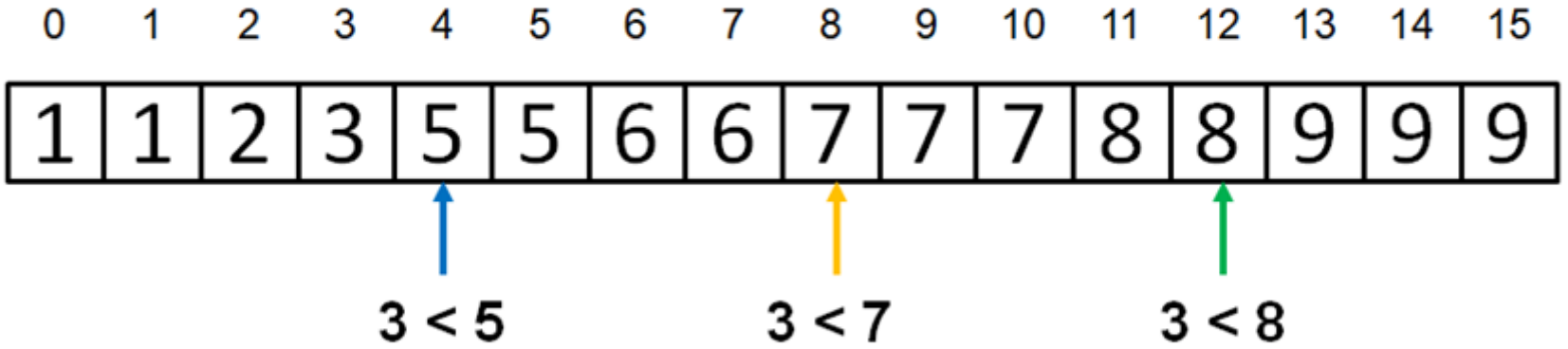



1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
Operations canNOT be executed in parallel
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9

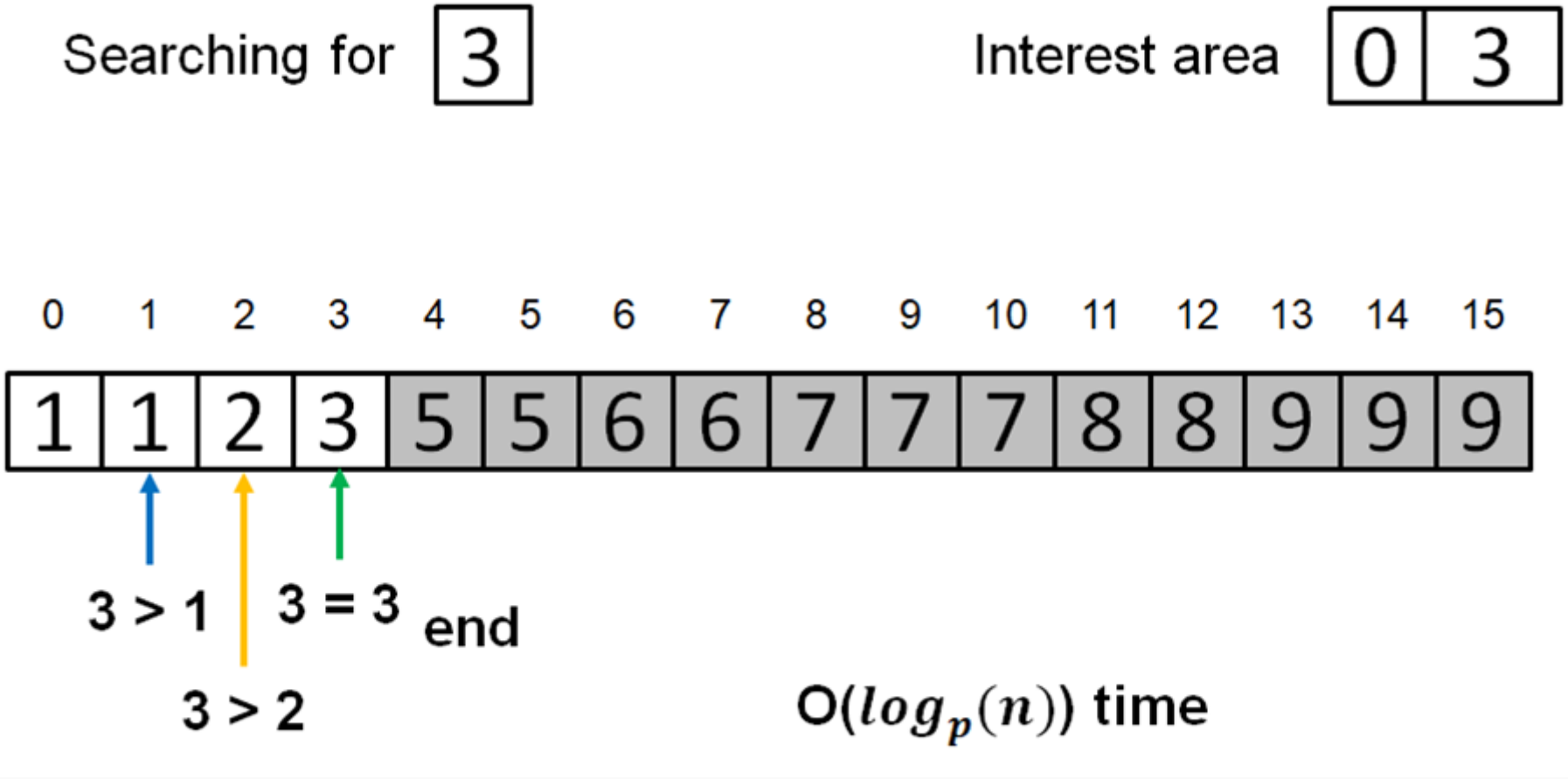

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9

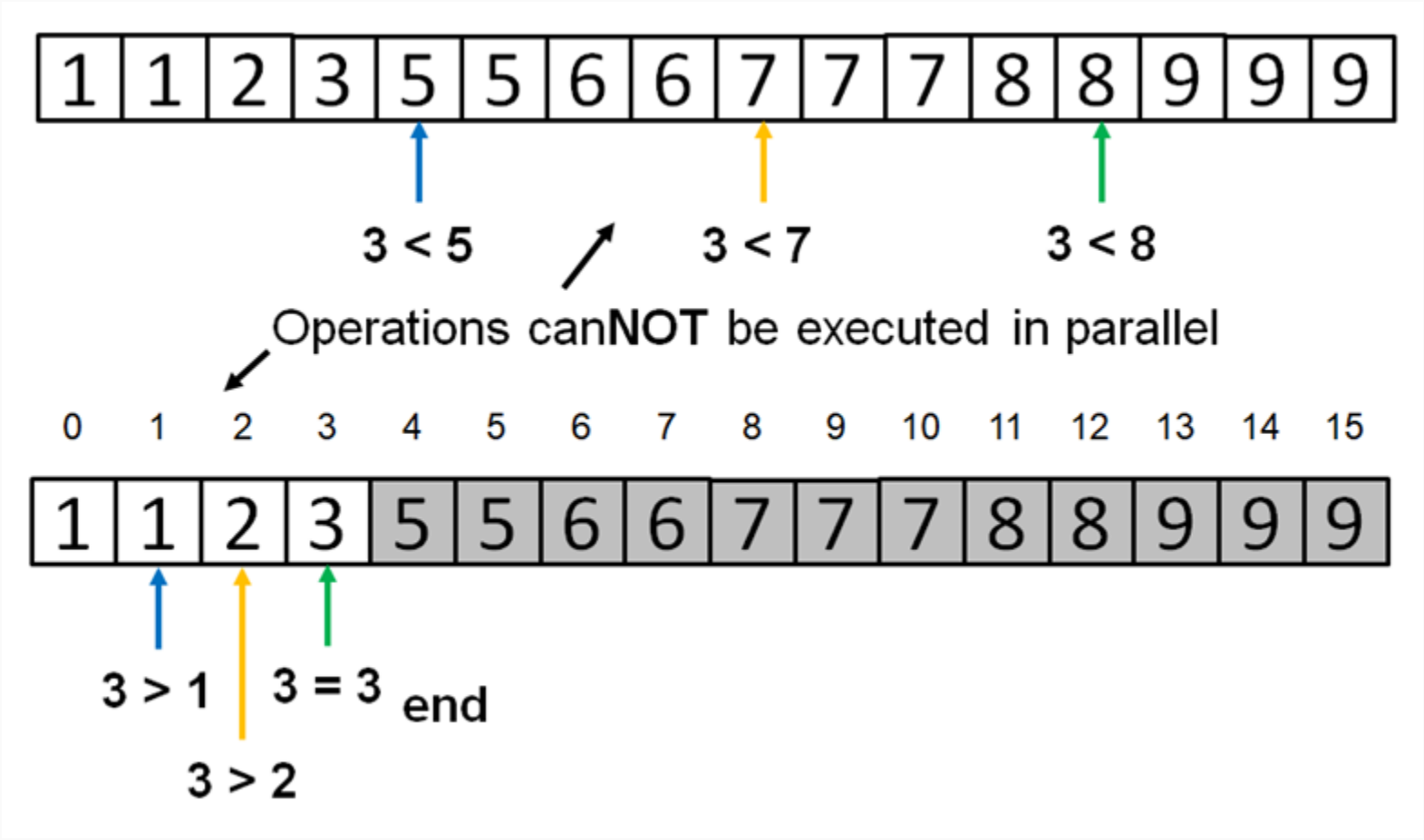
Searching for | 3 |

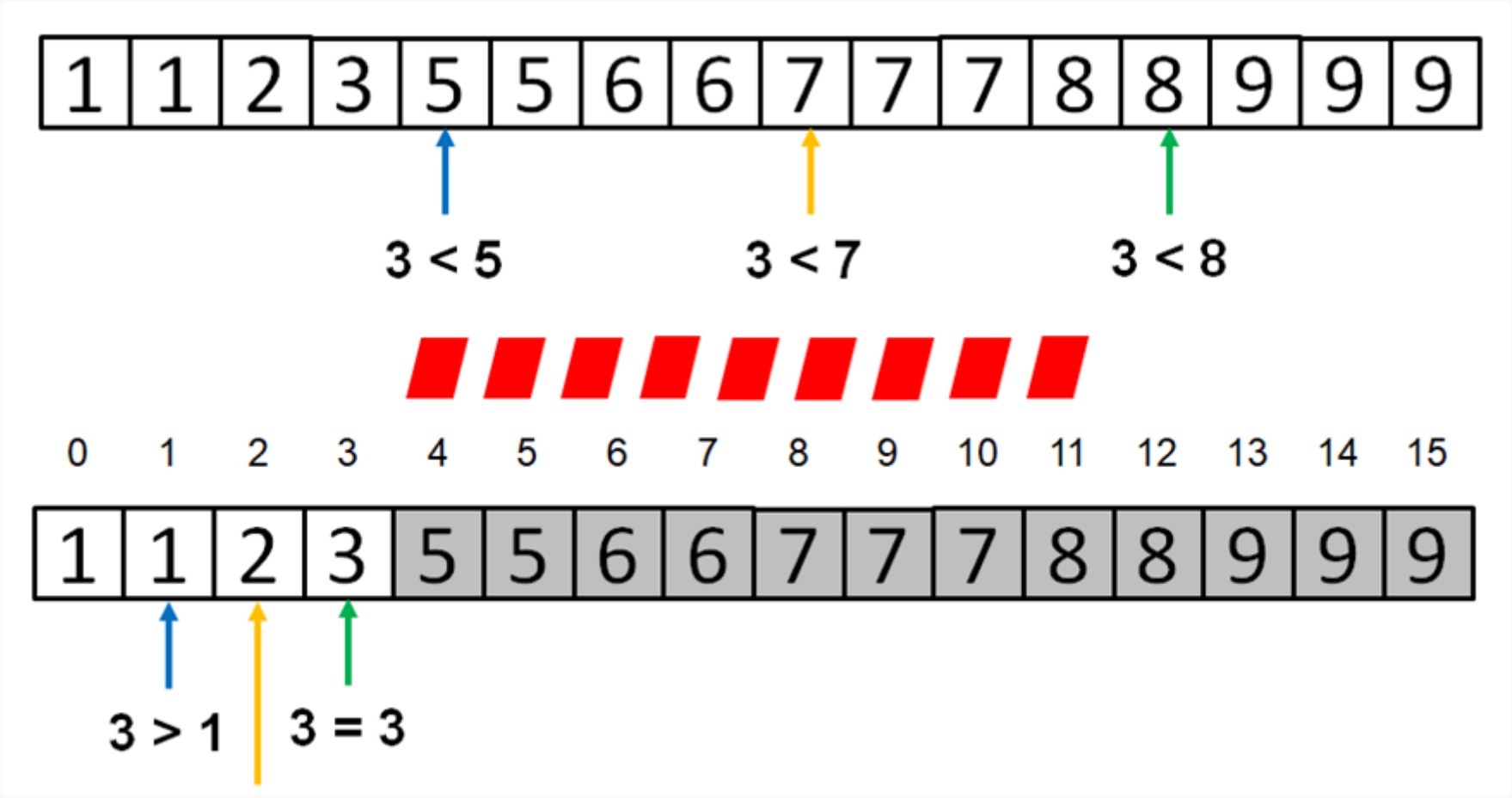
Interest area | 0 | 15 |



Operations can be executed in parallel


Searching for 3
Interest area 0 3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3
end
3 > 2
O($log_p(n)$) time


1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 < 5
3 < 7
3 < 8
Operations canNOT be executed in parallel
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3
end
3 > 2


1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 < 5
3 < 7
3 < 8
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3

Resurse

- schelet laborator updated

Întrebări

- ce reprezintă un deadlock? Dați un exemplu de caz / situație de deadlock
- modelați conceptul de producer consumer în viața reală

Laboratorul 6
Operații și structuri atomice



Semafor - remember lab 5

- `Semaphore sem = new Semaphore(x)`
- `acquire()` -> se decrementează x
- `release()` -> se incrementează x
- dacă x == 0, thread-urile care vor să acceseze zona critică așteaptă până când x > 0

Semafor - continuare

- la inițializarea semaforului, putem pune un număr negativ: `Semaphore sem = new Semaphore(-1)`
- dacă avem -1 la semafor și un thread vrea să între (acquire()), alte două thread-uri trebuie să facă release(), încât să avem valoare pozitivă, care face posibil un acquire()

Semafor - continuare

```
Semaphore sem = new Semaphore(-1);
```

T0	T1	T2

`// așteaptă la semafor`
`// după celelalte 2 thread-uri`
`sem.acquire();`
 	
`// sem = 1, deci poate trece`
`System.out.println(`
`"Am trecut de semafor!");`
	`do_work1();`
	`sem.release();`
	`// sem = -1 + 1 = 0`
		`do_work2();`
		`sem.release();`
		`// sem = 0 + 1 = 1`

AtomicInteger

- clasa AtomicInteger reprezintă o implementare atomică a clasei Integer (de care ne scapă de folosirea de locks, care este mai ineficientă ca timp de execuție - overhead de la acquire() și release() - demo)
- metode utile:
  - get() - întoarce valoarea de tip întreg
  - set(x) - îi setează valoarea la x
  - compareAndSet(y, x) - compară întregul cu x, iar, dacă cele două valori sunt egale, setează valoarea la y
  - addAndGet(delta) / getAndAdd(delta) - incrementează valoarea curentă cu delta și returnează valoarea nouă, respectiv pe cea veche

AtomicInteger - folosire

```
AtomicInteger nr = new AtomicInteger(4);

System.out.println(nr.get()); // 4

if (nr.compareAndSet(4, 7))

    System.out.println(nr.get()); // nr devine 7

System.out.println(nr.addAndGet(2)); // se aduna 2, deci devine 9

nr.set(1);

System.out.println(nr.get()); // 1
```

ConcurrentHashMap

- dacă am folosi un HashMap cu un singur lock pe întreaga structură, nu ar fi eficient deoarece doar un singur thread poate să acceseze HashMap-ul la un moment dat, chiar dacă thread-urile ar accesa elemente diferite
- dacă am folosi câte un lock pentru fiecare element, modul de lucru ar fi îngreunat
- soluție: ConcurrentHashMap - **nu admite chei null**

ConcurrentHashMap

- în ConcurrentHashMap, operațiile de get() și de put() blochează accesul la un anumit element printr-un lock (câte un lock pe fiecare element, deci nu e blocat accesul la întreaga structură) => două sau mai multe thread-uri pot face și get() și put(), dar avem garantat faptul că operațiile get() sunt executate după cele de put()

ConcurrentHashMap - metode

- `get(key)` - returnează valoarea asociată cu o cheie
- `contains(val)` - verifică dacă o valoare există în map
- `containsKey(key)` - verifică dacă o cheie există în map
- `put(key, val)` - adaugă o valoare la cheia specificată
- `putIfAbsent(key, val)` - adaugă o asociere doar dacă nu există deja o valoare pentru cheia respectivă; este întoarsă valoarea veche, dacă există, altfel e întors `null`
- `remove(key)` - şterge o asociere cheie-valoare
- `replace(key, val)` - modifică valoarea asociată cu o cheie.

ConcurrentHashMap - utilizare

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<Integer,
String>();

map.put(0, "Zero");

map.put(1, "One");

System.out.println(map.get(1)); // One

String val = map.putIfAbsent(1, "Unu"); // nu se modifică nimic, val va fi "Unu"

val = map.putIfAbsent(2, "Two"); // se adaugă maparea 2-"Two", val va fi null
```

BlockingQueue

- `BlockingQueue` este o interfață ce extinde `Queue`, iar clasele care implementează `BlockingQueue` sunt colecții concurente:
  - `ArrayBlockingQueue` - bazat pe array, se specifică capacitatea maximă la inițializare
  - `LinkedBlockingQueue` - bazat pe LinkedList
  - `PriorityBlockingQueue` - coadă cu priorități, se poate specifica un Comparator la inițializare

Operație	Aruncă excepție	Valoare specială	Se blochează	Dă timeout
Inserare	add(e)	offer(e)	put(e)	offer(e, time, unit)
Ștergere	remove()	poll()	take()	poll(time, unit)
Examinare	element()	peek()	-	-

Collections.synchronized

- dacă dorim să lucrăm cu colecții clasice, precum ArrayList, și să avem operațiile sincronizate, putem să le instanțiem folosind:
  - Collections.synchronizedList (pentru liste): `List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());`
  - Collections.synchronizedMap (pentru dicționare): `Map<Integer, String> syncMap = Collections.synchronizedMap(new HashMap<>());`

Arbori concurenți

- inserția unui copil a unui nod într-un arbore binar (normal, nu de căutare) se face prin verificarea dacă are un nod-copil în stânga:

```
if (node.left == null)
    node.left = child;
else
    node.right = child;
```

Arbori concurenți - probleme de concurență

T0	T1
Verifică dacă există copil în stânga	Verifică dacă există copil în stânga
Condiția este adevărată, trece în prima ramură a if-ului	Condiția este adevărată, trece în prima ramură a if-ului
Inserează nodul în stânga	
	Inserează nodul în stânga → se suprascrie valoarea scrisă de T0

Corect ar fi ca T0 să insereze în stânga și T1 în dreapta

Arbori concurenți - soluție

- dacă am avea un lock pe tot arborele, doar un singur thread poate accesa arborele la un moment dat. Acest lucru nu este eficient, deoarece putem avea două thread-uri care pot insera noduri-copil în două noduri-părinte diferite.
- soluție: lock pe fiecare nod

Resurse

https://crunchify.com/hashmap-vs-concurrenthashmap-vs-synchronizedmap-how-a-hashmap-can-be-synchronized-in-java/

Laboratorul 7
Modelul Replicated Workers



Modelul Replicated Workers

- **Replicated Workers (Thread Pool)** este un pattern folosit în programarea concurentă
- avem două componente principale:
  - **pool** - coadă de task-uri
  - **workers** - thread-uri care execută task-urile

Cum funcționează Replicated Workers?

```
while (true)
    ia un task din pool
    execută task-ul, care poate genera task-uri suplimentare (forked)
    adaugă zero sau mai multe task-uri rezultate anterior în pool
```

Utilitate Replicated Workers

Este util când:

- nu putem să știm dimensiunea problemei în avans
- putem avea task-uri mai mici ca dimensiune, în acest sens este mai eficient să refolosim thread-urile decât să se creeze altele la un pas nou
- putem avea task-uri care nu sunt echilibrate ca workload, în acest fel thread-urile nu vor avea un workload echilibrat, fapt ce duce la întârzieri în cadrul programului

ExecutorService

- este o interfață în Java, prin care putem lucra cu task-uri asincrone, executate concurent în background
- putem să trimite un număr de cereri, însă ar fi ineficient dacă le-am trimite secvențial, așteptând după fiecare să termine
- **soluție**: lucrul asincron - trimiterea unei cereri, fără să așteptăm terminarea ei (să se execute în background), și apoi să trimitem alte cereri concurent
- este util când dorim toate soluțiile problemei (trimitem toate cererile) sau cel puțin o soluție

ExecutorService - metode

```
ExecutorService tpe = Executors.newFixedThreadPool(4); // instanțierea unui ExecutorService cu 4 workers (threads)

tpe.submit(new MyRunnable(...)); // punerea în pool a unui task ce trebuie să fie executat și lansarea acestuia în execuție

tpe.shutdown(); // oprește ExecutorService din a mai primi task-uri noi
```

ForkJoin

- framework-ul Fork / Join este o implementare a modelului Replicated Workers, cu o abordare *divide et impera*:
  - se împart recursiv task-urile în subtask-uri (**fork**) mai mici și independente, încât să poate fi executate asincron
  - se colectează recursiv rezultatele subtask-urilor într-un singur rezultat (**join**)
- **ForkJoinPool** este o implementare de ExecutorService, cu threads de tip **ForkJoinWorkerThread**, unde fiecare thread are propria sa coadă de task-uri
- fiecare thread, când are coada sa goală, poate să preia task-uri din coada altui thread sau din pool-ul global de task-uri

ForkJoinPool - tipuri de task-uri

Avem două tipuri de task-uri folosite la ForkJoinPool:

- `RecursiveAction` - task care nu returnează nimic
- `RecursiveTask<V>` - task care returnează un rezultat (dacă nu este întors nimic, putem folosi `RecursiveTask<Void>`, cu `return null`)

ForkJoinPool - metode

```
ForkJoinPool fjp = new ForkJoinPool(4); // creare de thread pool

fjp.invoke(new MyTask("files")); // adăugare de task ce va fi executat
de workers și se așteaptă rezultatul acestuia

fjp.shutdown(); // închiderea thread pool
```

RecursiveTask - metode

- compute() - calculul unui task (execuția acestuia)
- fork() - crearea unui nou task (dar nu și lansarea acestuia în execuție)
- join() - execuția unui task și așteptarea terminării acestuia

Laboratorul 8
Introducere în MPI



Despre programarea distribuită

- Până acum, ați lucrat cu fire de execuție (threads), care executau funcții și care aveau memorie partajată, având acces la variabile globale.
- În programarea distribuite, lucrăm pe procese de tip nod, care simulează o mașină de calcul. Spre deosebire de thread-uri, procesele au propria lor memorie (astfel ele nu partajează nimic), astfel ele au propriile lor variabile.
- Pentru ca un proces să informeze restul proceselor legate valorile unor variabile, acesta va trebui să trimită un mesaj care conține valorile respective către celelalte procese.

Ce este MPI?

- MPI (Message Passing Interface) reprezintă un framework (o bibliotecă) pentru programarea distribuită (mai concret, pe procese de tip nod)
- MPI este bazat pe modelul transmiterii de mesaje între procese prin intermediul unor canale de comunicație
- În MPI, nu avem memorie partajată, astfel toate variabilele sunt locale fiecărui proces.
- Pentru a transmite date de la un proces la altul, se folosesc funcții precum Send, Recv, Scather, Gather

Compilare și rulare

- Includere bibliotecă: `#include <mpi.h>`
- Compilare: `mpicc send_recv.c -o send_recv`
- Rulare: `mpirun -np 3 send_recv` (rulare cu 3 procese)
- Rulare cu un singur proces: `./send_recv`

Funcții legate de mediul MPI

- `MPI_Init(int *argc, char **argv)` - construiește environment-ul în care procesele se vor rula. De regulă se pasează ca argumente cele din linie din comandă (adică cele din `int main (int argc, char **argv)`)
- `MPI_Finalize()` - se termină procesele.
- `MPI_Comm_size(MPI_Comm comm, int *size)` - se determină numărul de procese (size) asociate cu un comunicator (de regulă `MPI_COMM_WORLD`)
- `MPI_Comm_rank(MPI_Comm comm, int *rank)` - determină rangul / id-ul unui proces

Tipuri de date în MPI

Tipul de date în MPI	Tipul de date în C
MPI_SHORT	short
MPI_INT	int
MPI_LONG	long
MPI_LONG_LONG	long long
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED_LONG	unsigned long

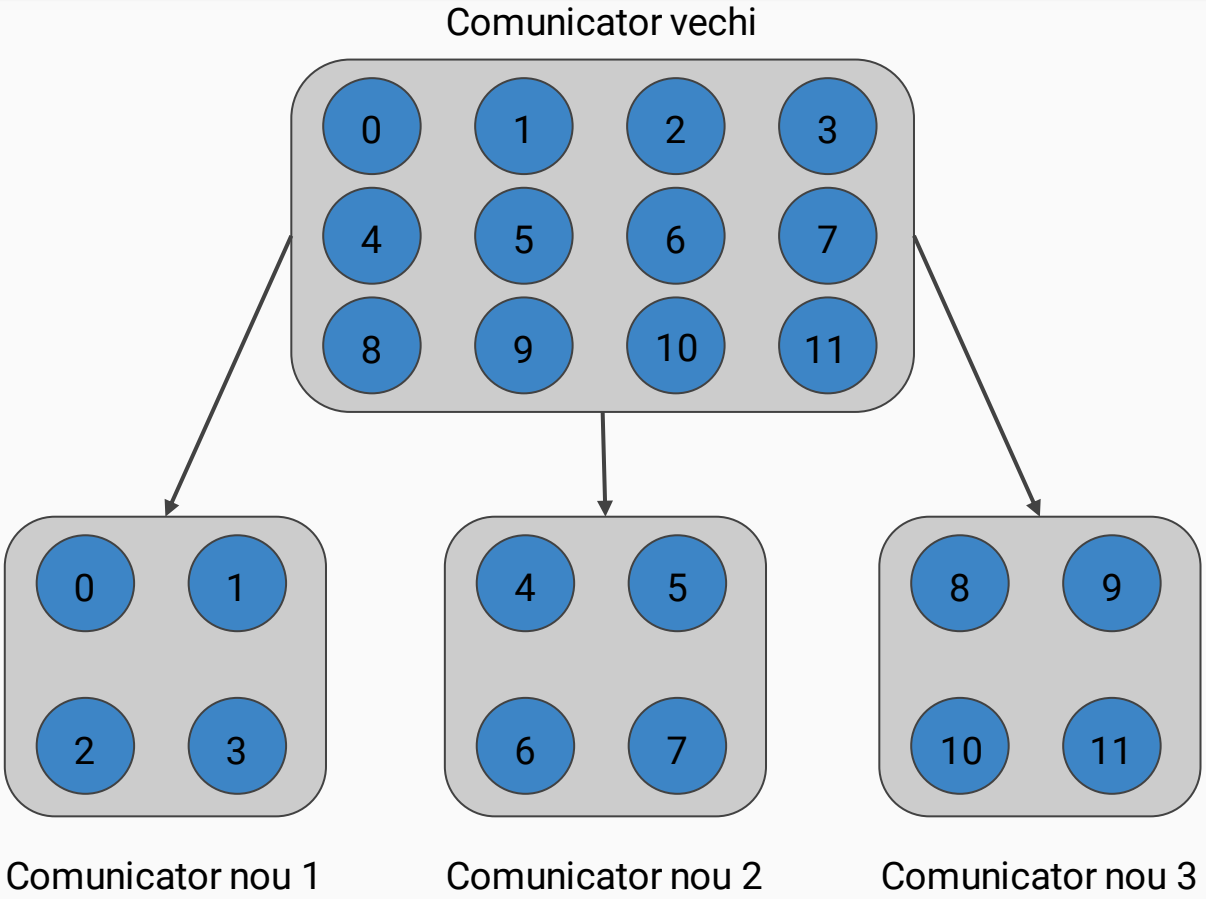
MPI_Comm_split

Putem crea comunicatoare dintr-un alt comunicator folosind funcția MPI_Comm_split, care împarte un comunicator în mai multe comunicatoare mai mici. Semnătura funcției este următoarea:

`int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)`, unde:

- comm - comunicatorul este împărțit
- color - identificator al noului comunicator, din care face parte un proces (de regulă rang_vechi_proces / dimensiune_comunicator_nou)
- key - noul rang al procesului în cadrul noului comunicator (de regulă rang_vechi_proces % dimensiune_comunicator_nou)
- newcomm - noul comunicator format

MPI_Comm_split



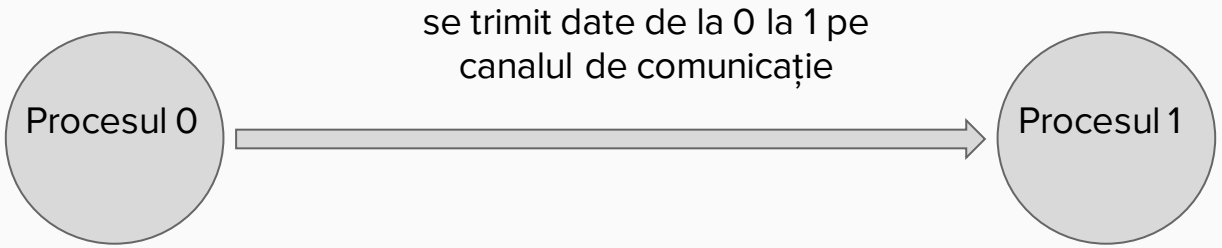
MPI_Send

- `MPI_Send(void* data, int count, MPI_Datatype datatype, int destination, int tag, MPI_Comm communicator)`
- un proces-sursă trimite date unui proces-destinație
- `data` = datele trimise
- `count` = numărul de date trimise
- `datatype` = tipul datelor trimise
- `destination` = id-ul procesului destinație
- `tag` = tag-ul mesajului trimis (adică al datelor trimise)
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

MPI_Recv

- `MPI_Recv(void* data, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm communicator, MPI_Status* status)`
- proces-destinație primește date de la un proces-sursă
- `data` = datele primite
- `count` = numărul de date primite
- `datatype` = tipul datelor primite
- `destination` = id-ul procesului sursă
- `tag` = tag-ul mesajului primite (adică al datelor primite)
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

Exemplificare send - receive (între două procese)



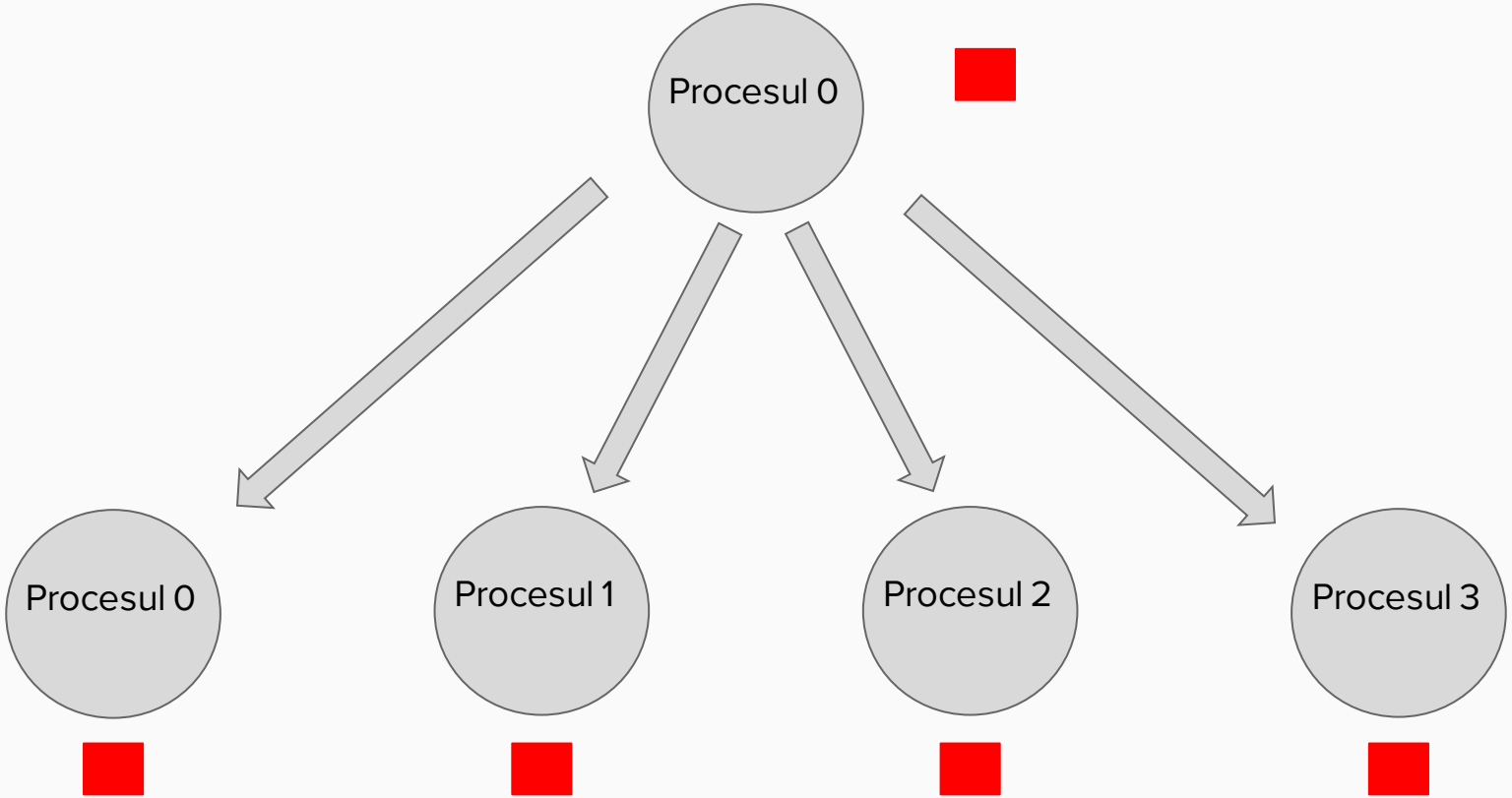
Execută MPI_Send (trimite date la procesul 1)

Execută MPI_Recv (primește date de la procesul 0)

MPI_Bcast

- `MPI_Bcast(void* data, int count, MPI_Datatype datatype, int root, MPI_Comm communicator)` - un proces trimite aceleași date tuturor proceselor din topologie (inclusiv lui însuși).
- `data` = datele trimise
- `count` = numărul de date trimise
- `datatype` = tipul datelor trimise
- `root` = id-ul procesului sursă
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

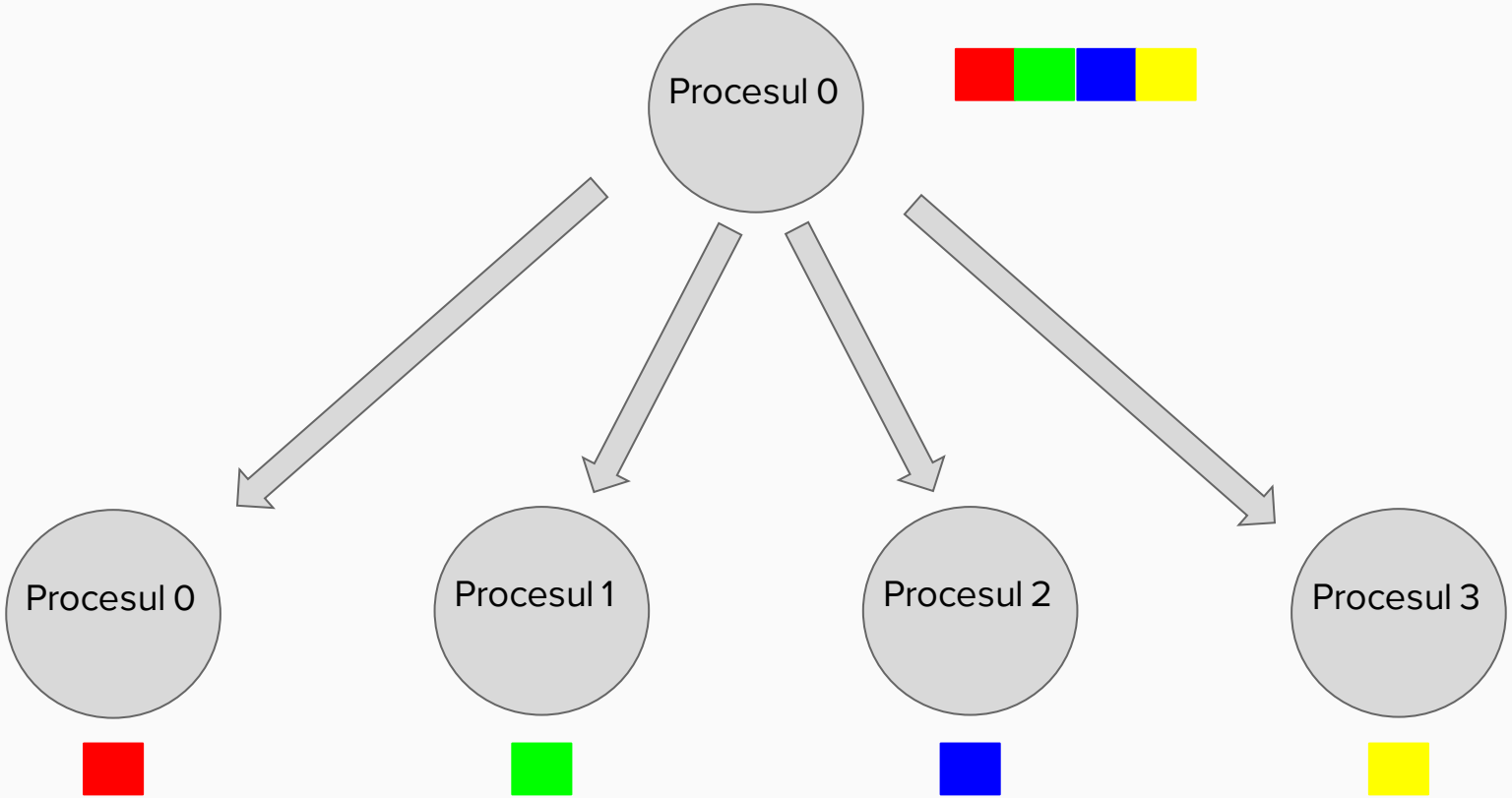
MPI_Bcast



MPI_Scatter

- `MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)` - un proces trimite câte o bucată dintr-un array / colecție tuturor proceselor în parte, inclusiv lui însuși (fiecare proces cu bucata sa din array)
- `send_data` = datele trimise
- `send_datatype` = tipul datelor trimise (de regulă recv_datatype == send_datatype)
- `send_count` = numărul de date trimise fiecărui proces în parte (de regulă recv_count == send_count)
- `recv_data` = datele primite
- `recv_count` = numărul de date primite de fiecare proces în parte (de regulă recv_count == send_count)
- `recv_datatype` = tipul datelor recepționate (de regulă recv_datatype == send_datatype)
- `root` = id-ul procesului sursă
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

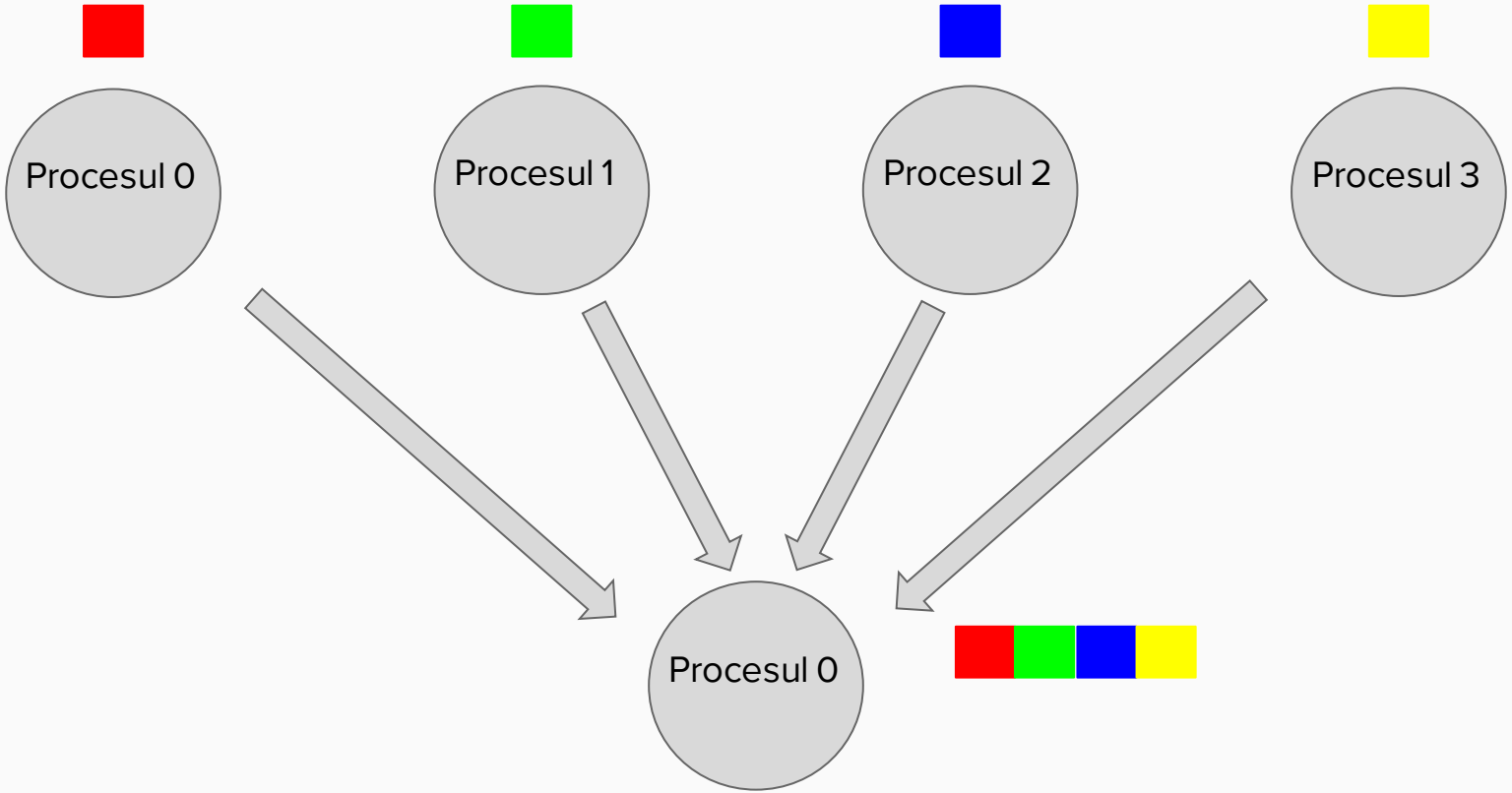
MPI_Scatter



MPI_Gather

- `MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)` - un proces primește câte o bucată dintr-un array / colecție de la toate procesele, inclusiv de la însuși (fiecare proces cu bucata sa din array) - reprezintă inversul lui Scatter.
- `send_data` = datele trimise
- `send_datatype` = tipul datelor trimise (de regulă recv_datatype == send_datatype)
- `send_count` = numărul de date trimise fiecărui proces în parte (de regulă recv_count == send_count)
- `recv_data` = datele primite
- `recv_count` = numărul de date primite de fiecare proces în parte (de regulă recv_count == send_count)
- `recv_datatype` = tipul datelor recepționate (de regulă recv_datatype == send_datatype)
- `root` = id-ul procesului destinație
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

MPI_Gather



Algoritmul inel

- Algoritmul inel este un algoritm undă care funcționează pe rețele inelare de procese (cicluri Hamiltoniene), într-un sens unic.
- Fiecare proces are un vecin dedicat (next).
- Avem un proces inițiator, care inițiază transmiterea mesajului în cadrul rețelei.
- Inițiatorul trimite un mesaj (token) care este pasat de fiecare proces de-a lungul ciclului până când mesajul ajunge înapoi la inițiator, care apoi ia o decizie.

Algoritmul inel - pseudocod

```
chan token[1..n] (tok_type tok)

// initiator

process P[i] {

    tok_type tok;

    send token[next](tok); // trimite token-ul la urmatorul nod

    receive token[i](tok); // primeste token-ul de la nodul precedent

    decizie;

}
```

Algoritmul inel - pseudocod

```
// neinitiator

process P[k, k = 1..n, k != n] {

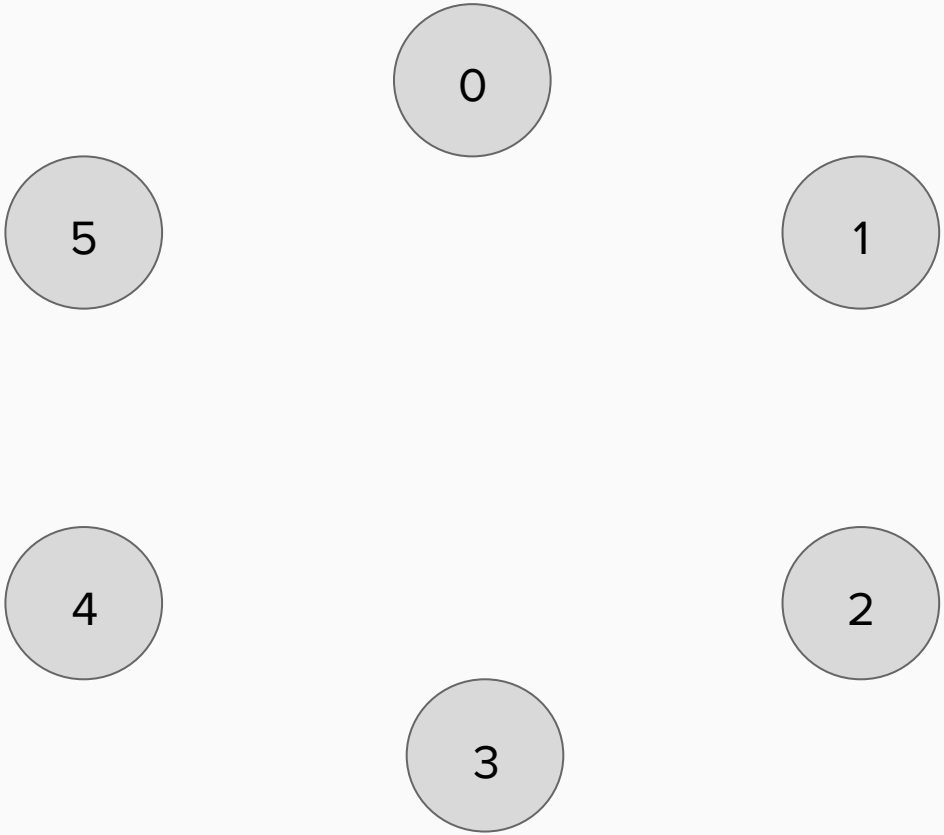
    tok_type tok;

    receive token[k](tok); // primeste token-ul de la nodul precedent

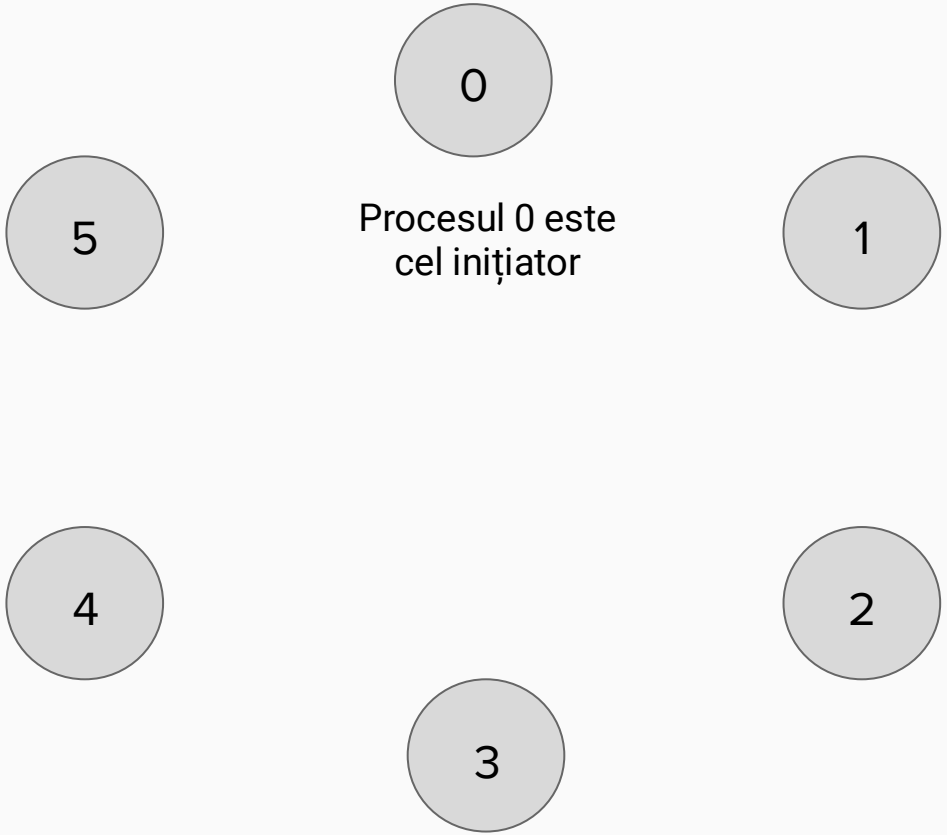
    send token[next](tok); // trimite token-ul la urmatorul nod

}
```

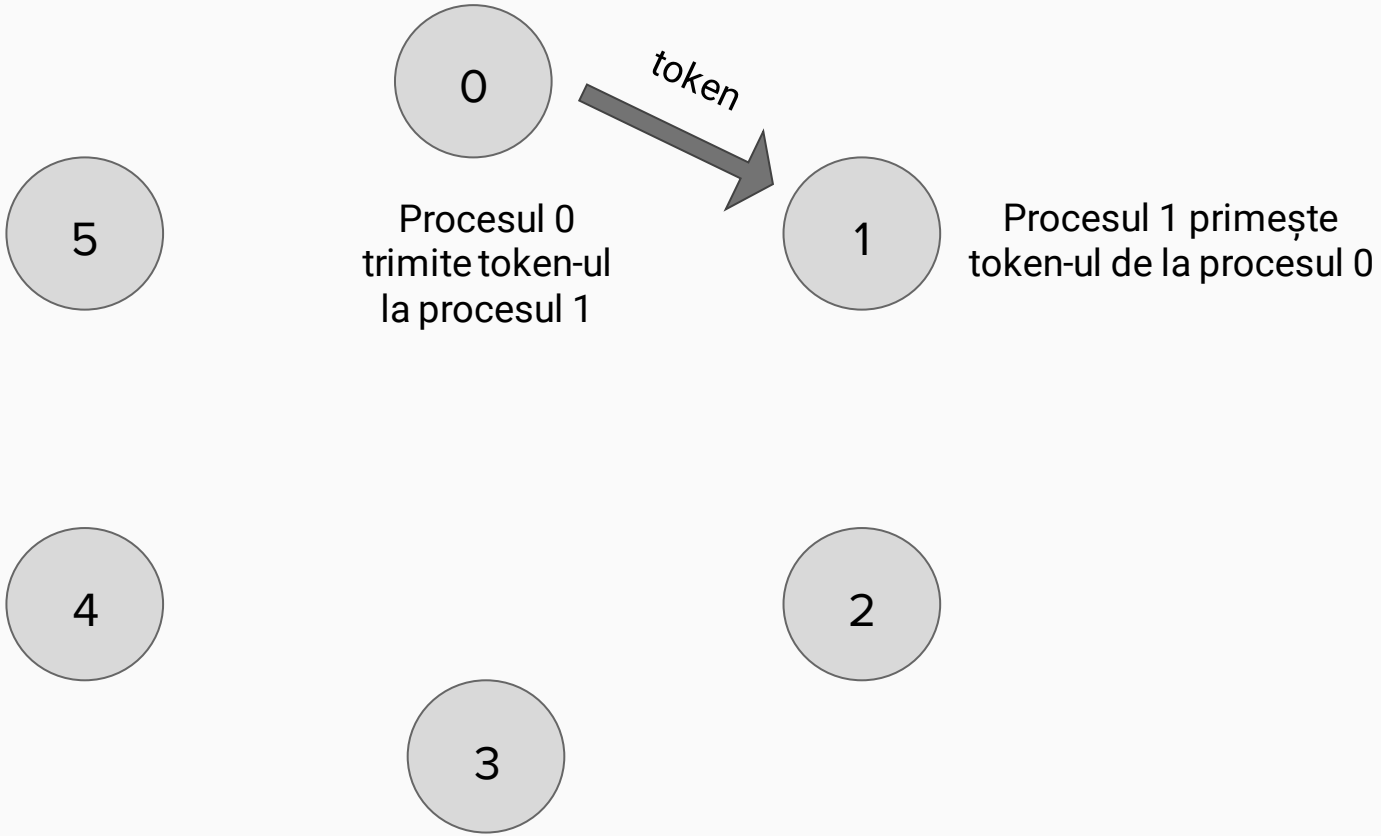
Algoritmul inel



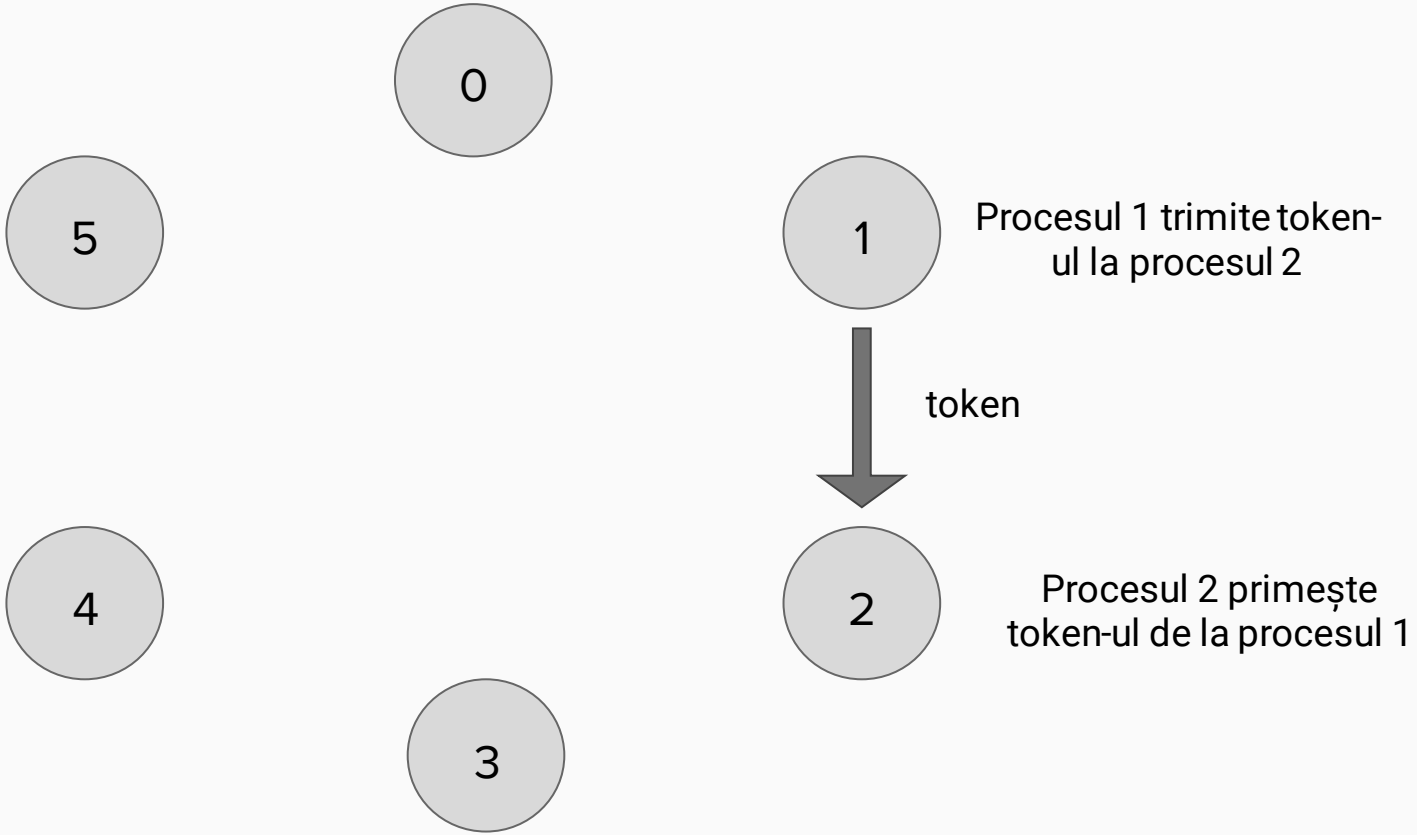
Algoritmul inel



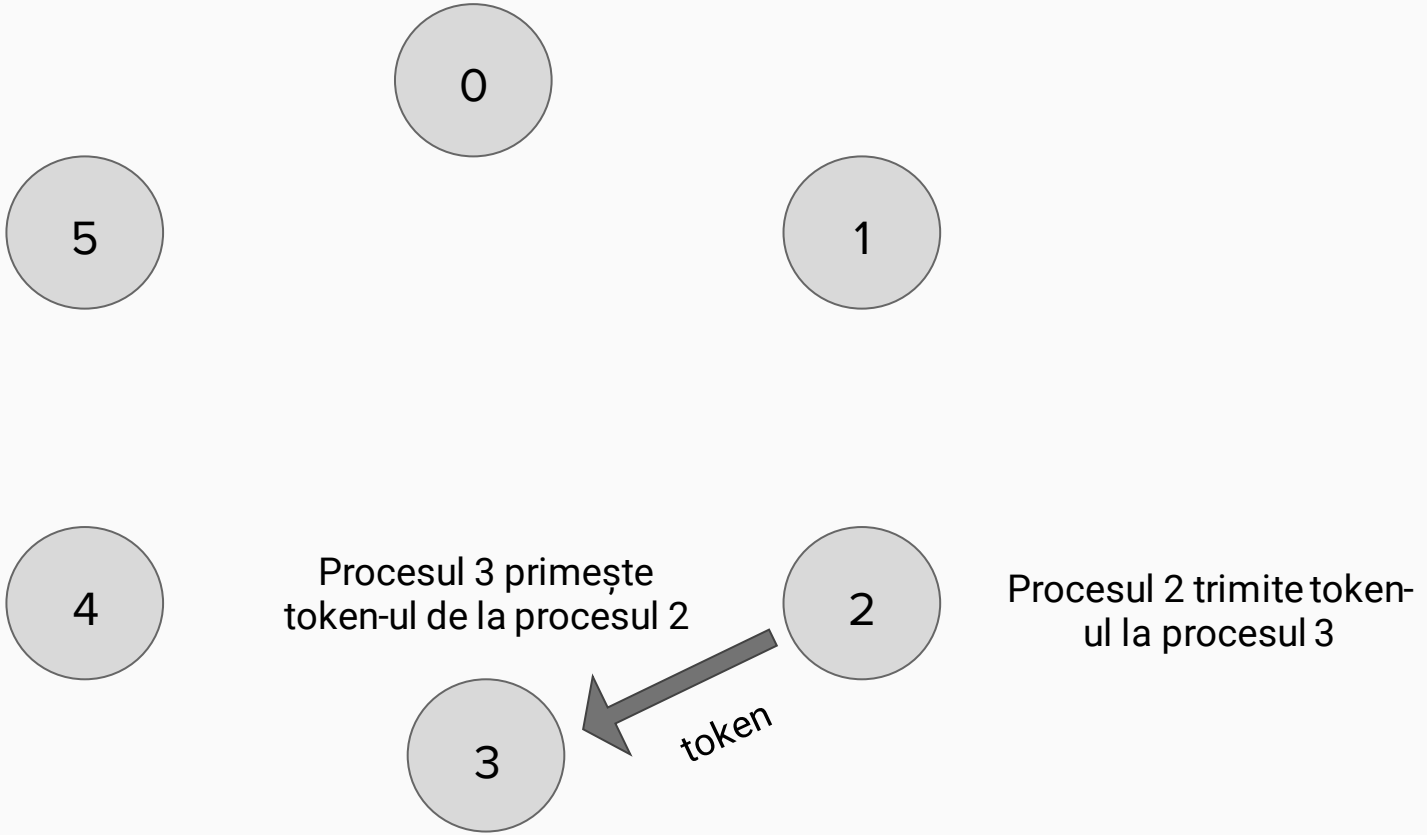
Algoritmul inel



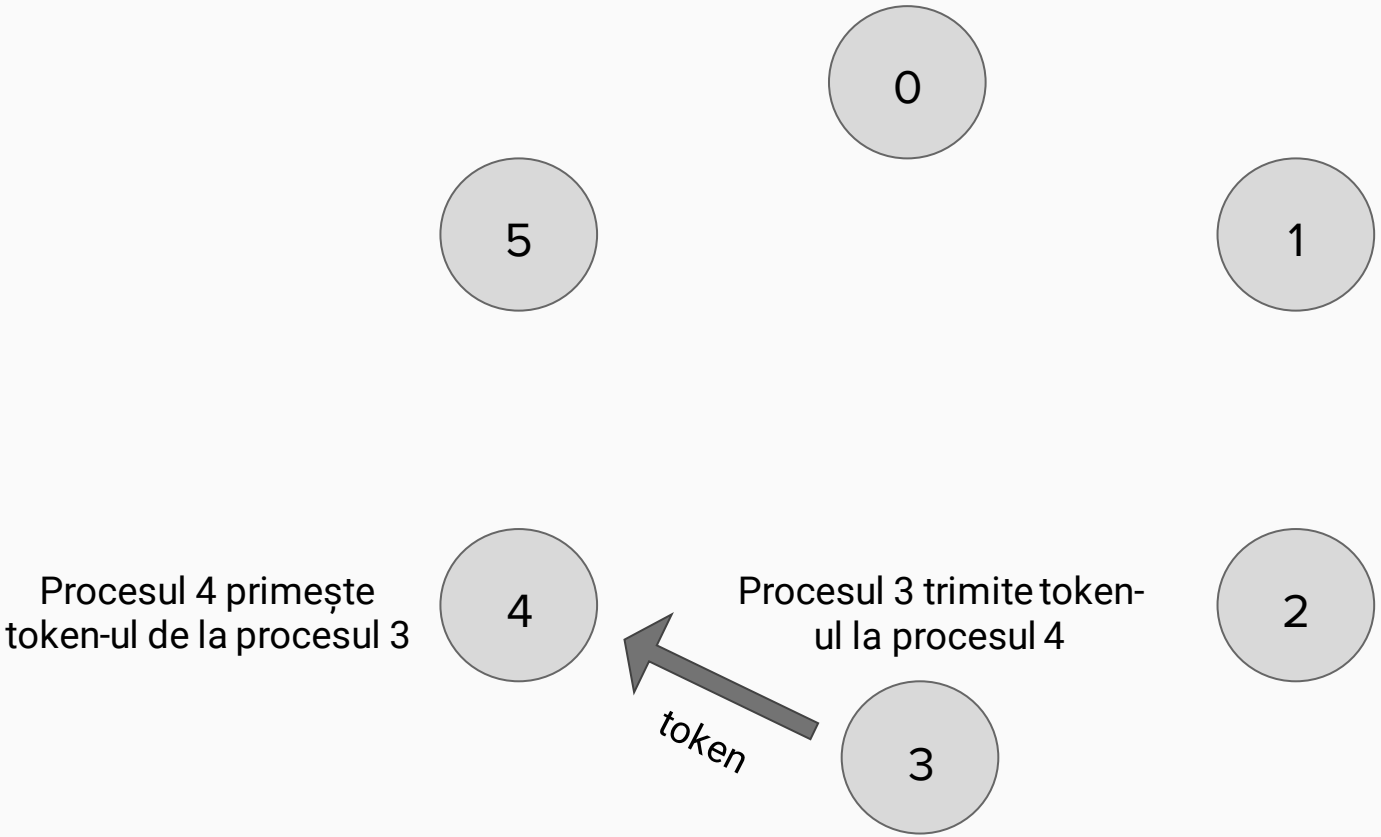
Algoritmul inel



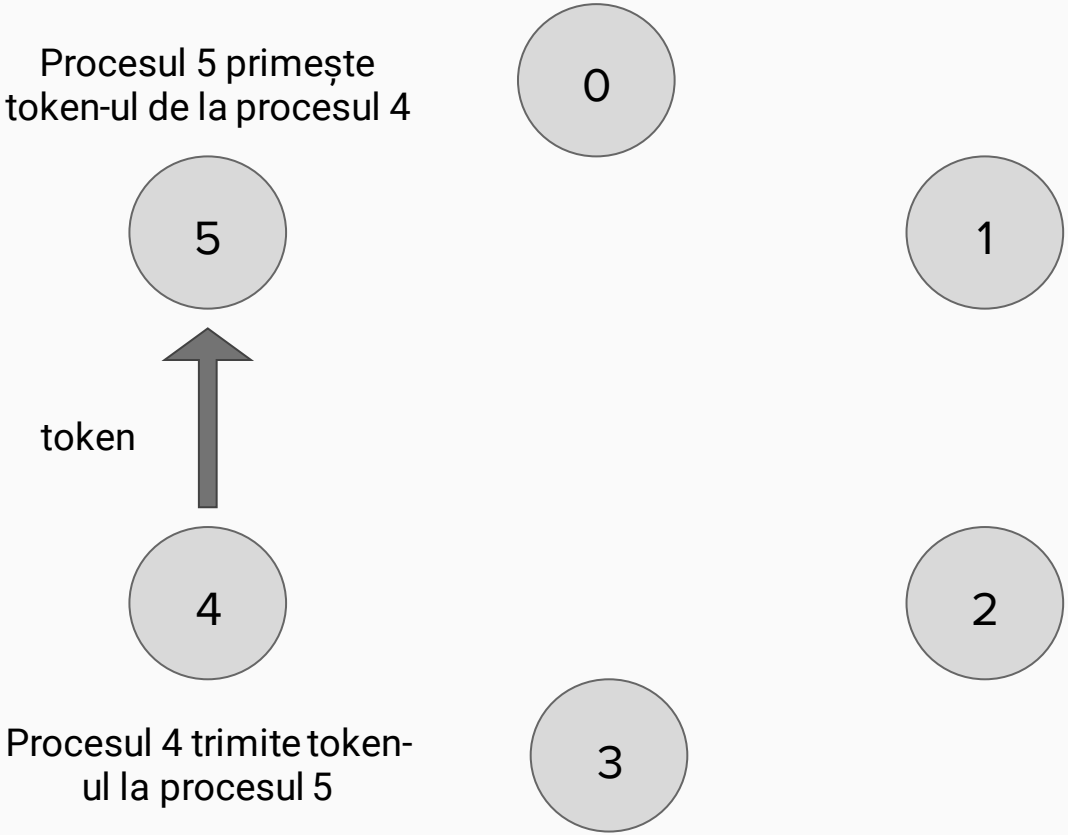
Algoritmul inel



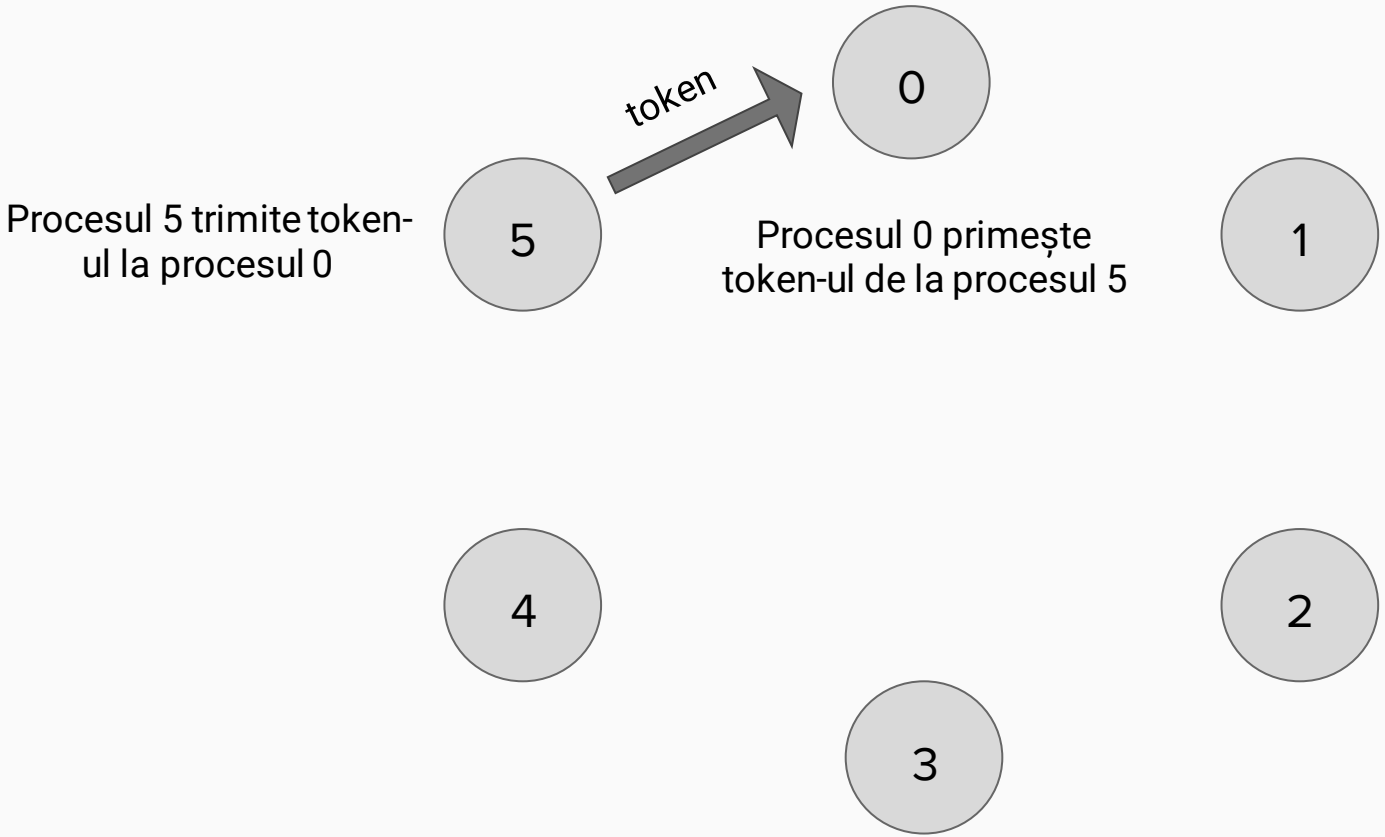
Algoritmul inel



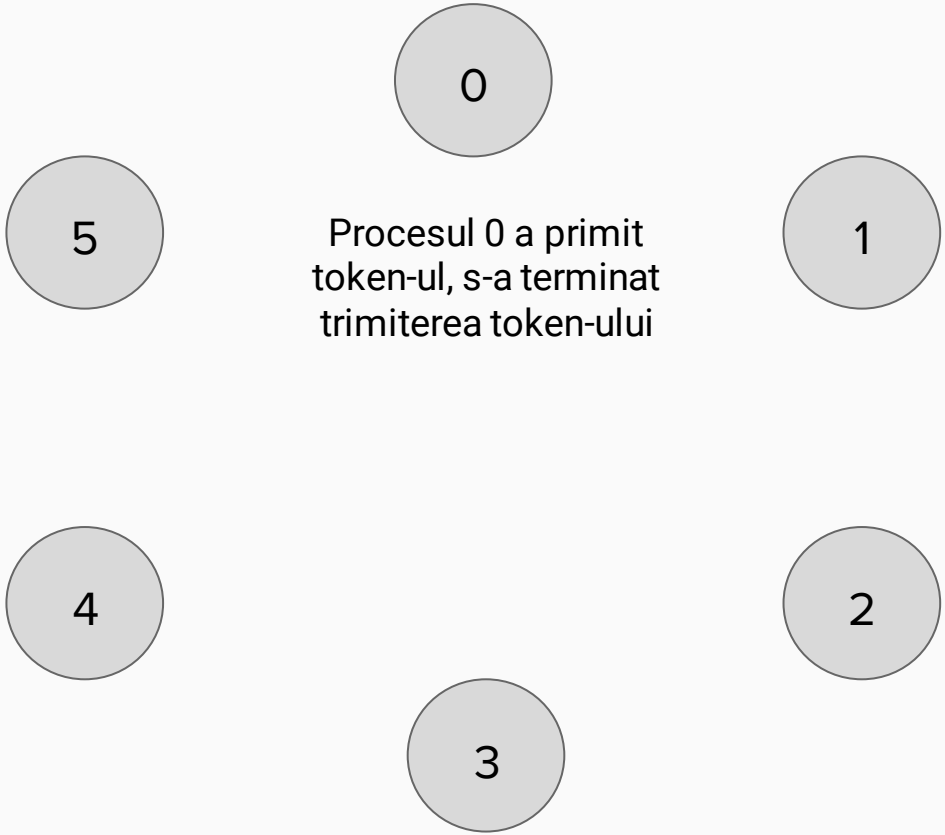
Algoritmul inel



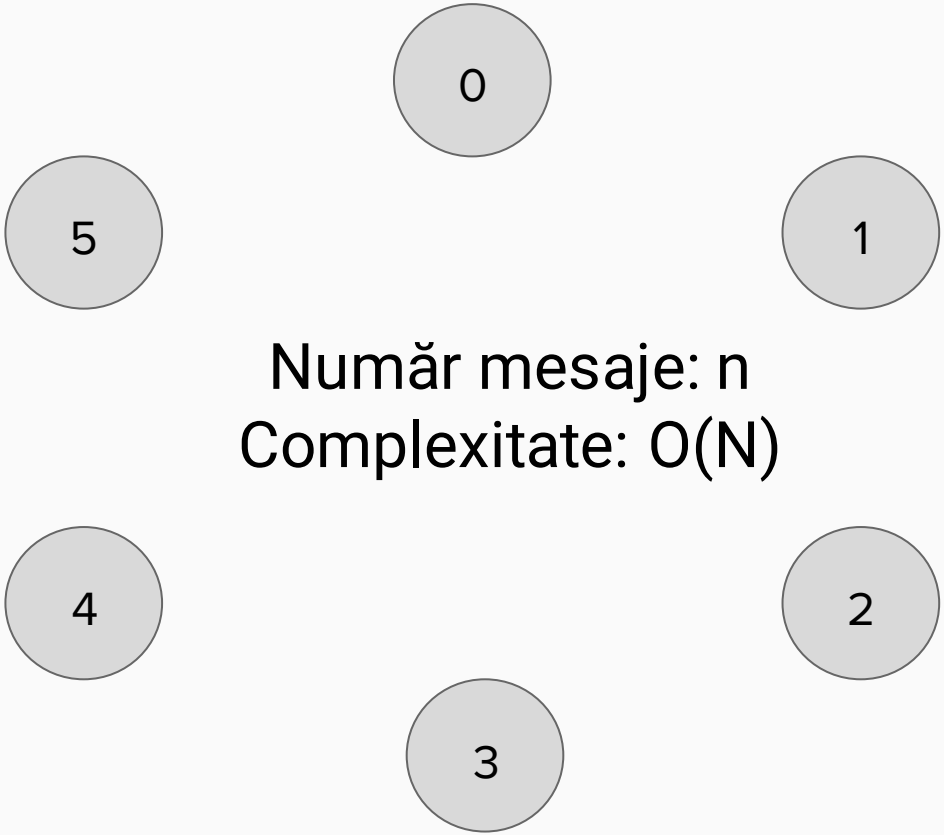
Algoritmul inel



Algoritmul inel



Algoritmul inel



Întrebări

Resurse

- https://mpitutorial.com/tutorials/
- https://computing.llnl.gov/tutorials/mpi/

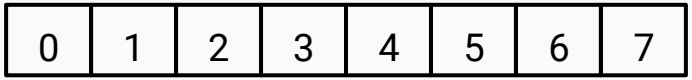
Laboratorul 9
Operații distribuite în MPI



Reduce

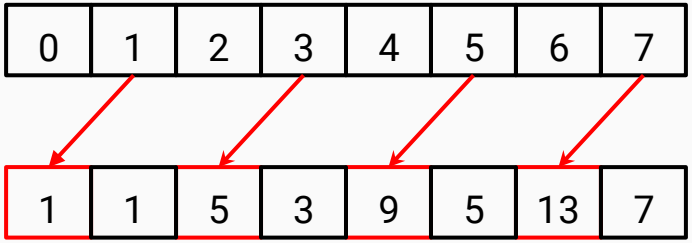
Operația de reduce (cunoscută în limbajele funcționale - Haskell, Racket - drept fold) reprezintă un tip de operație prin care elementele unei colecții sunt restrânse / acumulate într-un singur rezultat, printr-o singură operație aplicată între elementele unei colecții (+, *, min, max etc.)

Reduce



În acest exemplu, fiecare proces are câte un element (8 elemente - 8 procese)

Reduce

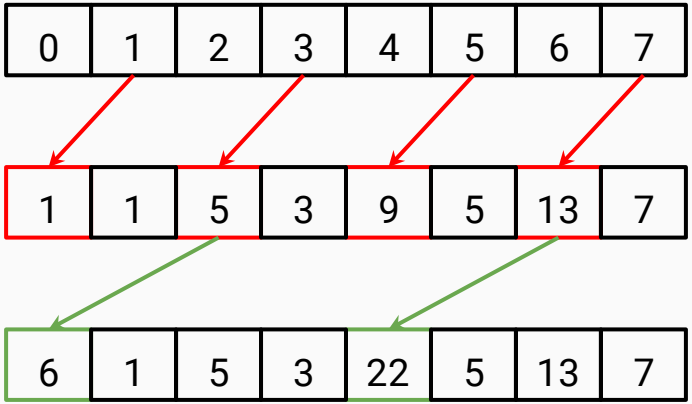


- procesul 1 trimite procesului 0, care adună valoarea primită (1 + 0 = 1)
- procesul 3 trimite procesului 2, care adună valoarea primită (3 + 2 = 5)
- procesul 5 trimite procesului 4, care adună valoarea primită (5 + 4 = 9)
- procesul 7 trimite procesului 6, care adună valoarea primită (7 + 6 = 13)

Deci:

- procesul 0 are valoarea 1
- procesul 2 are valoarea 5
- procesul 4 are valoarea 9
- procesul 6 are valoarea 13

Reduce

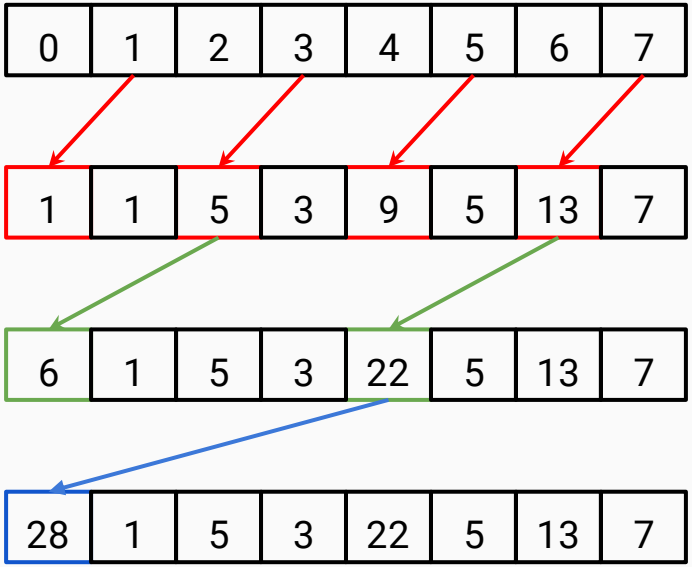


- procesul 2 trimite procesului 0, care adună valoarea primită (5 + 1 = 6)
- procesul 6 trimite procesului 4, care adună valoarea primită (13 + 9 = 22)

Deci:

- procesul 0 are valoarea 6
- procesul 4 are valoarea 22

Reduce



- procesul 4 trimite procesului 0, care adună valoarea primită (22 + 6 = 28)

Deci:

- procesul 0 are valoarea 28

Reduce - pseudocod

```
for (pas = 2; pas <= nr_procese; pas *= 2)
      if (rank % pas == 0 && (rank + (pas / 2)) < nr_procese)
            primește la procesul cu rank-ul [rank + (pas /
2)]
            adună
   else if (rank % (pas / 2) == 0 && (rank - (pas / 2)) >= 0)
      trimite la procesul cu rank-ul [rank - (pas / 2)]
```

Reduce - pseudocod - rulare

pas = 2:

- 1 (1 % (2 / 2) == 0) trimite lui 0 (1 - (2 / 2))
- 0 (0 % 2 == 0) primește de la 1 (0 + (2 / 2))

- 3 (3 % (2 / 2) == 0) trimite lui 2 (3 - (2 / 2))
- 2 (2 % 2 == 0) primește de la 3 (2 + (2 / 2))

- 5 (5 % (2 / 2) == 0) trimite lui 4 (5 - (2 / 2))
- 4 (4 % 2 == 0) primește de la 5 (4 + (2 / 2))

- 7 (7 % (2 / 2) == 0) trimite lui 6 (7 - (2 / 2))
- 6 (6 % 2 == 0) primește de la 7 (6 + (2 / 2))

Reduce - pseudocod - rulare

pas = 4:

- 2 (2 % (4 / 2) == 0) trimite lui 0 (2 - (4 / 2))
- 0 (0 % 4 == 0) primește de la 2 (0 + (4 / 2))

- 6 (6 % (4 / 2) == 0) trimite lui 4 (6 - (4 / 2))
- 4 (0 % 4 == 0) primește de la 6 (4 + (4 / 2))

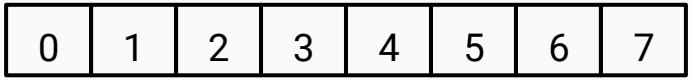
pas = 8:

- 4 (4 % (8 / 2) == 0) trimite lui 0 (4 - (8 / 2))
- 0 (0 % 8 == 0) primește de la 4 (0 + (8 / 2))

Scan

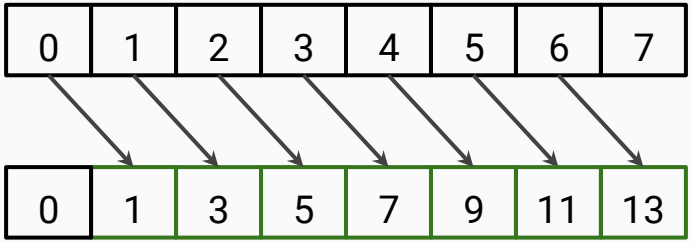
Operația de scan este similară operației de reduce (acumularea elementelor unei colecții într-un singur rezultat). Diferența față de reduce este că ultimul proces din comunicator deține rezultatul final, practic scan reprezintă inversul lui reduce.

Scan



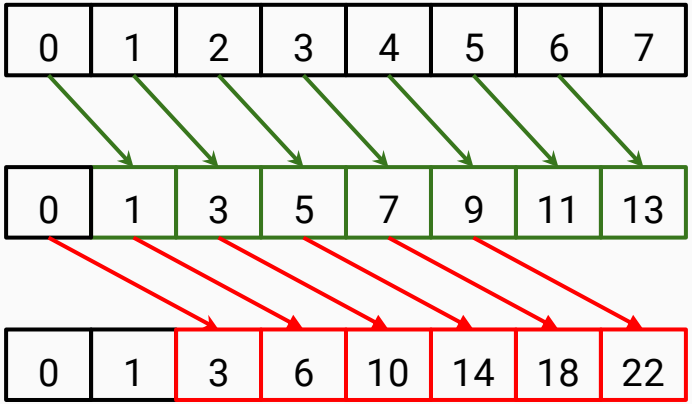
În acest exemplu, fiecare proces are câte un element (8 elemente - 8 procese)

Scan



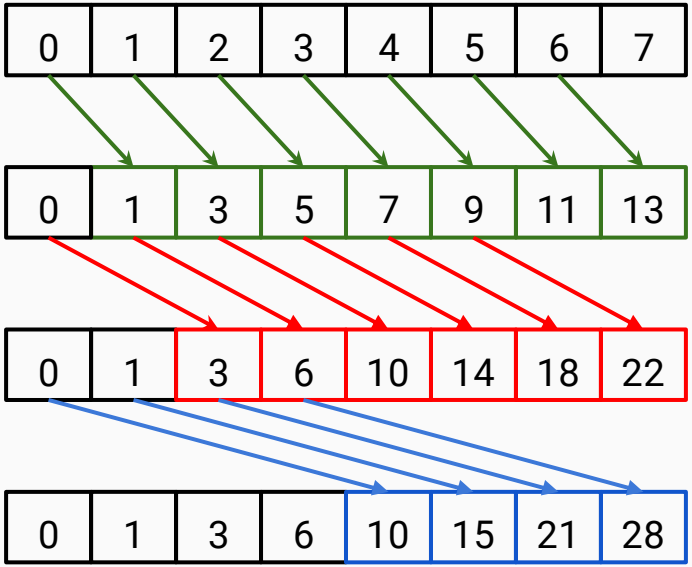
- procesul 0 trimite procesului 1, care adună valoarea primită (0 + 1 = 1)
- procesul 1 trimite procesului 2, care adună valoarea primită (1 + 2 = 3)
- procesul 2 trimite procesului 3, care adună valoarea primită (2 + 3 = 5)
- procesul 3 trimite procesului 3, care adună valoarea primită (3 + 4 = 7)
- procesul 4 trimite procesului 5, care adună valoarea primită (4 + 5 = 9)
- procesul 5 trimite procesului 6, care adună valoarea primită (5 + 6 = 11)
- procesul 6 trimite procesului 7, care adună valoarea primită (6 + 7 = 13)

Scan



- procesul 0 trimite procesului 2, care adună valoarea primită (0 + 3 = 3)
- procesul 1 trimite procesului 3, care adună valoarea primită (1 + 5 = 6)
- procesul 2 trimite procesului 4, care adună valoarea primită (3 + 7 = 10)
- procesul 3 trimite procesului 5, care adună valoarea primită (5 + 9 = 14)
- procesul 4 trimite procesului 6, care adună valoarea primită (7 + 11 = 18)
- procesul 5 trimite procesului 7, care adună valoarea primită (9 + 13 = 22)

Scan



- procesul 0 trimite procesului 4, care adună valoarea primită (0 + 10 = 10)
- procesul 1 trimite procesului 5, care adună valoarea primită (1 + 14 = 15)
- procesul 2 trimite procesului 6, care adună valoarea primită (3 + 18 = 21)
- procesul 3 trimite procesului 7, care adună valoarea primită (6 + 22 = 28)

Scan - pseudocod

```
for (pas = 1; pas < nr_procese; pas *= 2)
       if (rank + pas < nr_procese)
              trimite la procesul cu rank-ul [rank + pas]
    if (rank - pas >= 0)
       primește de la procesul cu rank-ul [rank - pas]
       adună
```

Scan - pseudocod - rulare

pas = 1:

- 0 trimite lui 1 (0 + 1)
- 1 primește de la 0 (1 - 1)

- 1 trimite lui 2 (1 + 1)
- 2 primește de la 1 (2 - 1)

- 2 trimite lui 3 (2 + 1)
- 3 primește de la 2 (3 - 1)

- 3 trimite lui 4 (3 + 1)
- 4 primește de la 3 (4 - 1)

pas = 1:

- 4 trimite lui 5 (4 + 1)
- 5 primește de la 4 (5 - 1)

- 5 trimite lui 6 (5 + 1)
- 6 primește de la 5 (6 - 1)

- 6 trimite lui 7 (6 + 1)
- 7 primește de la 6 (7 - 1)

Scan - pseudocod - rulare

pas = 2:

- 0 trimite lui 2 (0 + 2)
- 2 primește de la 0 (2 - 2)

- 1 trimite lui 3 (1 + 2)
- 3 primește de la 1 (3 - 2)

- 2 trimite lui 4 (2 + 2)
- 4 primește de la 2 (4 - 2)

- 3 trimite lui 5 (3 + 2)
- 5 primește de la 3 (5 - 2)

- 4 trimite lui 6 (4 + 2)
- 6 primește de la 4 (6 - 2)

- 5 trimite lui 7 (5 + 2)
- 7 primește de la 5 (7 - 2)

pas = 4:

- 0 trimite lui 4 (4 + 4)
- 4 primește de la 0 (4 - 4)

- 1 trimite lui 5 (1 + 4)
- 5 primește de la 1 (5 - 1)

- 2 trimite lui 6 (2 + 4)
- 6 primește de la 2 (6 - 4)

- 3 trimite lui 7 (3 + 4)
- 7 primește de la 3 (7 - 4)

Broadcast

Operația de broadcast este o operație prin care un proces trimite o valoare tuturor proceselor din cadrul comunicatorului. Această operație este reprezentată, în MPI, de MPI_Bcast (remember laboratorul 8).

Broadcast

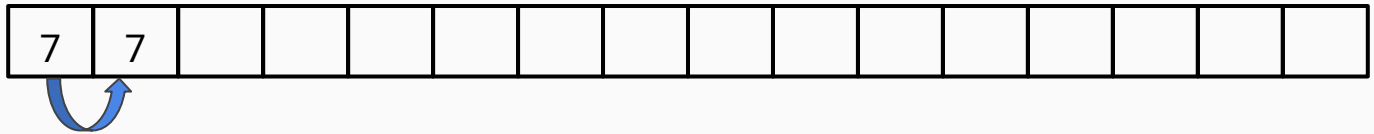


În acest exemplu, rulăm cu 16 procese (fiecare proces, mai puțin procesul 0, nu are nicio valoare)

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank < pas and rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank +
pas]
    else if (rank >= pas and rank < pas * 2)
        primește de la procesul cu rank-ul [rank - pas]
```

Broadcast

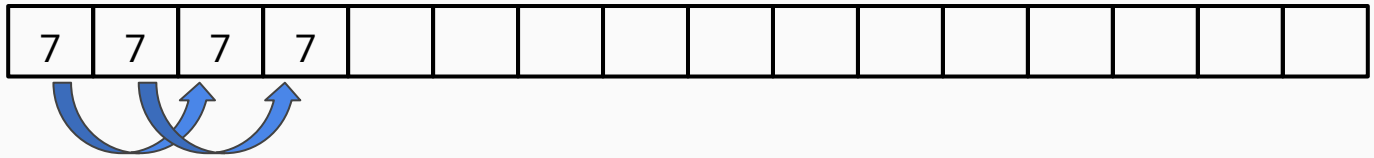
pas = 1



- procesul 0 (0 < 1, 0 + 1 < 16) trimite procesului 1 (0 + 1, 1 >= 1, 1 < 2 * 1)

Broadcast

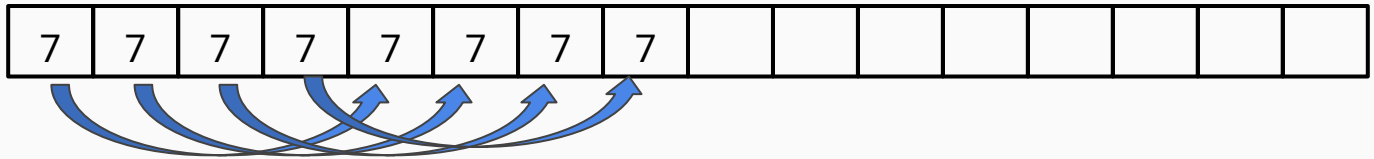
pas = 2



- procesul 0 (0 < 2, 0 + 2 < 16) trimite procesului 2 (0 + 2, 2 >= 2, 2 < 2 * 2)
- procesul 1 (1 < 2, 1 + 2 < 16) trimite procesului 3 (1 + 2, 3 >= 2, 3 < 2 * 2)

Broadcast

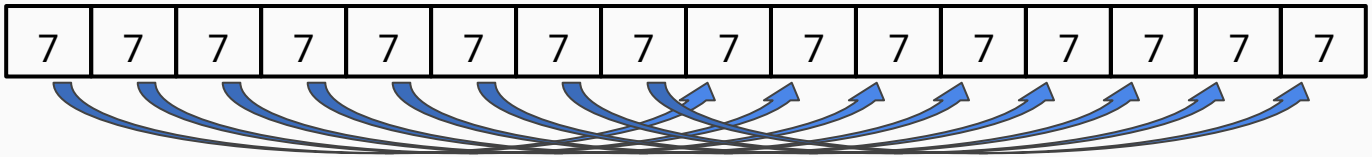
pas = 4



- procesul 0 (0 < 4, 0 + 4 < 16) trimite procesului 4 (0 + 4, 4 >= 4, 4 < 2 * 4)
- procesul 1 (1 < 4, 1 + 4 < 16) trimite procesului 5 (1 + 4, 5 >= 4, 5 < 2 * 4)
- procesul 2 (2 < 4, 2 + 4 < 16) trimite procesului 6 (2 + 4, 6 >= 4, 6 < 2 * 4)
- procesul 3 (3 < 4, 3 + 4 < 16) trimite procesului 7 (3 + 4, 7 >= 4, 7 < 2 * 4)

Broadcast

pas = 8



- procesul 0 (0 < 8, 0 + 8 < 16) trimite procesului 8 (0 + 8, 8 >= 8, 8 < 2 * 8)
- procesul 1 (1 < 8, 1 + 8 < 16) trimite procesului 9 (1 + 8, 9 >= 8, 9 < 2 * 8)
- procesul 2 (2 < 8, 2 + 8 < 16) trimite procesului 10 (2 + 8, 10 >= 8, 10 < 2 * 8)
- procesul 3 (3 < 8, 3 + 8 < 16) trimite procesului 11 (3 + 8, 11 >= 8, 11 < 2 * 8)
- procesul 4 (4 < 8, 4 + 8 < 16) trimite procesului 12 (4 + 8, 12 >= 8, 12 < 2 * 8)
- procesul 5 (5 < 8, 5 + 8 < 16) trimite procesului 13 (5 + 8, 13 >= 8, 13 < 2 * 8)
- procesul 6 (6 < 8, 6 + 8 < 16) trimite procesului 14 (6 + 8, 14 >= 8, 14 < 2 * 8)
- procesul 7 (7 < 8, 7 + 8 < 16) trimite procesului 15 (7 + 8, 15 >= 8, 15 < 2 * 8)

Pipeline

- În domeniul de computer science, un pipeline reprezintă un concept folosit în proiectarea procesoarelor și al echipamentelor hardware prin care se crește numărul de instrucțiuni ce sunt executate într-o unitate de timp.
- Acest concept poate fi extins în MPI, unde un proces produce rezultate intermerdiare, care sunt transmise către procesul următor, care procesează și prelucrează rezultatele primite, pe care le trimite mai departe, către procesul următor, totul repetându-se până la ultimul proces.

Pipeline



În acest exemplu, avem 5 pași, deci 5 procese, unde fiecare prelucrează date și le trimite mai departe, către următorul proces

Pipeline



Procesul 0 creează rezultate, pe care le trimite procesului 1

Pipeline



Procesul 1 primește date de la procesul 0, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 2

Pipeline



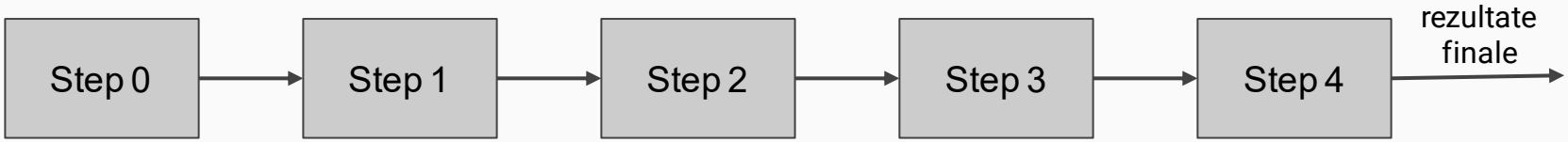
Procesul 2 primește date de la procesul 1, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 3

Pipeline



Procesul 3 primește date de la procesul 2, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 4

Pipeline



Procesul 4 primește date de la procesul 3, le procesează, aplică operații asupra acestora și se obțin rezultatele finale din cadrul procesului de pipeline

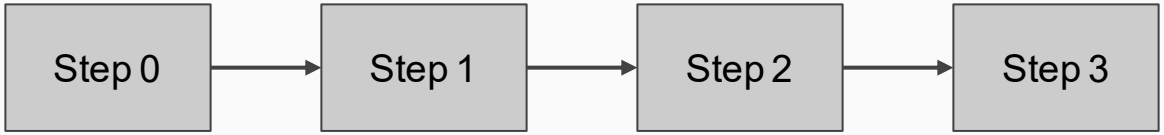
Calculul unui polinom cu pipeline

- Un polinom, în matematică, reprezintă o ecuație de tipul: f(x) = x^2+ 2 * x + 1, unde x reprezintă variabila din ecuație.
- În MPI, putem calcula un polinom folosind tehnica pipeline în felul următor: numărul de coeficienți va fi egal cu numărul de procese (care este egal cu numărul de pași din procesul de pipeline). În cazul exemplului de mai sus, o să avem 3 procese, fiecare proces corespunzând cu câte un coeficient din ecuație

Calcularea unui polinom

- Să luăm un exemplu de polinom: f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1
- Vrem să calculăm f(3) folosind polinom cu pipeline.
- În acest caz, avem un polinom cu 4 coeficienți (de grad 3), astfel vom lucra cu 4 procese (pipeline cu 4 pași de execuție)

Calcularea unui polinom



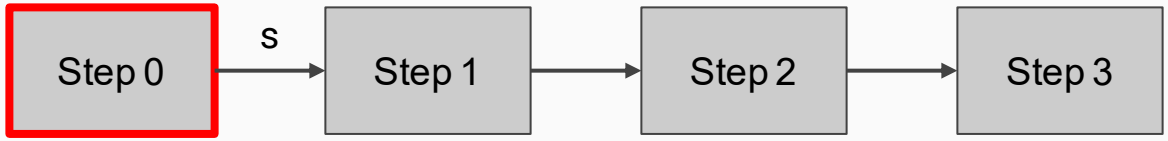
f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1

Calculăm f(3)

s = 0 (s va reprezenta în final rezultatul pipeline-ului, adică f(3)

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



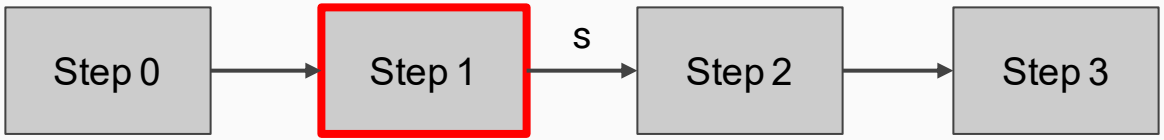
Procesul 0 se ocupă de x^0

```
s += 1 * (3 ^ 0) => s = 1
```

Procesul 0 trimite s către procesul 1

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



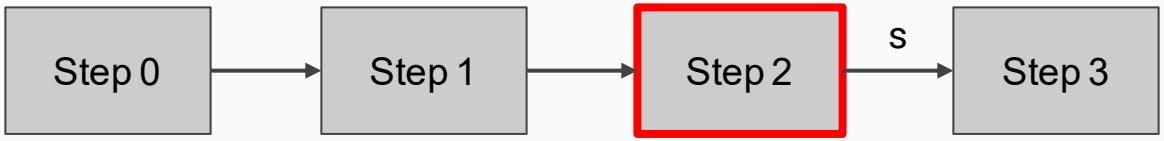
Procesul 1 primește rezultate
de la procesul 0
Procesul 1 se ocupă de x^1

```
s += 2 * (3 ^ 1) => s = 7
```

Procesul 1 trimite s către
procesul 2

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



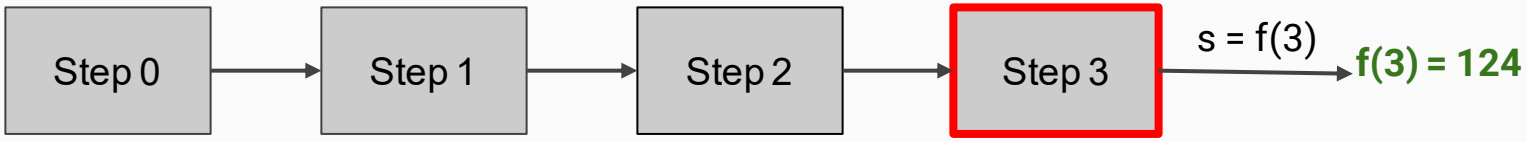
Procesul 2 primește rezultate
de la procesul 1
Procesul 2 se ocupă de x^2

```
s += 4 * (3 ^ 2) => s =
43
```

Procesul 2 trimite s către
procesul 3

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



Procesul 3 primește rezultate de la procesul 2
Procesul 3 se ocupă de x^3

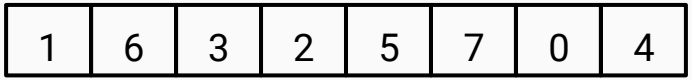
```
s += 3 * (3 ^ 3) => s = 124
```

Procesul 3 este ultimul și întoarce rezultatul final

Sortarea folosind pipeline

- Dându-se un vector cu n elemente în ordine aleatoare, dorim să-l sortăm prin metoda pipeline, folosind MPI. Pentru aceasta, vom folosi o rețea cu n + 1 noduri (1 master și n workeri).
- Punctul final este momentul în care fiecare proces worker deține stocat unul din numerele din vectorul inițial, iar rank[i]→saved_value < rank[i + 1]→saved_value pentru orice i din [1, n).
- Fiecare worker este inițializat la început cu o valoare (-1 sau depinde de restricții), va primi (n - rank) numere de la rank-ul anterior și va trimite mai departe rank-ului următor (n - rank - 1) numere.
- Atunci când primește un număr, worker-ul îl va compara cu valoarea stocată:
  - dacă recv_value < actual_value, atunci salvăm noua valoare și o trimitem pe cea veche procesului următor
  - dacă recv_value >= actual_value, atunci o trimitem direct procesului următor

Sortarea cu pipeline



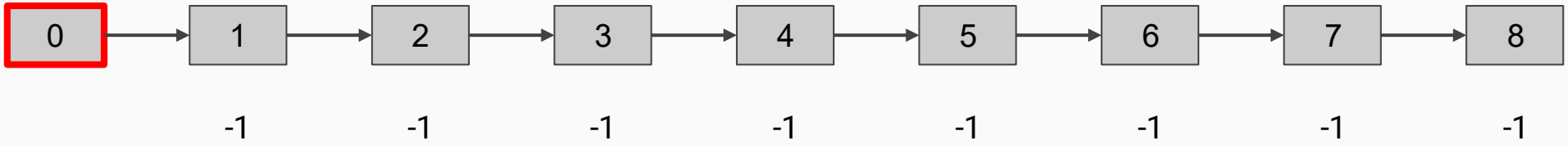
Avem un array nesortat de 8 elemente, așadar vom avea 9 procese:

- un proces master, care începe procesul de pipeline
- 8 procese worker, care se ocupă efectiv de sortarea array-ului

Sortarea cu pipeline



Sortarea cu pipeline



Procesul 0 (master):

- asignează o valoare inițială fiecărui proces worker (să spunem -1) - trimite acea valoare proceselor worker
- trimite întreg array-ul către procesul 1 (dim_array - rank, în acest caz dim_array - 0, rank = 0)

Sortarea cu pipeline



actual_value = -1
primește de la procesul 0: [1, 6, 3, 2, 5, 7, 0, 4]

```
if (actual_value == -1):
        actual_value = recv_value
else if (actual_value <= recv_value)
        trimite recv_value către următorul proces
else
        trimite actual_value către următorul proces
        actual_value = recv_value
```

Sortarea cu pipeline



actual_value = -1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 1
actual_value == -1 => actual_value = recv_value = 1



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 6
actual_value <= recv_value => trimite recv_value (6) către procesul 2



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 3
actual_value <= recv_value => trimite recv_value (3) către procesul 2



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 2
actual_value <= recv_value => trimite recv_value (2) către procesul 2



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 2



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 2



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 0
actual_value > recv_value => trimite actual_value (1) către procesul 2, actual_value = recv_value = 0



actual_value = 0

Sortarea cu pipeline



actual_value = 0
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 2



actual_value = 0

Stop: procesul 1 va avea valoarea 0, se trece la procesul 2, iar procesul 1 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 1: [6, 3, 2, 5, 7, 1, 4]

Sortarea cu pipeline



actual_value = -1
[6, 3, 2, 5, 7, 1, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[6, 3, 2, 5, 7, 1, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (6) către procesul 3, actual_value = recv_value = 3



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 2
actual_value > recv_value => trimite actual_value (3) către procesul 3, actual_value = recv_value = 2



actual_value = 2

Sortarea cu pipeline



actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 3



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 3



actual_value = 3

Sortarea cu pipeline



actual_value = 2
[6, 3, 2, 5, 7, 1, 4]

recv_value = 1
actual_value > recv_value => trimite actual_value (2) către procesul 3, actual_value = recv_value = 1



actual_value = 1

Sortarea cu pipeline



actual_value = 1
[6, 3, 2, 5, 7, 1, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 3

actual_value = 1

Stop: procesul 2 va avea valoarea 1, se trece la procesul 3, iar procesul 2 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 2: [6, 3, 5, 7, 2, 4]

Sortarea cu pipeline



actual_value = -1
[6, 3, 5, 7, 2, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[6, 3, 5, 7, 2, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (6) către procesul 4, actual_value = recv_value = 3



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 4



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 4



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 2
actual_value > recv_value => trimite actual_value (3) către procesul 4, actual_value = recv_value = 2



actual_value = 2

Sortarea cu pipeline



actual_value = 2
[6, 3, 5, 7, 2, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 4



actual_value = 2

Stop: procesul 3 va avea valoarea 2, se trece la procesul 4, iar procesul 3 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 3: [6, 5, 7, 3, 4]

Sortarea cu pipeline



actual_value = -1
[6, 5, 7, 3, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[6, 5, 7, 3, 4]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 5, actual_value = recv_value = 5



actual_value = 5

Sortarea cu pipeline



actual_value = 5
[6, 5, 7, 3, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 5



actual_value = 5

Sortarea cu pipeline



actual_value = 5
[6, 5, 7, 3, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (5) către procesul 5, actual_value = recv_value = 3



actual_value = 3

Sortarea cu pipeline



actual_value = 3
[6, 5, 7, 3, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 5

actual_value = 3

Stop: procesul 4 va avea valoarea 3, se trece la procesul 5, iar procesul 4 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 4: [6, 7, 5, 4]

Sortarea cu pipeline



actual_value = -1
[6, 7, 5, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[6, 7, 5, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[6, 7, 5, 4]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 6, actual_value = recv_value = 5



actual_value = 5

Sortarea cu pipeline



actual_value = 5
[6, 7, 5, 4]

recv_value = 4
actual_value > recv_value => trimite actual_value (5) către procesul 6, actual_value = recv_value = 4



actual_value = 4

Stop: procesul 5 va avea valoarea 4, se trece la procesul 6, iar procesul 5 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 5: [7, 6, 5]

Sortarea cu pipeline



actual_value = -1
[7, 6, 5]

recv_value = 7
actual_value == -1 => actual_value = recv_value = 7



actual_value = 7

Sortarea cu pipeline



actual_value = 7
[7, 6, 5]

recv_value = 6
actual_value > recv_value => trimite actual_value (7) către procesul 7, actual_value = recv_value = 6



actual_value = 6

Sortarea cu pipeline



actual_value = 6
[7, 6, 5]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 7, actual_value = recv_value = 5



actual_value = 5

Stop: procesul 6 va avea valoarea 5, se trece la procesul 7, iar procesul 6 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 6: [7, 6]

Sortarea cu pipeline



actual_value = -1
[7, 6]

recv_value = 7
actual_value == -1 => actual_value = recv_value = 7



actual_value = 7

Sortarea cu pipeline



actual_value = 6
[7, 6]

recv_value = 6
actual_value > recv_value => trimite actual_value (7) către procesul 8, actual_value = recv_value = 6



actual_value = 6

Stop: procesul 7 va avea valoarea 6, se trece la procesul 8, iar procesul 7 trimite valoarea procesului master

Sortarea cu pipeline



actual_value = -1
primește de la procesul 7: [7]

Sortarea cu pipeline



actual_value = -1
[7]

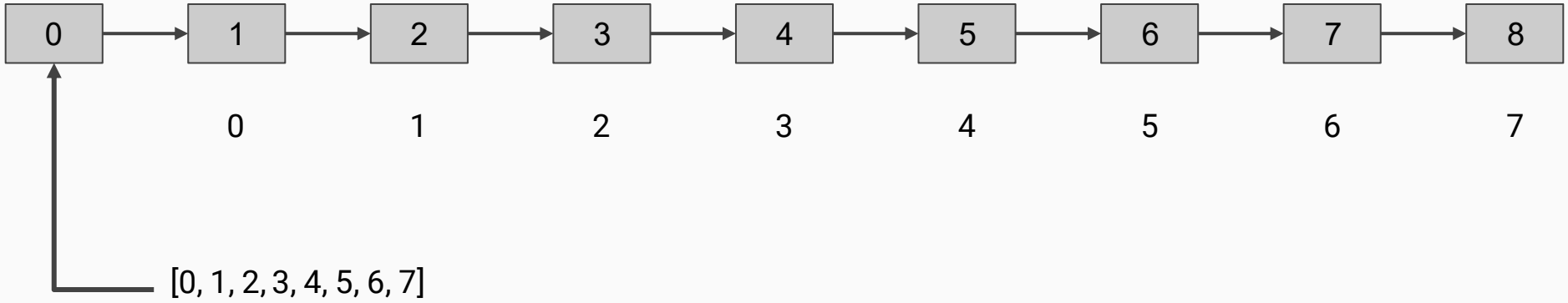
recv_value = 7
actual_value == -1 => actual_value = recv_value = 7

actual_value = 7

Stop: procesul 8 va avea valoarea 7, fiind ultimul proces pipeline-ul se oprește, iar procesul 7 trimite valoarea procesului master

Procesul master are acum toate valorile primite de la procesele worker, în ordine sortată

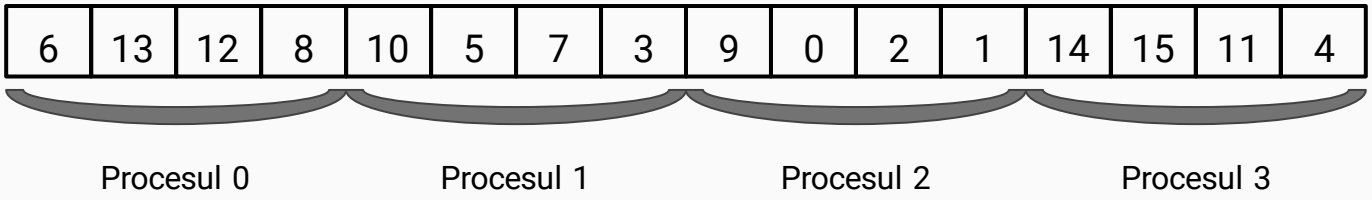
Sortarea cu pipeline



Rank Sort

- Rank Sort este un algoritm de sortare a unui vector folosindu-se de rangul fiecărui element din vector. Rangul unui element presupune câte numere sunt mai mici decât el în vector.
- Pași:
  - Se calculează rangul fiecărui element: pentru fiecare element din vector v[i], unde 0 <= i < N (N - dimensiunea vectorului), se numără câte elemente din vector v[j], j != i sunt mai mici: v[j] < v[i], 0 <= i, j < N, i != j.
  - Elementul de pe poziția “i” va fi pe poziția “rang” în noul vector sortat: q[rank] = v[i].
- Algoritmul “Parallel rank sort” are ca scop paralelizarea pasului 1. Rangul fiecărui element poate fi căutat în paralel, adică fiecare proces sau fir de execuție va realiza căutarea rangurilor pentru elementele din intervalul său: [start, end)

Rank Sort



- În acest exemplu, avem 4 procese.
- Trebuie să vedem, pentru fiecare element din array, câte elemente sunt mai mici decat elementul respectiv.
- Fiecare proces va număra câte elemente sunt mai mici decât un anumit element în bucata sa din array (în acest caz de dimensiune 4 = dim_array / nr_procese).

Rank Sort

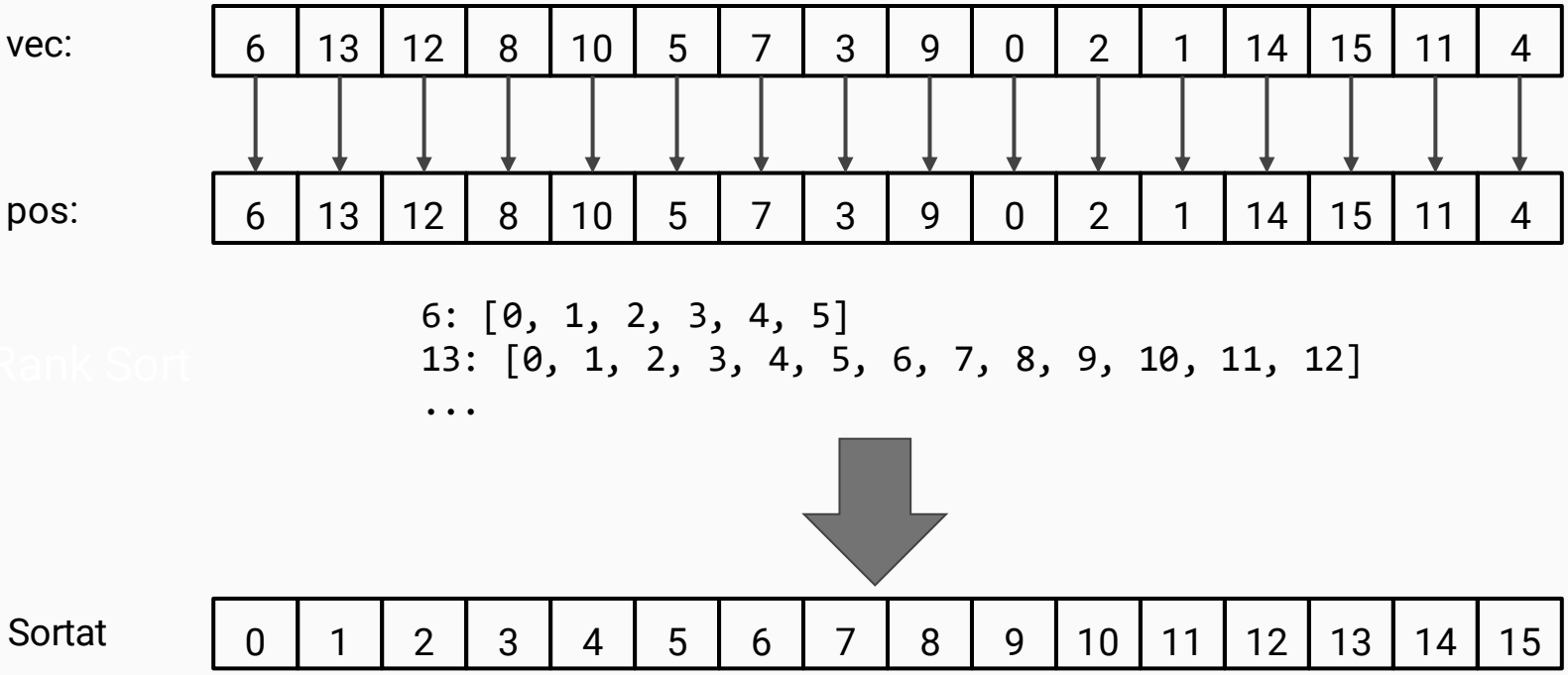
6	13	12	8	10	5	7	3	9	0	2	1	14	15	11	4

- Fiecare proces va avea întreg array-ul și va face numărarea elementelor mai mici în bucata sa de dimensiune dim_array / nr_procese, după cum este în pseudocodul de mai jos:

```
dim_proc = dim_array / nr_procese
for i = 0 to dim_proc
     for j = 0 to dim_array
               if v[rank * dim_proc + i] > v[j]
                         pos[rank * dim_proc + i]++
```

- Procesele 1, 2, 3 etc. vor trimite bucata lor din array-ul pos către procesul 0

Rank Sort



Rank Sort

Laboratorul 10
Stabilirea topologiei și alegerea unui lider



Ce este un algoritm undă?

Un algoritm undă este un tip de algoritm distribuit, folosit pentru propagarea informațiilor în cadrul unei rețele distribuite de procese.

Un algoritm undă este caracterizat de următoarele trăsături:

- există un proces inițiator, care declanșează trimiterea de mesaje în cadrul rețelei
- fiecare proces trimite informații către vecinii sai, cu excepția părintelui, de la care a primit informația pe care o pasează mai departe
- fiecare proces primește informații de la vecini (cu excepția părintelui) și o trimite părintelui
- la final se ia o decizie în legătură cu informațiile primite

Tipuri de arhitecturi de rețele distribuite

Avem 4 tipuri de arhitecturi de rețele de noduri (procese) distribuite din cadrul unui cluster:

- grid - fiecare nod poate comunica pe orizontală și pe verticală
- inel - forma de cerc, un nod comunica cu vecinul din stânga și din dreapta
- arbore - informația se propagă de la rădăcina către nodurile-copil
- graf - un nod poate comunica cu cel puțin un vecin de-al său

Stabilirea topologiei

La început, fiecare nod își cunoaște topologia locală (știe doar cine sunt vecinii sai), nu știe despre topologia globală a rețelei de noduri.

Putem sa stabilim topologia globală a rețelei de noduri în două feluri:

- nodurile transmit vecinilor topologia locală de mai multe ori până când toate nodurile au topologia stabilită - numărul de “valuri” de trimitere reprezintă convergența - simplu de realizat, dar ineficient
- se alege un lider (de regulă nodul cu rangul maxim), care calculează la final topologia rețelei

Stabilirea topologiei

- Nodurile își aleg un lider, însă acestea nu știu calea către lider, ci doar cine este acela.
- Pentru a rezolva aceasta problema, se construiește un arbore de acoperire, prin care se afla căile dintre noduri în așa fel încât orice nod poate comunica cu nodul lider, prin intermediul unor noduri intermediare care pasează informația de la nodul sursă la nodul lider: un nod trimite către părintele său, părintele către părintele său, totul se repetă până când se ajunge la nodul rădăcină din arbore, care e lider

Alegerea unui lider

- Rețelele de noduri pot fi ierarhizate (sau nu).
- Un lider poate avea mai multe roluri esentiale în cadrul unei rețele (în cazul de fata: stabilirea topologiei și distribuirea ei către restul nodurilor din rețea)
- Liderul unei rețele poate să fie un singur nod sau un grup de noduri, iar acesta este ales de către rețea pe baza unei euristici / unui algoritm.
- Pentru stabilirea unui lider, folosim algoritmul Heartbeat, prin care alegem ca lider nodul cu rangul cel mai mare.

Algoritmul Heartbeat

- Algoritmul Heartbeat funcționează în două părți:
  - sistolă / expandare - un nod trimite informația mai departe către vecini
  - diastolă / contractie - un nod primește informație de la vecini
- Algoritmul are următorii pași de execuție:
  - nodurile își trimit rangurile către vecini
  - fiecare nod va face maxim dintre valoarea curentă (lider local) și cea primită
  - vecinii își trimit liderul local înapoi către nodurile părinte
  - nodurile ce primesc liderul local al vecinilor își vor actualiza valoarea de lider local

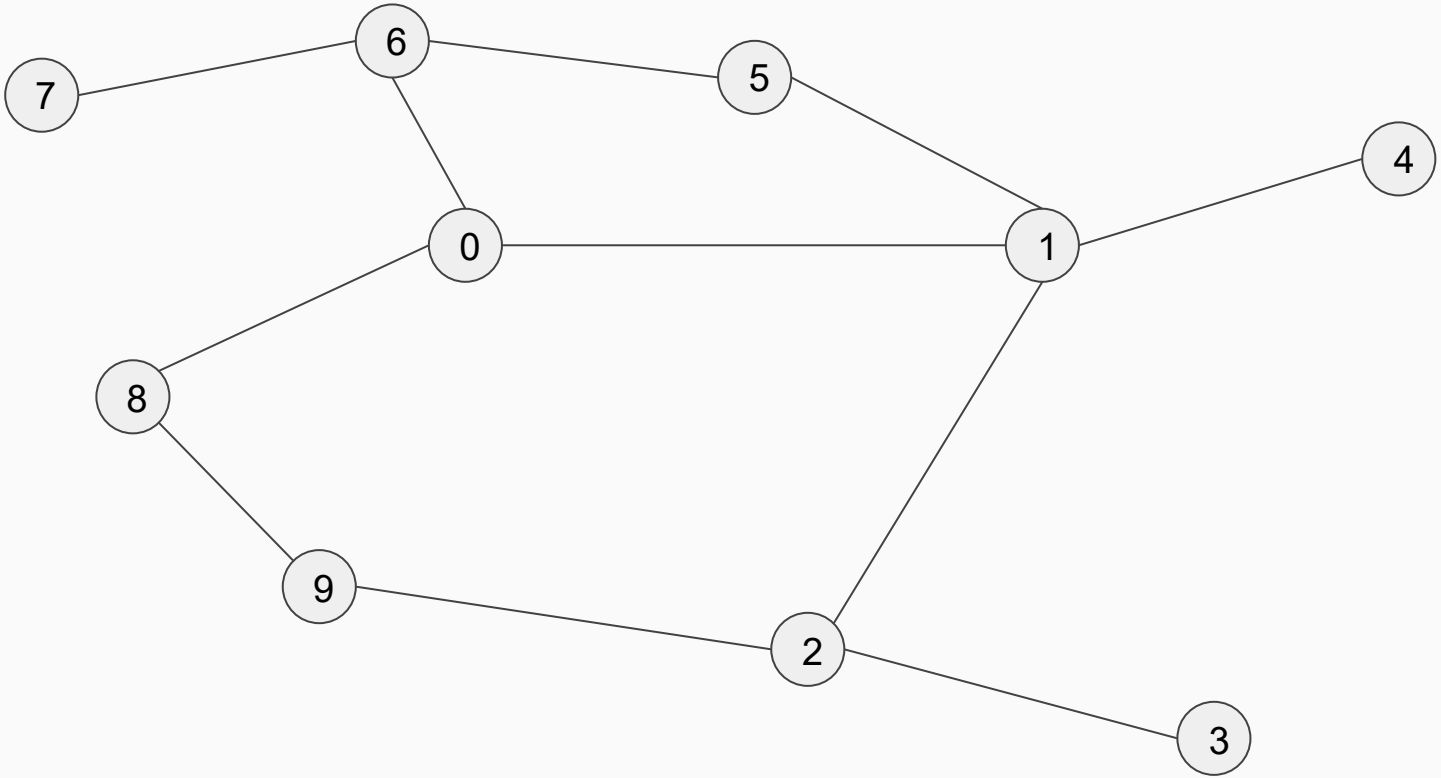
Algoritmul Heartbeat - pseudocod

```
leader = rank; // la inceput, fiecare nod se considera lider

for (pas = 0; pas < convergenta; pas++) {
          for (i = 0 to nr_vecini) {
          int recv_val;
          trimite leader catre procesul vecin[i]
          primeste recv_val de la oricare proces // aici nu are relevanta, ca se
primeste de la orice vecin
          leader = max(recv_val, leader);
     }
}
```

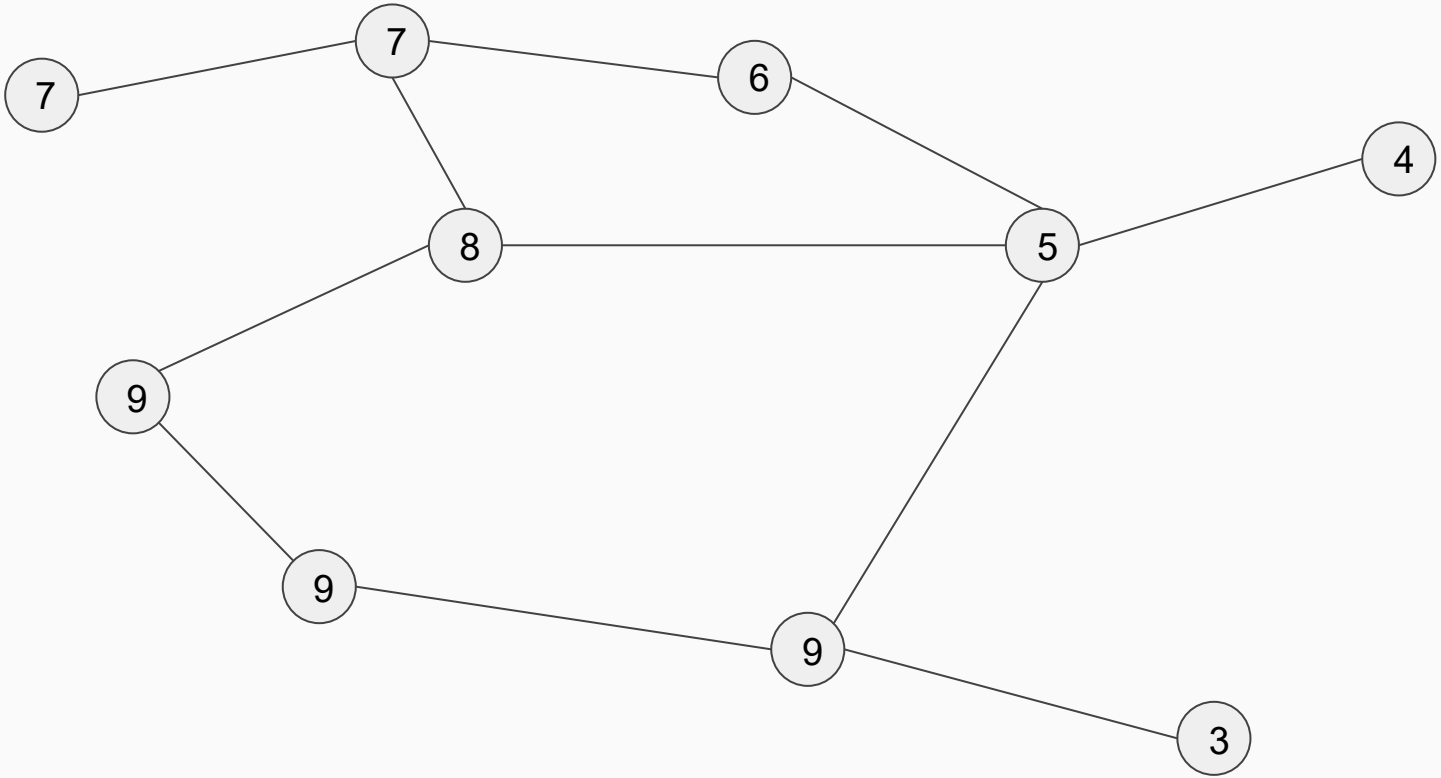
Algoritmul Heartbeat - desfășurare

initial



Algoritmul Heartbeat - desfășurare

pas = 1

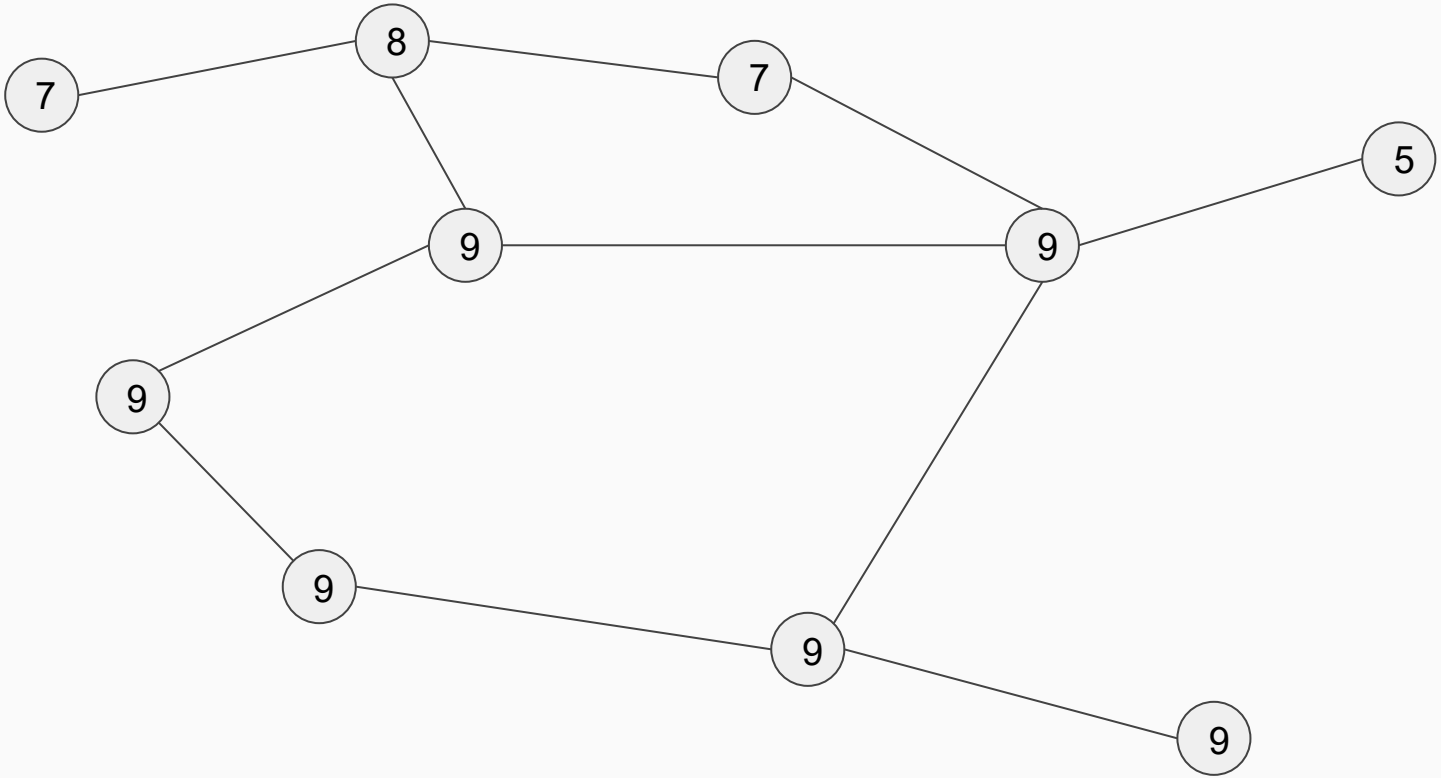


Algoritmul Heartbeat - desfășurare: pas = 1

- 0: max(0, 1, 6, 8) = 8
- 1: max(1, 0, 2, 4, 5) = 5
- 2: max(2, 1, 3, 9) = 9
- 3: max(3, 2) = 3
- 4: max(4, 1) = 4
- 5: max(5, 1, 6) = 6
- 6: max(6, 0, 5, 7) = 7
- 7: max(7, 6) = 7
- 8: max(8, 0, 9) = 9
- 9: max(9, 2, 8) = 9

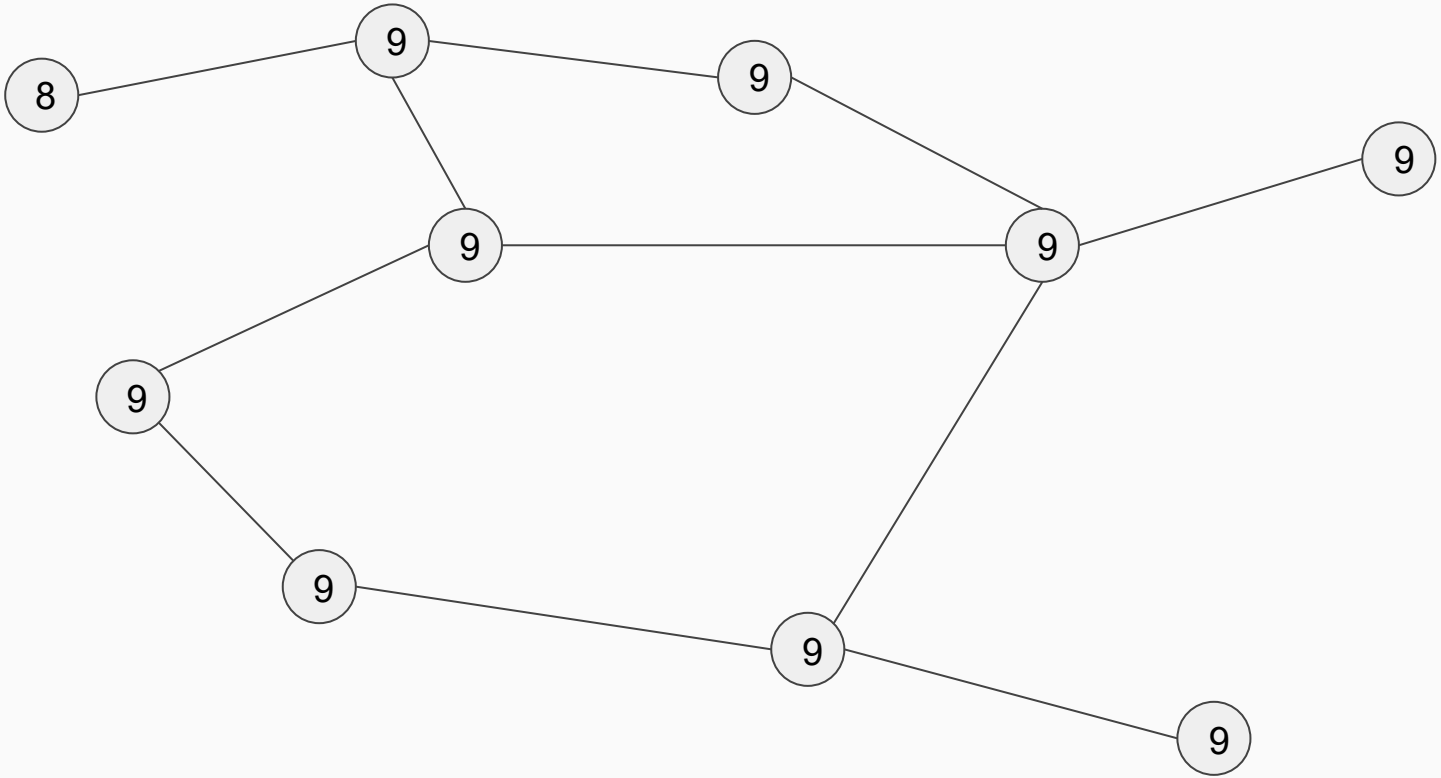
Algoritmul Heartbeat - desfășurare

pas = 2



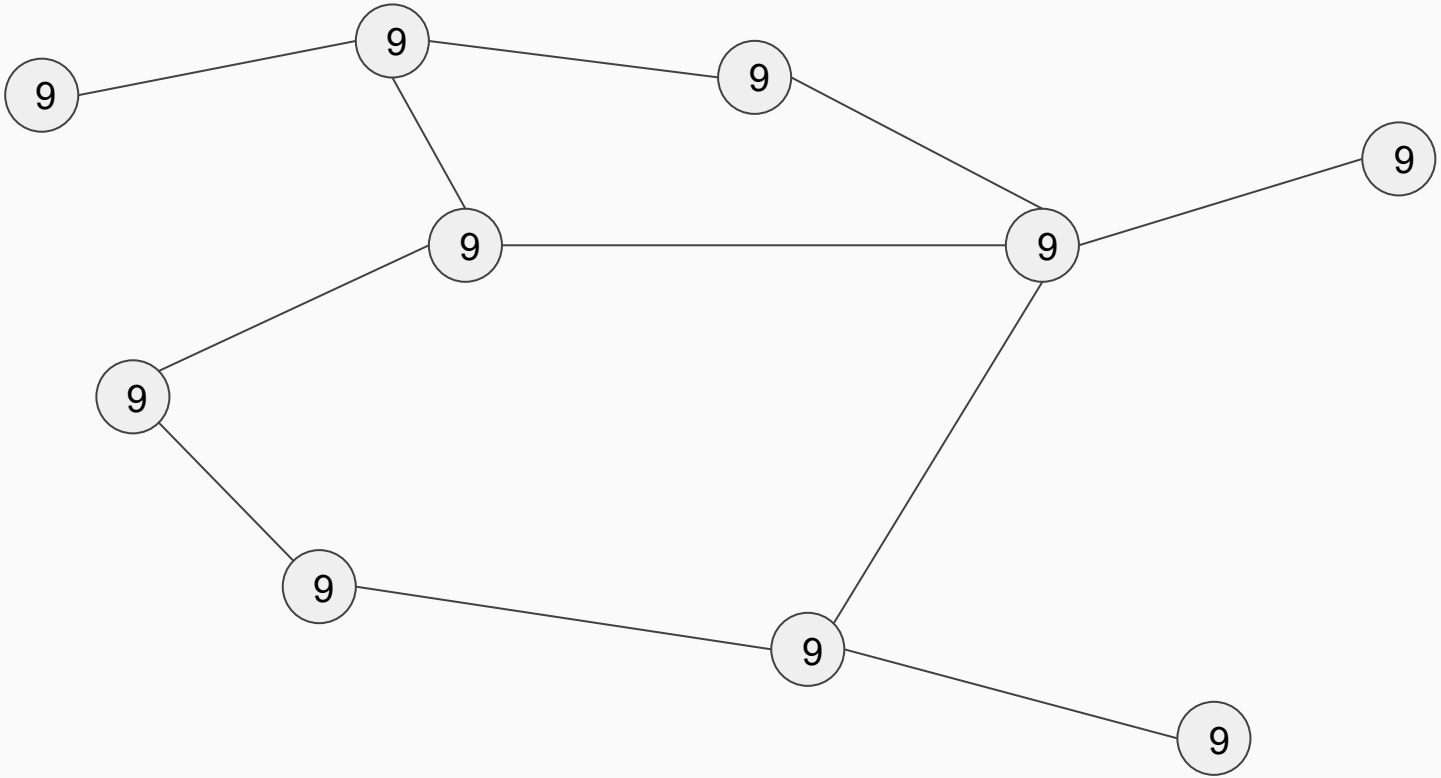
Algoritmul Heartbeat - desfășurare

pas = 3

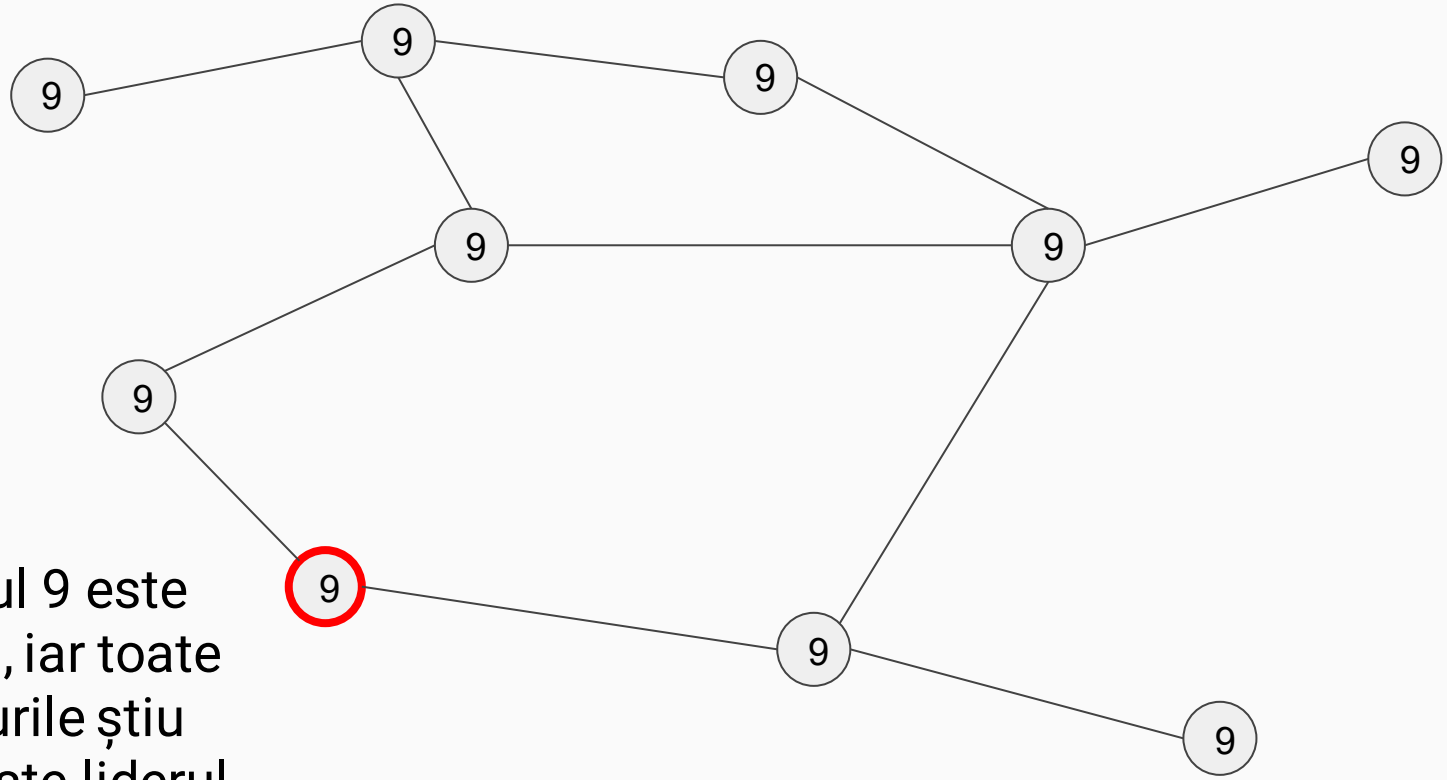


Algoritmul Heartbeat - desfășurare

pas = 4



Algoritmul Heartbeat - desfășurare



Nodul 9 este liderul, iar toate nodurile știu care este liderul

Construirea arborelui de acoperire

- După ce liderul a fost ales, trebuie construit un arbore de acoperire, pentru stabilirea topologiei, prin care fiecare nod să știe cum să ajungă la lider.
- Un arbore de acoperire îl reprezentăm folosind un vector de părinți.
- Pentru construirea unui arbore de acoperire, folosim algoritmul sondă-ecou (undă-ecou).

Algoritmul undă - ecou

- Algoritmul undă-ecou (sondă-ecou) este un algoritm undă care două părți:
  - unda - toate nodurile își trimit informațiile către vecini (cu excepția nodului părinte - liderul nu are părinte, el inițiază trimiterea de mesaje)
  - ecou - nodurile primesc mesaje de la vecini (cu excepția părintelui) și trimit mai departe către părinte. Informațiile primite de la vecini sunt adăugate într-un array de părinți, care e pasat părintelui.
- La final, liderul va obține array-ul de părinți, care va reprezenta arborele de acoperire, care este ulterior distribuit în rețea

Algoritmul undă ecou (sondă ecou) - pseudocod

```
for (i = 0; i < nr_vecini; i++) {
          // vedem daca vecinul curent nu este parintele nodului, caci primim date de la parinte pe care
le pasam celorlalti vecini
          if (vecin[i] != parinte[rank]) {
                    trimite array-ul de parinti (parinte[]) catre vecin[i] // sonda sau unda
     }
}

for (i = 0; i < nr_vecini; i++) {
          // vedem daca vecinul curent nu este parintele nodului
          if (vecin[i] != parinte[rank]) {
                    primeste array-ul de parinti (parinteRecv[]) de la vecin[i] // ecou
                    for (j = 0; j < nr_procese; j++) {
                              // daca nodul / procesul j are parinte (parinte[j] == -1 => nodul j nu
are parinte)
                              // se completeaza array-ul de parinti (bottom - top, de la radacina pana
in varf)
                              if (parinteRecv[j] != -1) {
                                        parinte[j] = parinteRecv[j];
               }
          }
     }
```

Algoritmul undă ecou (sondă ecou) - pseudocod - continuare

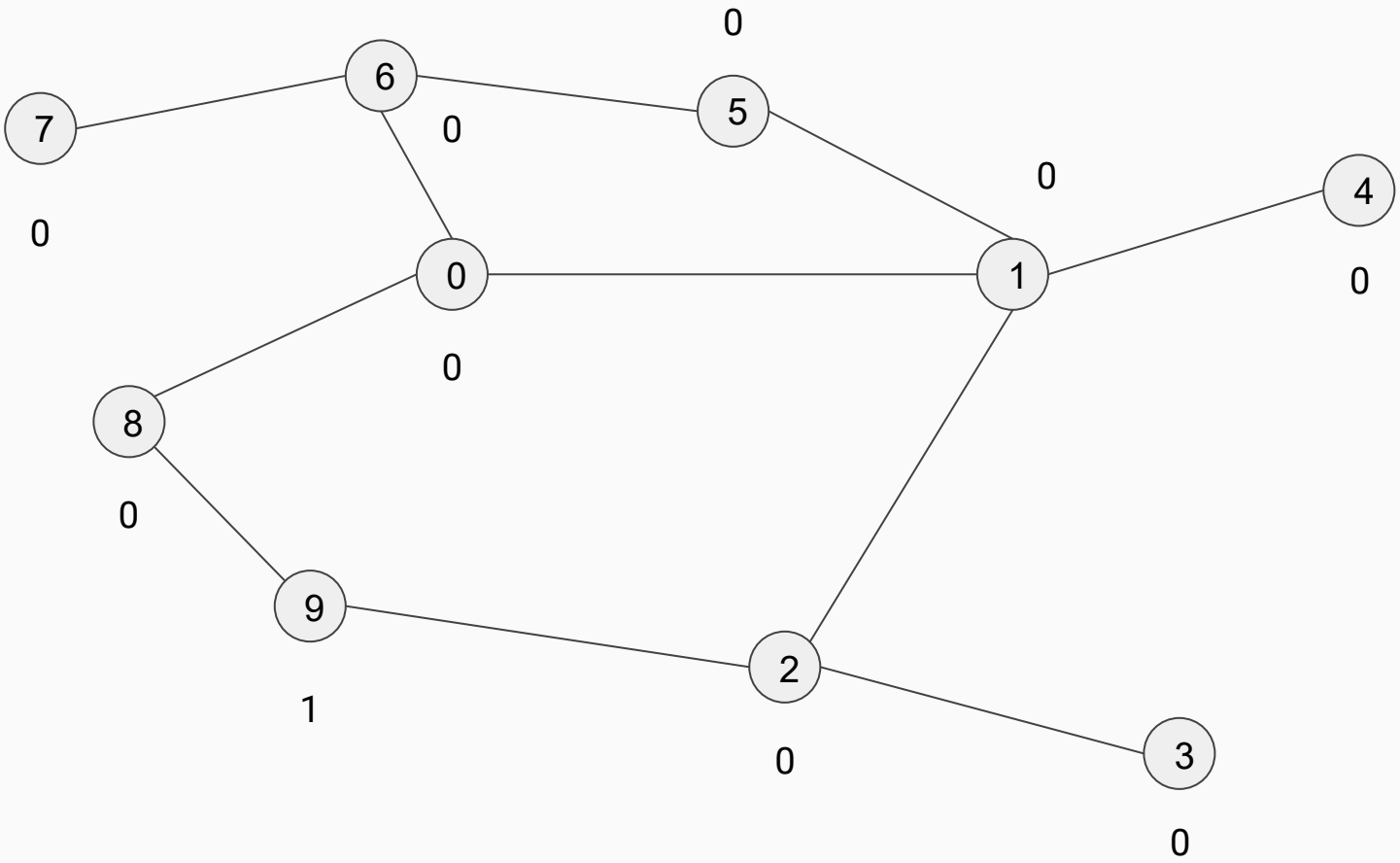
```
// topologia e stabilita (adica nodul curent are topologia formata de nodurile copii pana la nivelul
curent)
if (rank != lider) {
          // fiecare proces care nu e lider trimite topologia curenta catre parintele sau, care o va
completa
          trimite array de parinti (parinti[]) catre parinte[rank] // unda / sonda
          // parintele, dupa ce a completat topologia, o trimite inapoi catre nodul copil
          primeste array de parinit (parinti[]) de la parinte[rank] // ecou
}

// acum topologia este completa
for (i = 0; i < nr_proc; i++) {
          // procesul curent va trimite copiilor sai topologia completata
          if (parinte[i] == rank) {
                    trimite array de parinti (parinti[]) catre procesul / nodul i
     }
}
```

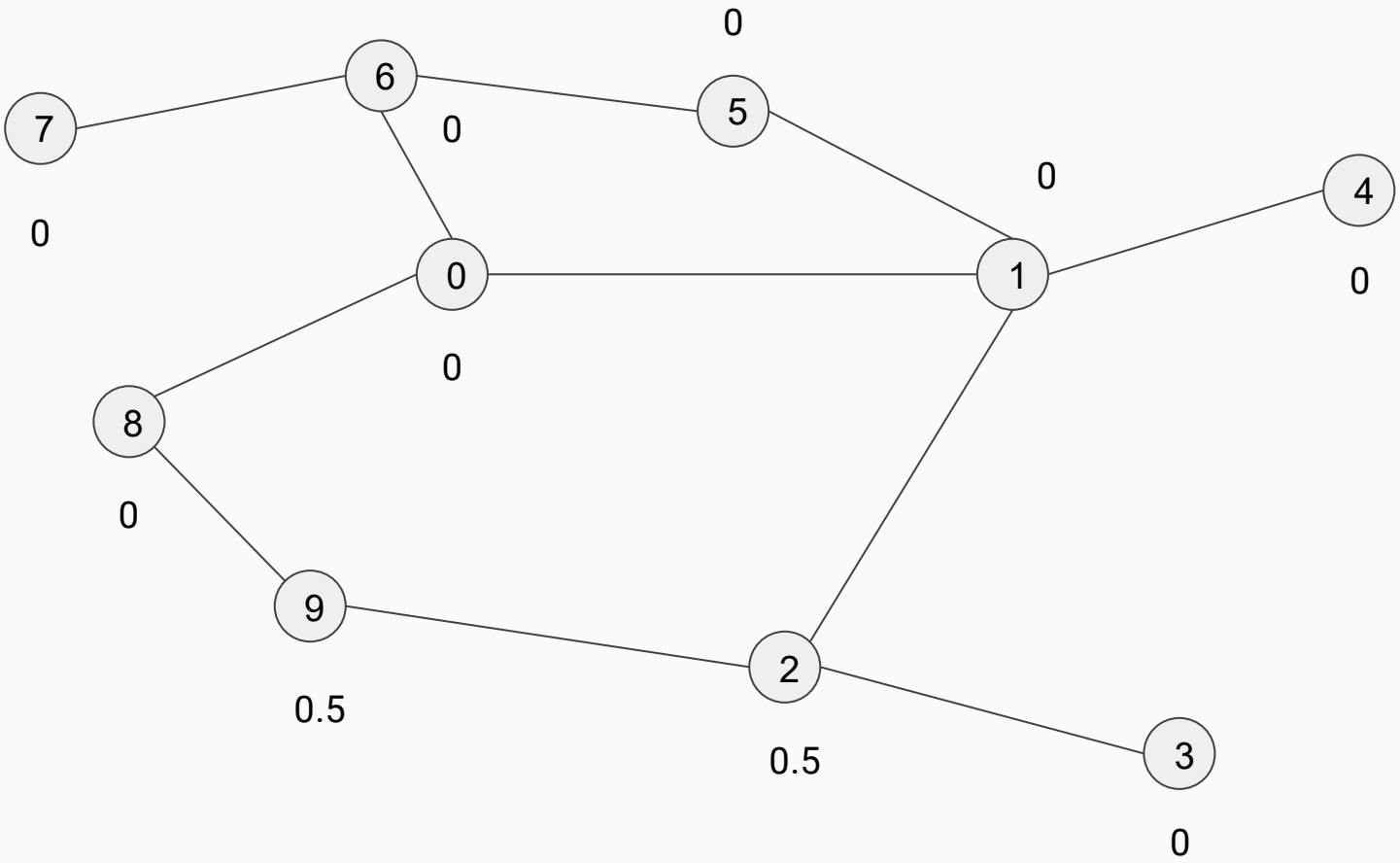
Numărarea nodurilor din topologie

- După ce am ales liderul și am distribuit topologia rețelei către toate nodurile, este nevoie să validăm această informație. În acest caz, validarea se face prin numărarea de noduri prezente în rețea, folosind un algoritm de tip epidemic.
- Algoritmul epidemic funcționează în felul următor:
  - nodul lider (pacient zero) își trimite informația către vecini
  - vecinii nodului lider primesc informația (se “infectează”) și trimit informația mai departe către vecini
  - un nod care o valoare val, iar cand primește o valoare recv_val de la un vecin, nodul curent își actualizează valoarea val în felul următor: val = (val + recv_val) / 2
  - numărul de noduri = 1 / val

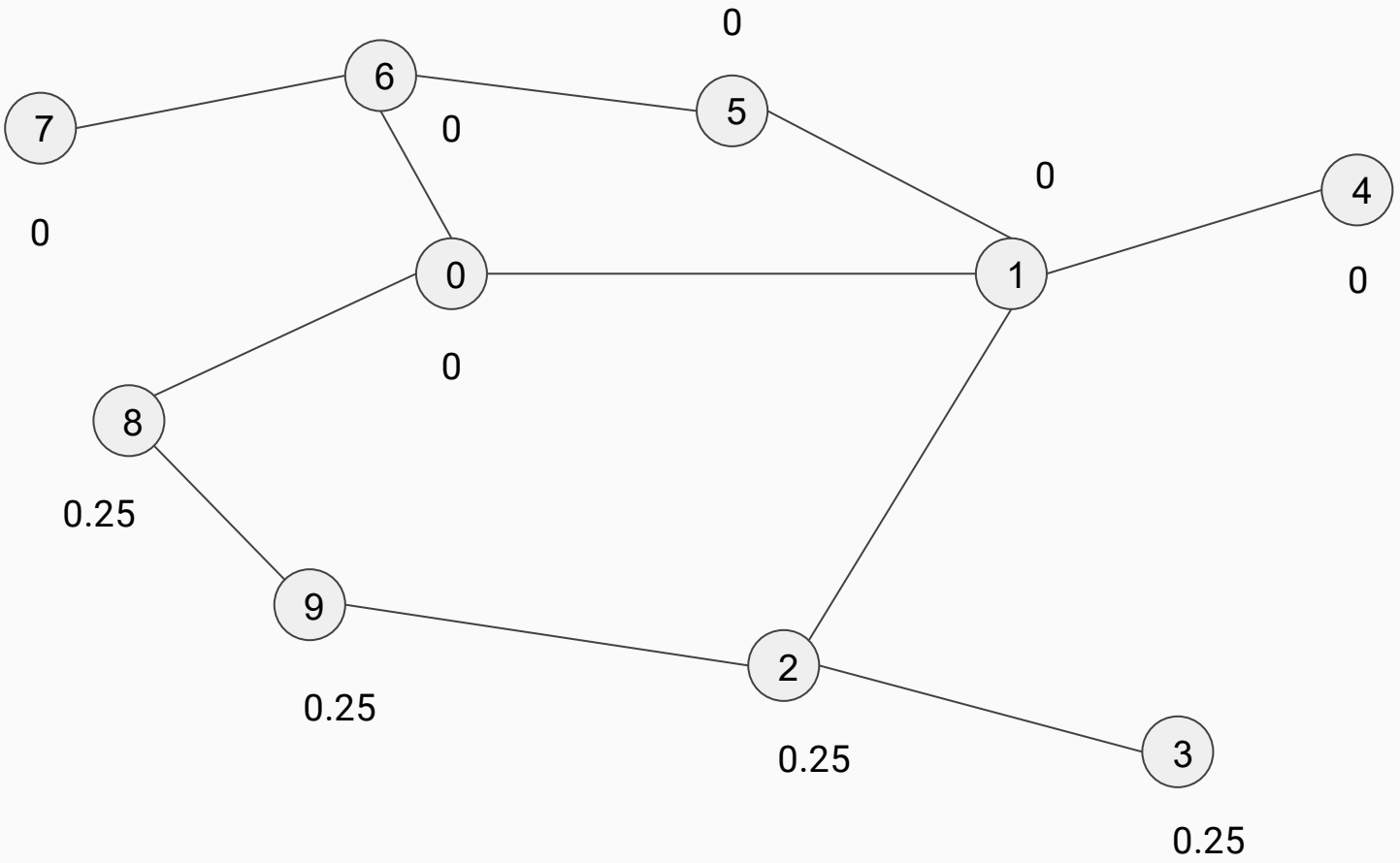
Numărarea nodurilor din topologie



Numărarea nodurilor din topologie



Numărarea nodurilor din topologie



Numărarea nodurilor din topologie - pseudocod

```
double val = 0

if (rank == leader)
          val = 1
else
          val = 0

for (pas = 0; pas < convergenta; pas++) {
          for (i = 0 to nr_vecini) {
          double recv_val;
          trimite val catre procesul vecin[i]
          primeste recv_val de la vecin[i]
          val = (val + recv_val) / 2;
     }
}

return (round) (1 / val); // numarul de noduri din rețea
```

Matricea de topologie a rețelei de noduri

- Folosindu-ne de array-ul de părinți, putem construi o matrice N x N (N fiind numărul de procese) care să arate, în ansamblu, topologia întreagă a rețelei de noduri, folosindu-ne de algoritmul sondă-ecou.
- Pașii sunt următorii:
  - fiecare nod primește topologia copiilor săi (mai puțin de la părinte), rând cu rând, și își actualizează matricea de topologie
  - fiecare nod își trimite matricea de topologie către părinte
  - fiecare nod (cu excepția liderului) primește înapoi matricea completată de părinte
  - fiecare nod trimite topologia completată copiilor săi

Matricea de topologie a rețelei de noduri - pseudocod

```
// se primesc informatii de la toti copiii procesului / nodului curent si se actualizeaza
matricea de topologie
for (i = 0 to nr_proc) {
        // pentru copiii procesului / nodului curent
        if (rank == parents[i]) {
                for (j = 0 to nr_proc) {
                        // se primeste, linie cu linie, topologia locala procesului /
nodului i
                        se primeste recvTopology[j] de la nodul i
                        for (k = 0 to nr_proc) {
                if (top[j][k] == 0) {
                        // actualizam matricea de topologie
                        top[j][k] = recvTopology[j][k];
                }
            }
        }
    }
}
```

Matricea de topologie a rețelei de noduri - pseudocod

```
// nodul curent trimite matricea de topologie proprie catre parinte, rand cu rand
for (i = 0 to nr_proc) {
            // daca procesul curent are parinte
            if (parents[rank] != -1) {
                        se trimite top[i] catre parents[rank]
      }
}

// daca nodul curent nu e lider, primeste topologia, rand cu rand, de la parinte
if (rank != leader) {
            // nodul lider nu are parinte
      for (i = 0 to nr_proc) {
            se primeste top[i] de la parents[rank]
      }
}

// se trimite topologia completa catre nodurile-copii, linie cu linie
for (i = 0 to nr_vecini) {
            for (j = 0 to nr_proc) {
                        trimite top[j] catre vecin[i]
      }
}
```

Alegerea liderului în topologii inel



Algoritmul LeLann

- Algoritmul LeLann este un algoritm undă pentru alegerea unui lider în cadrul unei rețele de tip inel de noduri.
- Acest algoritm se desfășoară în felul următor:
  - fiecare nod are o listă cu identificatorii pe care i-a primit
  - fiecare nod își trimite rangul către celelalte procese și colectează rangurile celorlaltor procese în lista de identificatori
  - fiecare nod primește valoarea de la nodul precedent și o pasează nodului următor
  - când nodul își primește rangul de la nodul precedent, se consideră că nodul curent are toate rangurile nodurilor din rețea în lista de identificatori
  - la final va face maximul din lista respectivă: dacă rangul său este egal cu maximul, atunci este nod lider

Algoritmul LeLann - pseudocod

```
enum STATE {CANDIDATE, LEADER, LOST}

process P[rank] {
            list = []
            list.append(rank)
            state = STATE.CANDIDATE

            int recv_val, val = rank
            trimite val către procesul următor
            primește recv_val de la procesul precedent
            val = recv_val

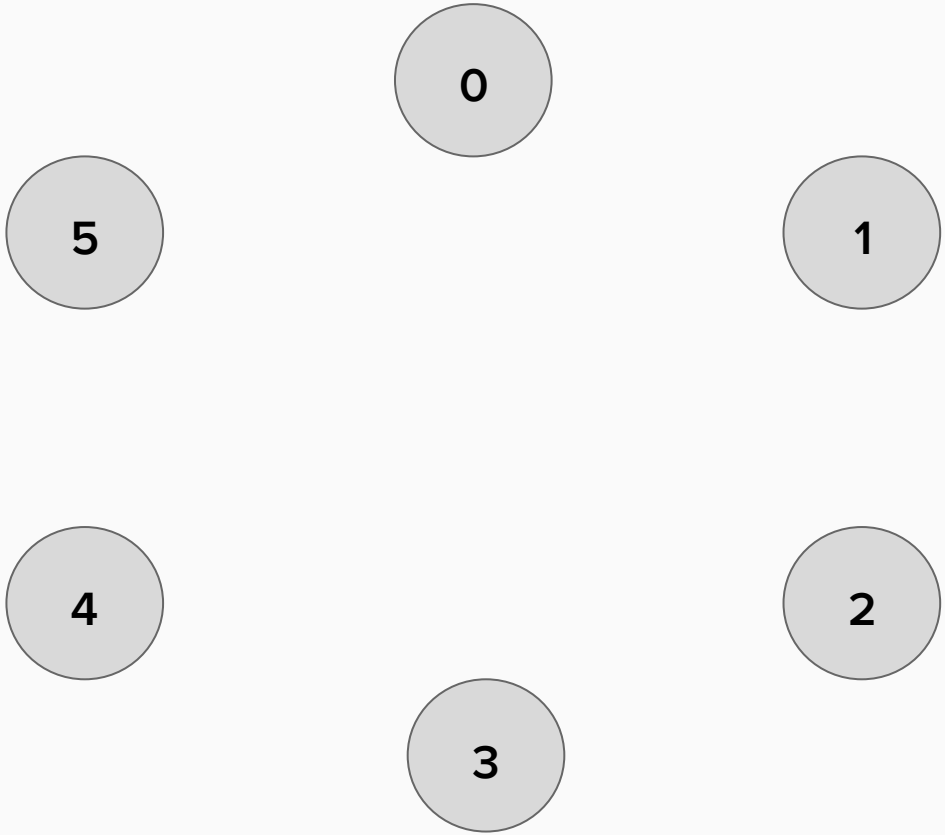
            while (recv_val != rank) {
                        list.append(recv_val)
                        trimite val către procesul următor
                        primește recv_val de la procesul precedent
                        recv_val = val
      }

      if (rank == max(list))
            state = STATE.LEADER
      else
            state = STATE.LOST

}
```

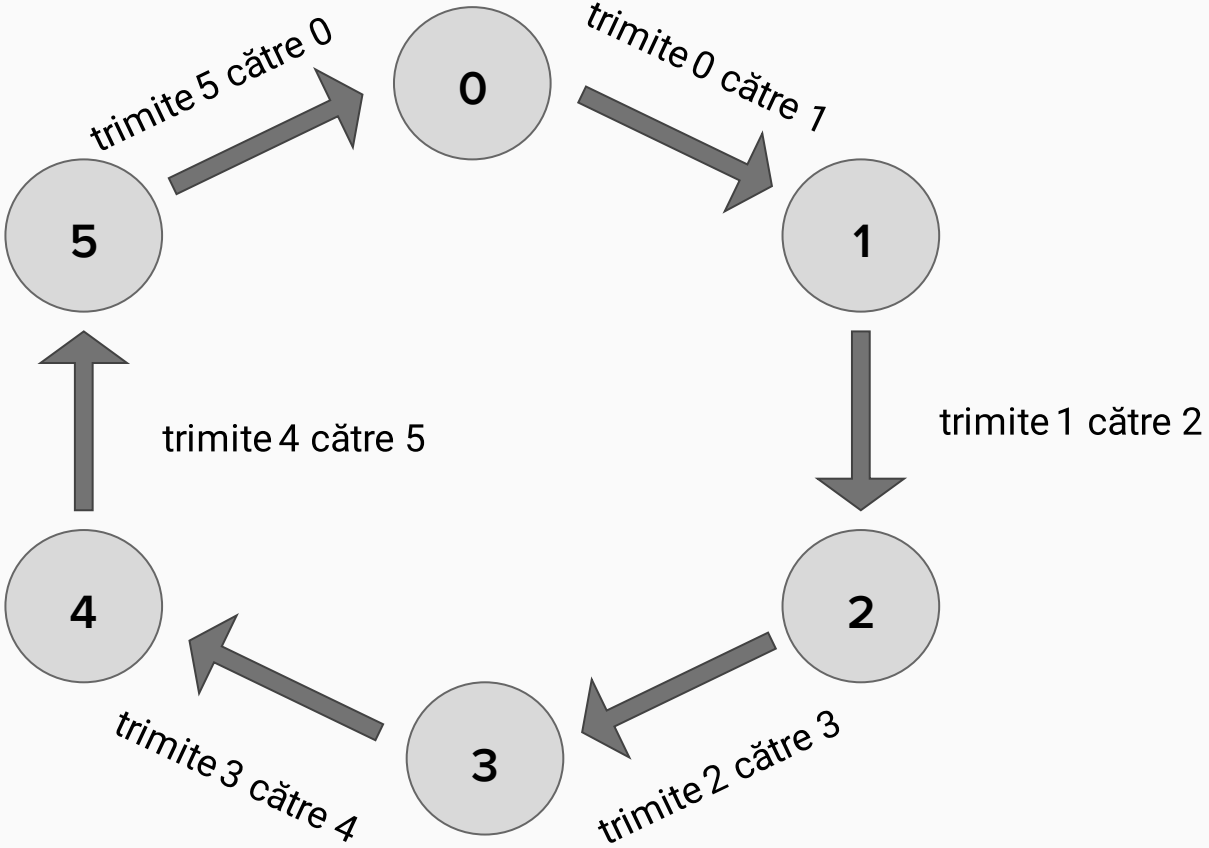
Algoritmul LeLann

0: [0]
1: [1]
2: [2]
3: [3]
4: [4]
5: [5]



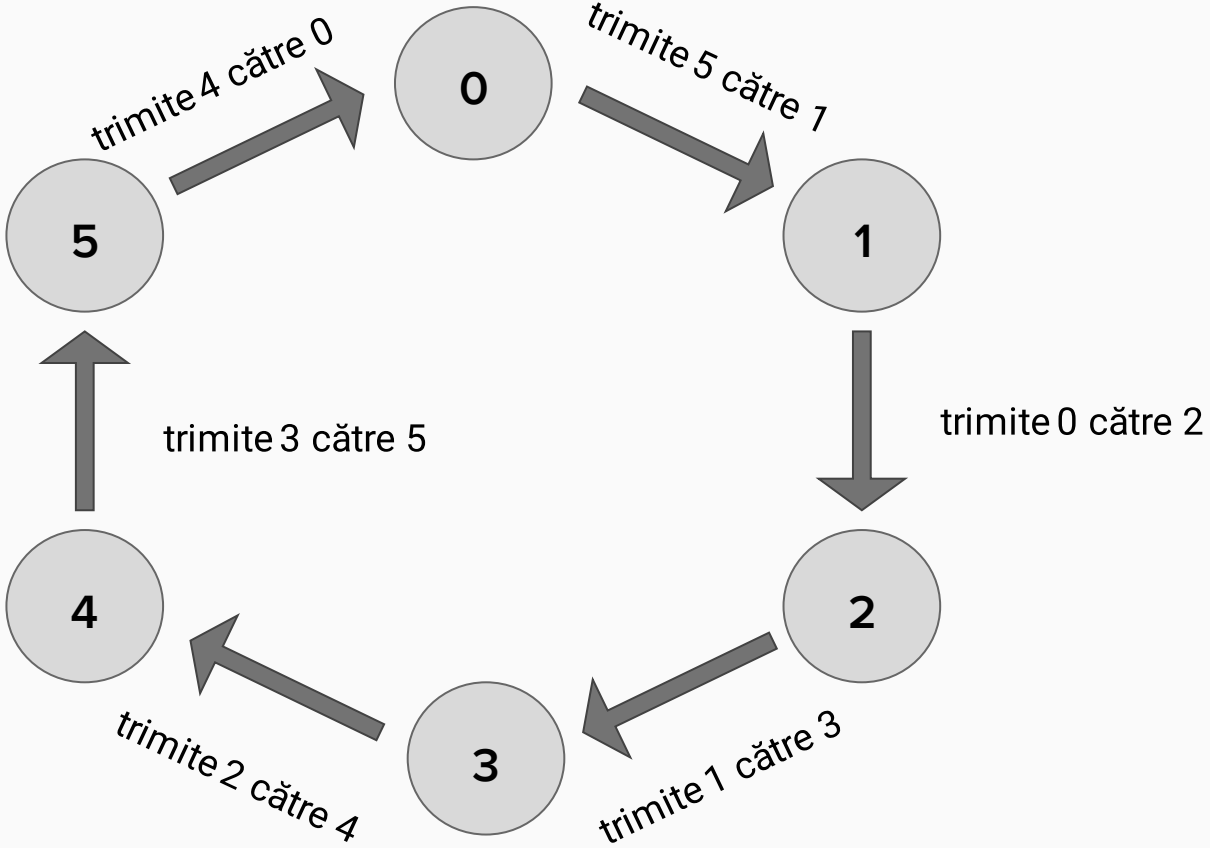
Algoritmul LeLann

0: [0, 5]
1: [1, 0]
2: [2, 1]
3: [3, 2]
4: [4, 3]
5: [5, 4]



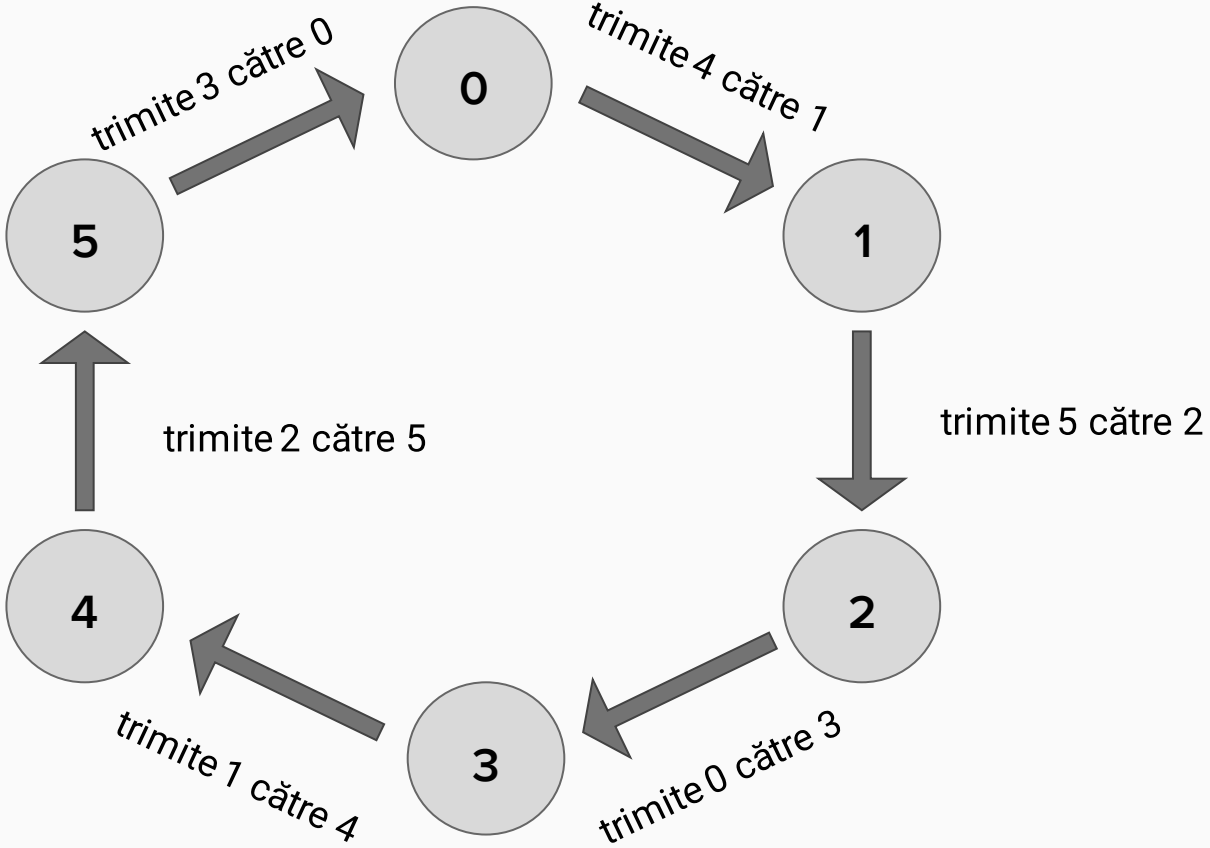
Algoritmul LeLann

0: [0, 5, 4]
1: [1, 0, 5]
2: [2, 1, 0]
3: [3, 2, 1]
4: [4, 3, 2]
5: [5, 4, 3]



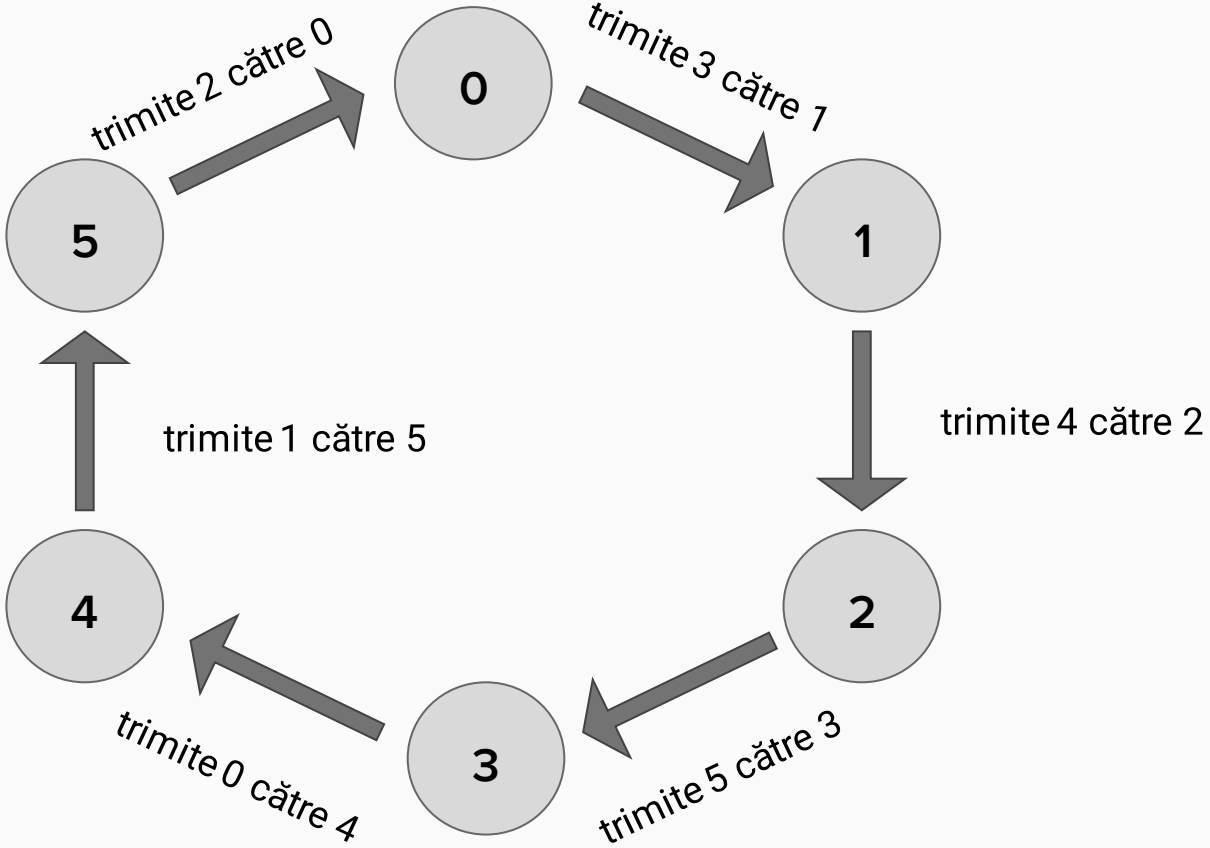
Algoritmul LeLann

0: [0, 5, 4, 3]
1: [1, 0, 5, 4]
2: [2, 1, 0, 5]
3: [3, 2, 1, 0]
4: [4, 3, 2, 1]
5: [5, 4, 3, 2]



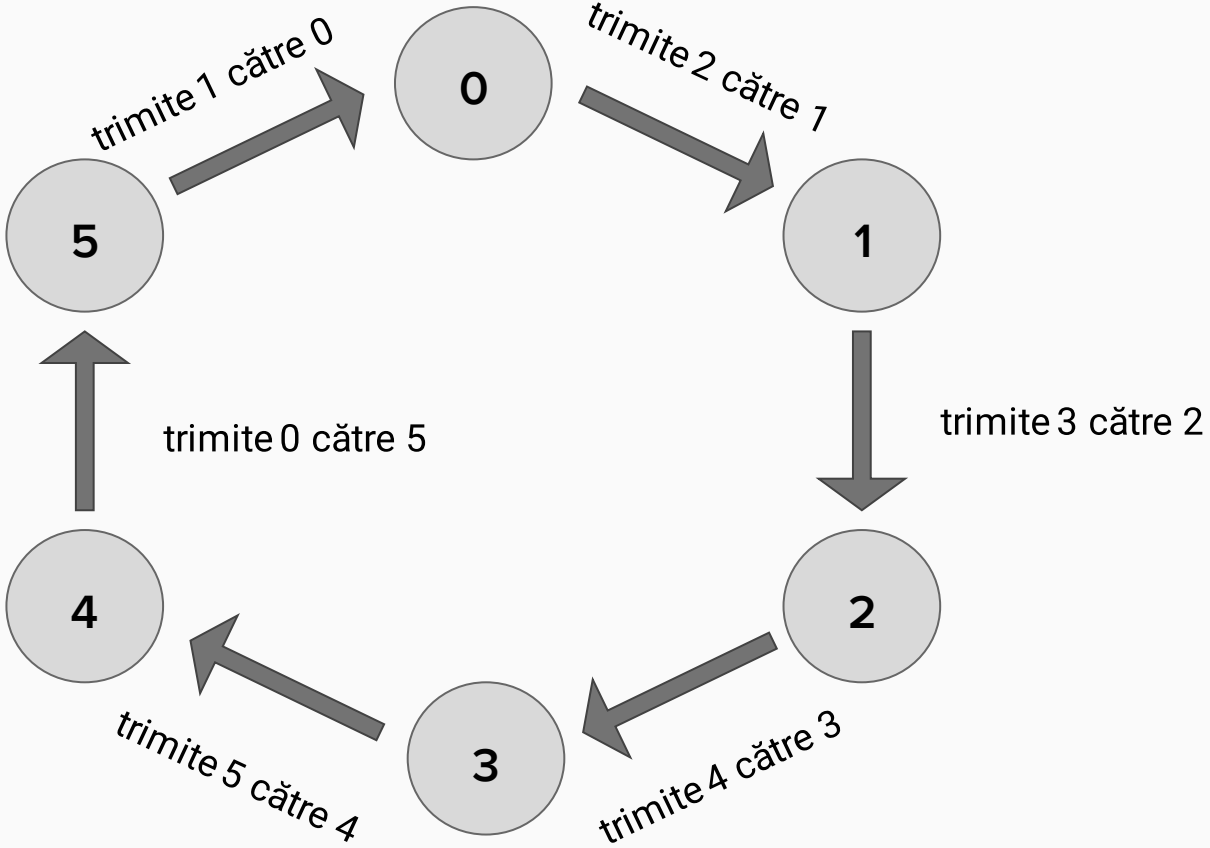
Algoritmul LeLann

0: [0, 5, 4, 3, 2]
1: [1, 0, 5, 4, 3]
2: [2, 1, 0, 5, 4]
3: [3, 2, 1, 0, 5]
4: [4, 3, 2, 1, 0]
5: [5, 4, 3, 2, 1]



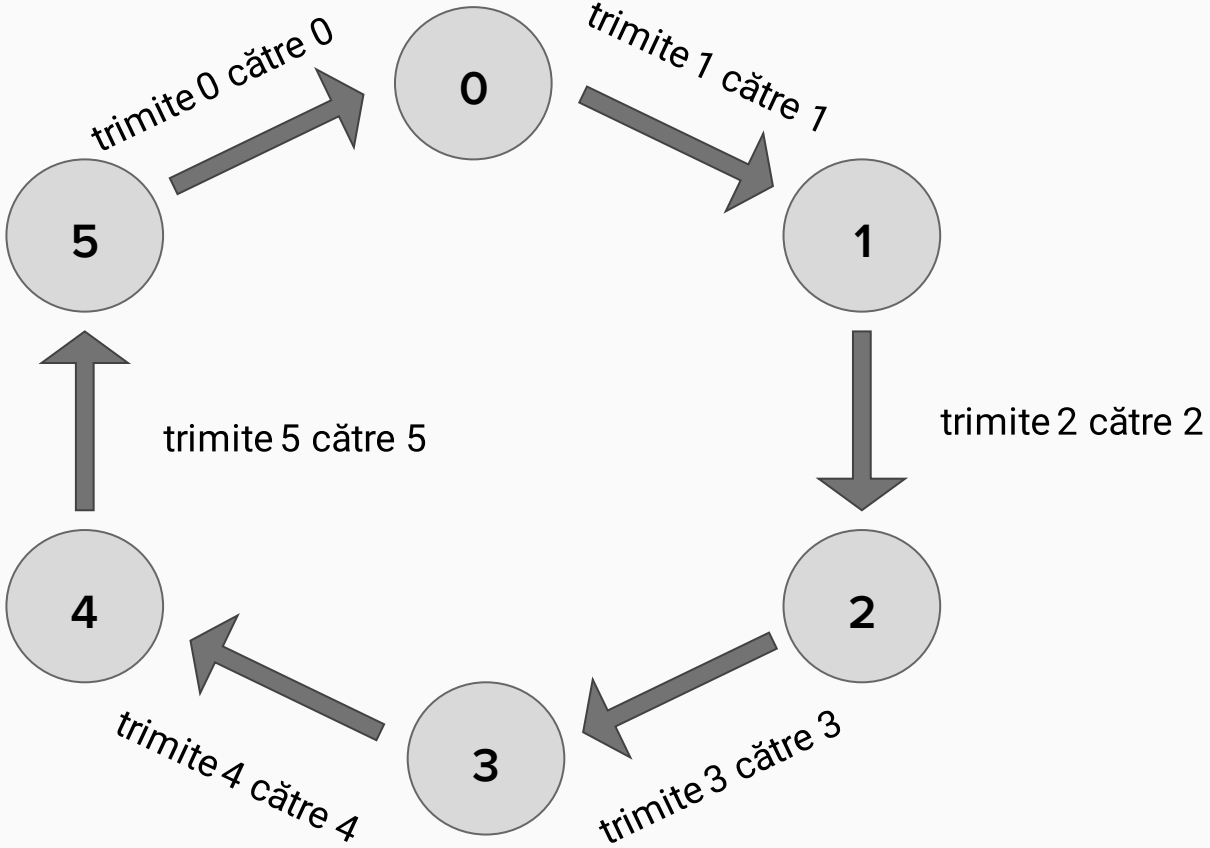
Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]



Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]

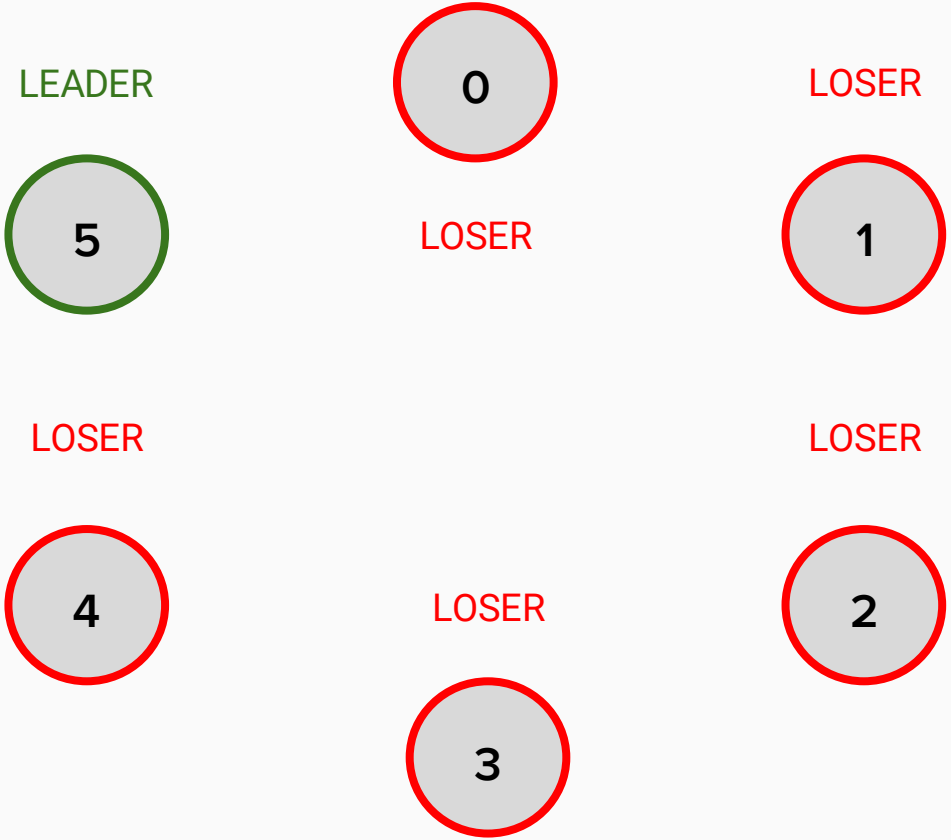


În acest moment, fiecare proces își primește propriul său rang -> **STOP**

Nodul 5 devine lider

Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]



Algoritmul LeLann-Chang-Roberts

- Algoritmul LeLann-Chang-Roberts este un algoritm undă pentru alegerea unui lider în cadrul unei rețele de tip inel de noduri.
- Fiecare proces își trimite rangul mai departe, către celelalte procese și primește rangurile celorlaltor procese, pe care le pasează către vecini, ca la LeLann.
- Diferența este următoarea, la primirea de mesaje:
  - dacă un proces își primește rangul, acesta devine lider
  - dacă un proces primește un id mai mare decât rangul său, acesta este un nod loser, care pasează către următorul proces ce primește de la procesul precedent
  - dacă un proces primește un id mai mic decât rangul său, ignoră mesajul primit

Algoritmul LeLann-Chang-Roberts - pseudocod

```
enum STATE {CANDIDATE, LEADER, LOST}

process P[rank] {
            state = STATE.CANDIDATE

            int recv_val, val = rank
            trimite val către procesul următor

            while (state != STATE.LEADER) {
                        primește recv_val de la procesul precedent
                        if (recv_val == rank) {
                        state = STATE.LEADER;
            } else if (recv_val > rank) {
                        if (state == CANDIDATE) {
                                    state = STATE.LOST;
                                    val = recv_val;
                                    trimite val către procesul următor
                        }
            }
      }
}
```

Algoritmul LeLann-Chang-Roberts

0: [0]
1: [1]
2: [2]
3: [3]
4: [4]
5: [5]

